CX and ITSM Platform: Project, Architecture and Workflow

Multi-Tenant Customer Experience and IT Service Management SaaS with AI-Assisted Support

Portfolio project inspired by the public job description for a Lead Full Stack Engineer (IC3). It does not reproduce, and makes no claim about, any proprietary software, architecture or production system of any company.

Honesty notice: all test, benchmark and measurement figures in this document come from commands actually executed on a single developer laptop (local profile: H2, in-memory cache and relay). Components that need Docker (PostgreSQL, Redis, RabbitMQ, MongoDB, Grafana, k6, Testcontainers) were not available and are labelled "not executed" wherever they appear.

The AI assistant is a deterministic, rule-based mock provider. It is not machine learning, and a real provider is optional and disabled by default.

Generated 2026-10-17

Contents

1. Executive summary **5**
- 1.1 What this project is 5
- 1.2 What was built 5
- 1.3 What was verified 6
- 1.4 Five ideas that explain the design 7
- 1.5 How to read this document 7

2. Business use cases and personas **8**
- 2.1 The business problem 8
- 2.2 Personas and roles 8
- 2.3 Core use cases 8
- 2.4 Why multi-tenancy is the central concern 9

3. Functional and non-functional requirements **10**
- 3.1 Functional requirements 10
- 3.2 Non-functional requirements 10
- 3.3 Assumptions 11
- 3.4 Out of scope 11

4. System architecture **12**
- 4.1 System context 12
- 4.2 Service architecture 12
- 4.3 Runtime profiles 13
- 4.4 A request, end to end 15

5. Service responsibility matrix **16**
- 5.1 Deployable units 16
- 5.2 Responsibilities inside core-service 16
- 5.3 Who may call what 17

6. Frontend architecture **18**
- 6.1 Stack and structure 18
- 6.2 Component tree 18
- 6.3 Routing and guards 19
- 6.4 State management 20
- 6.5 Handling the API contract in the UI 20
- 6.6 Accessibility, performance and testing 21

7. Backend architecture **22**
- 7.1 Modules and layering 22
- 7.2 Package dependencies 22
- 7.3 Cross-cutting mechanisms 23
- 7.4 Configuration 24
- 7.5 Gateway design 24

8. Tenant isolation architecture **25**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8.1 Model: shared database, shared schema, discriminator column 25
8.2 Defence in depth 25
8.3 The pooled-connection detail 27
8.4 Rules enforced by the build 27
8.5 Observable behaviour 27
8.6 Known weaknesses 28
9. Data model 29
9.1 Overview 29
9.2 Core tables 29
9.3 Knowledge base, AI and notification tables 31
9.4 PostgreSQL-only additions (not executed) 33
9.5 Data ownership and lifecycle 33
10. Ticket lifecycle and assignment 34
10.1 Status model 34
10.2 What a transition does 36
10.3 Creating a ticket 36
10.4 Assignment and escalation 37
10.5 Comments, first response and activity 38
11. SLA and escalation 39
11.1 Policies 39
11.2 Business-hours arithmetic 39
11.3 Worked example 39
11.4 Status values and the scan job 40
11.5 Limits of the design 42
12. Knowledge base 43
12.1 Purpose and rules 43
12.2 Search flow 43
12.3 Storage split 45
12.4 Document store implementations 45
12.5 Seed data 46
13. AI-assisted support and human approval 47
13.1 What the feature is, and is not 47
13.2 Analyze and decide 47
13.3 Suggestion states and the approval gate 48
13.4 Resilience 49
13.5 Safety model 51
13.6 Persistence, audit and metrics 52
13.7 Evaluation 52
14. Messaging and the transactional outbox 53

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

14.1 The problem 53
14.2 Outbox and relay 53
14.3 Event catalogue 55
14.4 Topology and consumer 55
14.5 Idempotent processing 56
14.6 Local mode: the embedded module 56
15. Caching 57
15.1 Design 57
15.2 What is cached 59
15.3 Measured effect and caveats 59
15.4 Trade-offs 59
16. Authentication and authorization flow 60
16.1 Four different questions 60
16.2 Login and request flow 60
16.3 Authorization layers 61
16.4 Logout and revocation 61
16.5 Known gaps 61
17. Analytics and reporting 62
17.1 The dashboard contract 62
17.2 How each number is defined 62
17.3 Live versus precomputed 62
17.4 Reporting beyond the dashboard 63
18. Local infrastructure and observability 64
18.1 What runs locally 64
18.2 The Docker Compose stack 64
18.3 Observability design 65
18.4 Testing and measurement tools 66
18.5 What would change before sharing the stack 66
19. End-to-end business scenarios 67
19.1 Scenario A: a customer reports a security issue and double-clicks submit 67
19.2 Scenario B: two agents race to move the same ticket 68
19.3 Scenario C: AI-assisted reply with human approval 68
19.4 Scenario D: a ticket breaches its SLA over a weekend 69
19.5 Scenario E: a tenant administrator changes the workflow 69
19.6 Scenario F: another tenant probes for data 69
19.7 Scenario G: a customer finds an answer in the knowledge base 70
20. Failure and recovery workflows 71
20.1 Failure catalogue 71
20.2 Recovery procedures 71

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

20.3 What the tests prove about failure 72
21. Performance bottlenecks and optimisation decisions 73
21.1 Baseline load runs 73
21.2 The ticket-create tail: hypothesis, experiment, fix 73
21.3 Cache hit versus miss 74
21.4 Query shape evidence (H2 only) 74
21.5 N+1 evidence 75
21.6 Front-end bundle 75
21.7 Decisions and their price 75
21.8 Not measured 76
22. Architecture decision records: summary 77
23. Engineering roadmap and technical debt 79
23.1 Honest gaps 79
23.2 Suggested roadmap 80
23.3 What is solid 81
24. Interview questions and detailed answers about the project 82
24.1 Architecture and design 82
24.2 Data, concurrency and APIs 83
24.3 Messaging and caching 84
24.4 SLA, workflow and the lifecycle 85
24.5 AI, security and operations 86

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. Executive summary

1.1 What this project is

This document describes a multi-tenant Customer Experience (CX) and IT Service Management (ITSM) platform. It is a self-built portfolio project, written to demonstrate the engineering themes of a publicly posted full-stack lead role: multi-tenant SaaS design, REST APIs, caching, messaging, a React and TypeScript front end, an SLA engine, a knowledge base, AI-assisted support with human approval, observability and honest testing. It is inspired by a public job description. It is not Freshworks software, it does not describe how Freshworks is built, and all data in it is synthetic.

The platform lets several organisations (tenants) share one deployment. Each tenant has customers who raise support or IT-service tickets, agents who work them, managers who watch queues and service levels, and administrators who configure users, workflows, SLA policies and categories. A knowledge base and a rule-based suggestion engine help agents answer faster, and every suggested reply must be approved by a human before a customer can see it.

> **Honesty statement.** No Docker was available on the authoring machine. PostgreSQL, Redis, RabbitMQ, MongoDB, PostgreSQL row-level security (RLS), the Docker Compose stack, Prometheus and Grafana were therefore never executed. Everything marked "executed" in this document ran on the `local` runtime profile: H2 in PostgreSQL mode, a Caffeine in-process cache, and an in-JVM event dispatcher. The AI feature is a deterministic keyword rule engine, not machine learning. Where this document says "designed" or "written", the code exists but did not run.

1.2 What was built

The repository contains five deployable or library modules and a front end:

- **core-service** (Spring Boot 3.3.5, Java 21, port 8081): identity and tenancy, tickets, workflow, SLA, knowledge base, AI assist, analytics, audit, caching, and the transactional outbox.
- **notification-service** (port 8082 when standalone): an event consumer that creates in-app notifications and simulated email deliveries, with retry and dead-letter handling. In the local profile it is embedded inside core-service as a library.
- **gateway** (port 8080): a small hand-written WebFlux reverse proxy that adds a correlation id, an edge JWT check, a token-bucket rate limit and proxying with timeouts.
- **platform-events**: the shared event envelope (`DomainEvent`) and the `EventListener` contract.
- **frontend**: React 18, TypeScript (strict), Vite, TanStack Query, React Hook Form and Zod.
- **frontend-angular**: a second, feature-equivalent UI (Angular 20, standalone components, signals, typed reactive forms) against the same API.
- **infra**: a Docker Compose stack, Dockerfiles, Prometheus and Grafana provisioning, all unexecuted.
- **dsa-lab**: a data-structures practice library that is educational and not on any request path.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

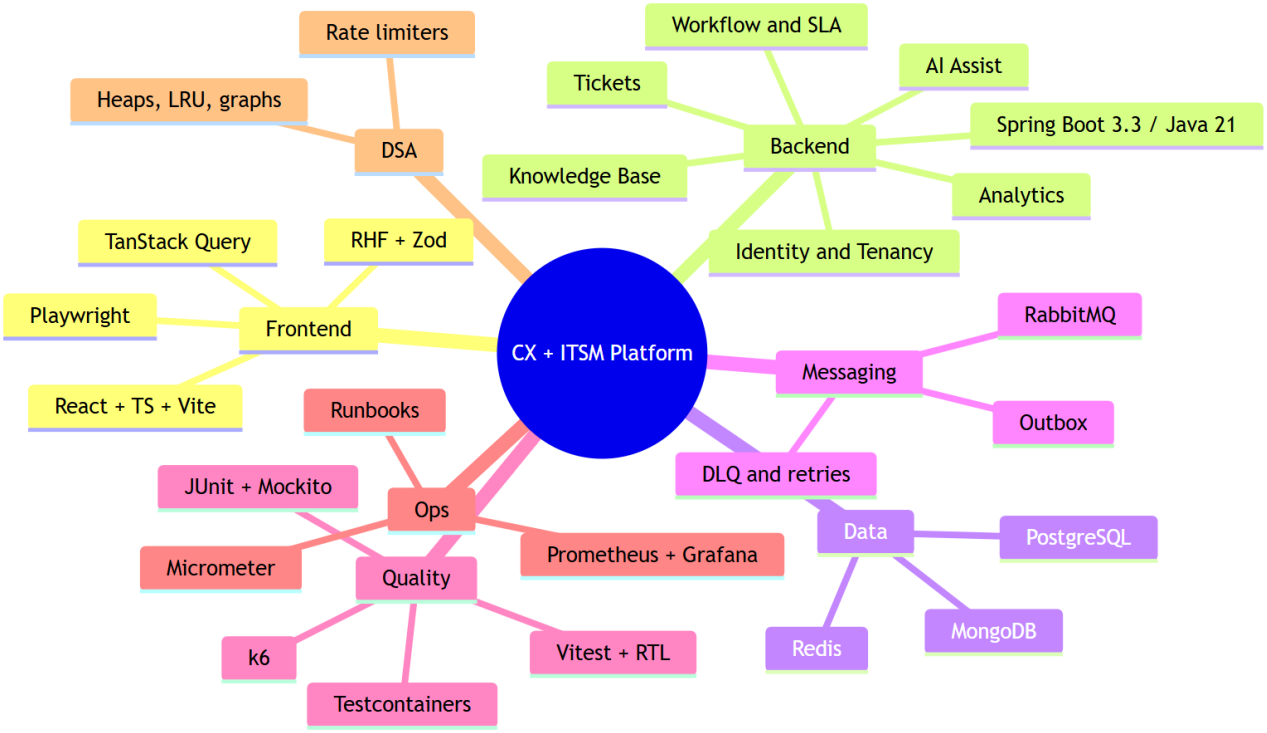


Figure 1: Project mind map showing the technology areas covered by the platform

Figure 1 is a map of the topics the project touches, grouped into frontend, backend, data, messaging, quality, operations and data-structures practice. Use it as a table of contents for the rest of the document: every branch corresponds to a chapter that explains the real implementation and its limits. The testing branch lists Testcontainers and k6; both exist as code but did not run, which later chapters call out explicitly.

1.3 What was verified

The following facts come from the recorded validation and performance documents, all executed on 2026-10-17 on one Windows laptop with GraalVM JDK 21.0.7 and no Docker:

Evidence	Result
core-service automated tests (JUnit 5, MockMvc, H2)	179 tests, 0 failures; 7 docker-tagged tests excluded
core-service HTTP smoke script against the running local app	57 checks, 57 passed
Gateway tests (stub upstreams)	14 tests, 0 failures (whole-backend run)
Notification-service tests	46 tests, 0 failures, Docker integration test excluded
Whole backend re-run (core 179, notification 46, gateway 14, dsa-lab 53)	292 tests, 0 failures, recorded in the validation document
AI golden set (rule regression test)	36 of 36 checks match the designed rules
N+1 guard	6 SQL statements for a 2-row page and for a 30-row page
Frontend tests	React Vitest 86 tests (10 files), Angular Karma 83 specs, Playwright React e2e 17 run / 17 passed (16 passed + 1 skipped without the fault-injected second stack), Angular real-backend smoke 25 of 25 steps

> The test totals above come from the validation record at the time of writing. If the code changes, re-run the suite (see the run guide in the repository) before quoting these numbers.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1.4 Five ideas that explain the design

1. **The tenant comes from the signed token, never from the client.** Every layer then repeats the check: Hibernate filter, explicit predicates, composite foreign keys, tenant-keyed caches and events, and optional PostgreSQL RLS.
2. **Business change and event commit together.** The transactional outbox writes the event row in the same transaction as the ticket change. A relay delivers it at least once, and consumers de-duplicate.
3. **Rules are data, not code.** Tenant workflows are validated JSON with a closed set of fields, operators and actions. No scripting.
4. **A human is the control point for AI.** A suggestion is an inert `PENDING_REVIEW` row. Only an authenticated agent can approve it, which posts exactly one public comment.
5. **Degrade, do not fail.** Redis outages, AI timeouts and bad workflow definitions produce slower or reduced behaviour, not errors, and each case has a test or a documented procedure.

1.5 How to read this document

Chapters 2 and 3 state the business context and requirements. Chapters 4 to 9 describe structure: architecture, services, front end, back end, tenant isolation and data. Chapters 10 to 18 walk through behaviours: tickets, SLA, knowledge base, AI, messaging, caching, authentication, analytics and infrastructure. Chapters 19 to 21 apply everything to scenarios, failures and measured performance. Chapters 22 to 24 record decisions, gaps and interview questions with answers.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Business use cases and personas

2.1 The business problem

Organisations that support customers or run internal IT desks need one place where requests arrive, get triaged, are answered within an agreed time, and are measured. Software vendors serve many such organisations at once, so the platform must keep each organisation's data strictly separate while sharing infrastructure to keep cost low. This project models that situation with three synthetic demo tenants.

Tenant slug	Name	Time zone	Business hours	Days	Seeded tickets
acme-retail	Acme Retail	America/New_York	09:00 to 17:00	Monday to Friday	150
globex-it	Globex IT Services	Europe/London	08:00 to 18:00	Monday to Friday	100
initech-health	Initech Health	America/Chicago	07:00 to 19:00	Monday to Saturday	60

Each tenant also receives 12 knowledge base articles and six article categories at seed time. The different calendars matter later: SLA due dates are computed in each tenant's own time zone.

2.2 Personas and roles

The platform has four roles. They map to real job functions, and the role hierarchy is `ADMIN` above `MANAGER` above `AGENT`. `CUSTOMER` sits outside the hierarchy because it is a restriction rather than a privilege level.

Persona	Role	Main goals	What the platform lets them do
Customer or employee	CUSTOMER	Report a problem, see progress, find self-service answers	Create tickets, see only their own tickets, read only PUBLIC comments, read PUBLISHED and PUBLIC articles, reopen a resolved or closed ticket
Support or IT agent	AGENT	Work the queue, reply fast, meet SLAs	Search and filter tickets, assign, transition, post PUBLIC and INTERNAL comments, write knowledge base articles, request and approve or reject AI suggestions
Team manager	MANAGER	Watch workload and service levels	Everything an agent can do, plus the analytics dashboard and read access to workflows and SLA policies
Tenant administrator	ADMIN	Configure the organisation	Everything a manager can do, plus users and roles, tenant settings, SLA policies, categories, workflow versions and the audit log

Two non-human actors also matter. The **SLA monitor job** runs every 60 seconds and writes escalations and breach flags. The **outbox relay** runs every second and moves events to listeners. Both are system jobs that bind the tenant themselves, one tenant at a time, rather than inheriting a tenant from a request.

2.3 Core use cases

#	Use case	Primary actor	Outcome
UC1	Register an organisation	New admin	Tenant, admin user, default categories, default SLA policies and a default workflow are created, and a login response is returned

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Use case	Primary actor	Outcome
UC2	Submit a support or IT-service ticket	Customer	Ticket created with a per-tenant number and SLA due dates; double-clicks do not create duplicates
UC3	Triage and assign	Agent or workflow	Ticket gets a priority, tags and an assignee, either by hand or by workflow rules (least-loaded agent)
UC4	Work and resolve a ticket	Agent	Status moves through the lifecycle; PENDING and ON_HOLD pause the SLA clock; every change is audited
UC5	Meet or breach an SLA	System	Due dates in business minutes; escalation levels at policy thresholds; one breach event per ticket
UC6	Answer from the knowledge base	Agent, customer	Keyword search with visibility rules; article versions; links between articles and tickets
UC7	Get an AI-assisted draft	Agent	Suggested category, priority, summary, related articles and reply, held for review
UC8	Approve or reject the AI draft	Agent	Approval posts exactly one PUBLIC comment marked as AI-assisted; rejection posts nothing
UC9	Be notified	Any user	In-app notifications for assignments, status changes, comments, escalations and breaches
UC10	Monitor service quality	Manager	Dashboard: open tickets, SLA compliance, response and resolution averages, workload, AI acceptance rate
UC11	Configure automation	Admin	Versioned workflow rules and transitions; validate, create, activate
UC12	Audit what happened	Admin	Append-only audit events by entity or actor

2.4 Why multi-tenancy is the central concern

A cross-tenant data exposure is treated as a severity-1 defect in this design. That single concern shapes almost every chapter: how identity is resolved (chapter 16), how data is stored (chapters 8 and 9), what is cached (chapter 15), what an event carries (chapter 14), and how background jobs run (chapter 11).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Functional and non-functional requirements

3.1 Functional requirements

Area	Requirement	Implemented in
Identity and tenancy	Register organisation; log in by tenant slug, email and password; four roles; user administration; tenant settings (time zone, hours, days, language, integrations)	core/auth, core/identity
Tickets	Two types (SUPPORT, IT_SERVICE); seven statuses; four priorities; three severities; categories; tags; attachment metadata; PUBLIC and INTERNAL comments; activity history; search, filters, whitelisted sort, bounded paging	core/tickets
Workflow	Versioned rule sets and custom transition tables, validated against a closed model	core/workflow
SLA	Per-priority policy; business-hours clock; pause and resume; escalation thresholds; breach detection	core/sla
Knowledge base	Articles with categories, visibility, status, version history, ticket links, scored search	core/kb, core/docstore
AI assist	Suggestion, sanitisation, resilience, human approval, audit, acceptance metric	core/ai
Analytics	Tenant-scoped dashboard cached for 60 seconds	core/analytics
Audit	Append-only audit trail with correlation ids	core/audit
Notifications	Event-driven in-app notifications; simulated email channel	notification-service
Edge	Correlation id, edge JWT check, rate limit, proxy	gateway
UI	Customer portal, ticket list and detail, knowledge base, dashboard, administration	frontend

3.2 Non-functional requirements

These are design intents. They are not measured service-level objectives, and no document in the repository claims production numbers.

Quality	Requirement	How the design addresses it
Isolation	A tenant must never read or write another tenant's data	Token-derived tenant, Hibernate filter, predicates, composite foreign keys, tenant-keyed caches and events, optional RLS; tested on H2, RLS not executed
Consistency	A state change and its event are atomic	Transactional outbox, Propagation.MANDATORY
Delivery	Events must not be lost; duplicates must be harmless	At-least-once relay, idempotent consumers keyed by eventId
Concurrency	Concurrent edits must not silently overwrite each other	@Version optimistic locking, 409 on a stale version
Idempotency	Retried creation must not duplicate	Idempotency-Key with request hash

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Quality	Requirement	How the design addresses it
Availability	Optional dependencies degrade gracefully	Redis fallback, AI degraded response, workflow fallback
Performance	Reads of reference data and the dashboard are cheap; list queries are bounded	Cache-aside, paging cap of 100, N+1 guard, tenant-first indexes
Security	Secure defaults; least privilege; no secrets in code	BCrypt, JWT with revocation, role matrix, startup guards, dummy secrets only
Observability	Every request traceable	X-Correlation-Id in logs, audit rows and event headers; Micrometer metrics
Testability	Behaviour provable without external infrastructure	Profile-based substitution (H2, Caffeine, in-JVM events), docker-tagged tests for the rest

3.3 Assumptions

1. One region and one PostgreSQL primary. There is no multi-region design.
2. Tenants are small to medium: hundreds to low thousands of open tickets. The dashboard folds at most 50 000 rows in memory.
3. Attachments are metadata only. No file bytes are accepted or stored.
4. Email and Slack settings are stored but never used to send anything.
5. A single shared HS256 secret is acceptable for a demonstration; a production system would use an identity provider.

3.4 Out of scope

The following are not implemented: SSO, SAML, OIDC, multi-factor authentication, billing and plan limits, per-tenant quotas, data residency, a GDPR erasure workflow, backup and restore tooling, multi-region failover, real email, SMS or Slack delivery, file upload and virus scanning, a distributed tracing backend, a real large language model call, and a CI/CD pipeline.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. System architecture

4.1 System context

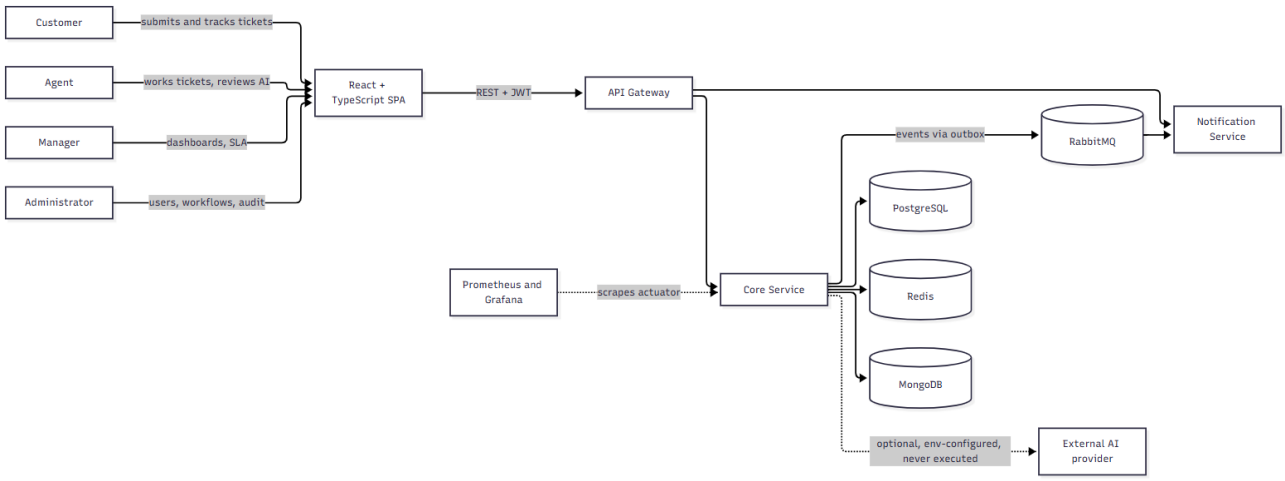


Figure 2: System context showing four personas, the SPA, the gateway, core and notification services and the data stores

Figure 2 shows who talks to what. All four personas use one single-page application. The SPA talks only to the gateway, over REST with a bearer token. The gateway routes `/api/v1/**` to core-service and the notification endpoints to the notification service. Core-service owns the relational data (PostgreSQL in the docker profile), the cache (Redis), the document store (MongoDB) and emits events through the outbox to RabbitMQ, from which the notification service consumes. An external AI provider is drawn dashed because it is optional, configured only through environment variables, and was never called. Prometheus and Grafana scrape the actuator endpoints.

In the local profile the picture is simpler: H2 replaces PostgreSQL, Caffeine replaces Redis, a table replaces MongoDB, and an in-JVM dispatcher replaces RabbitMQ. The same Java interfaces make this possible, as Figure 4 shows.

4.2 Service architecture

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

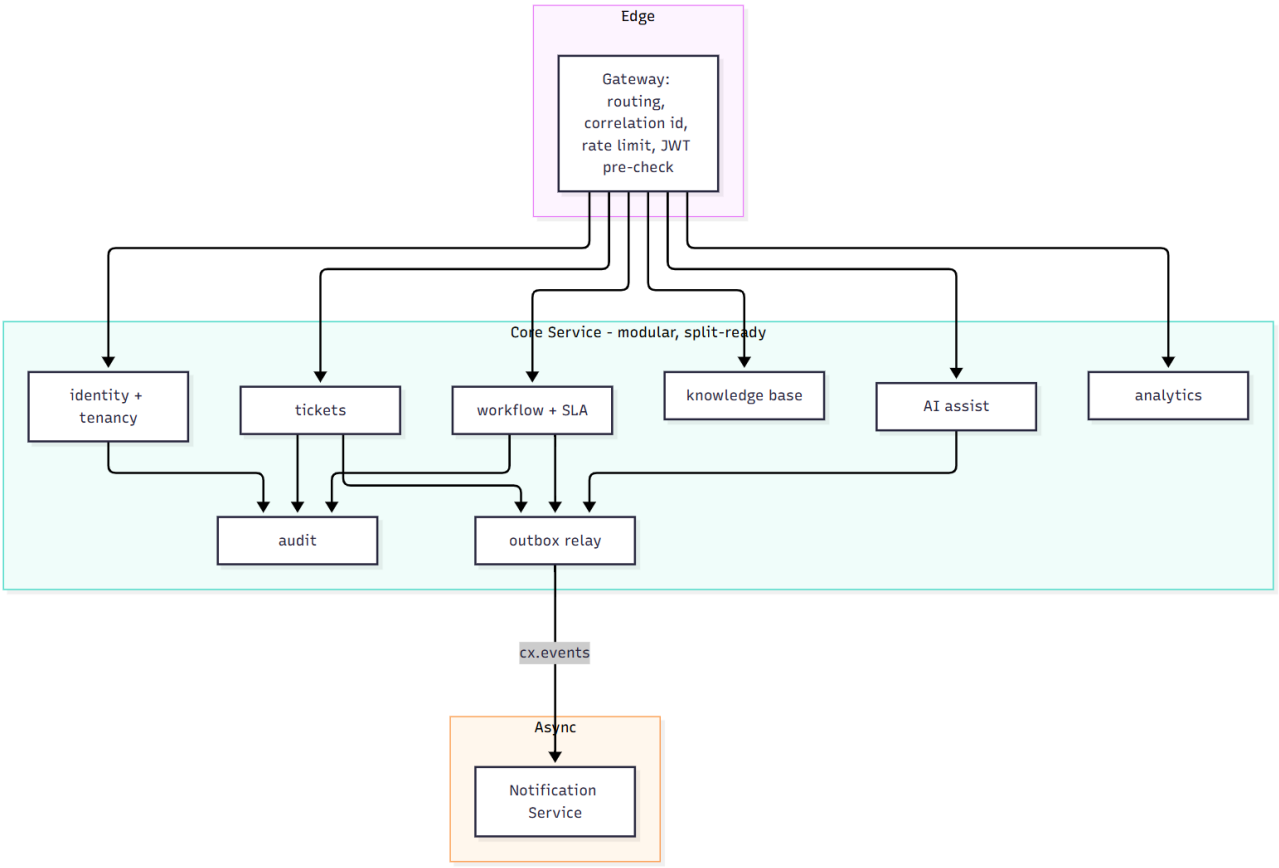


Figure 3: Service architecture with a gateway edge, a modular core service and an asynchronous notification service

Figure 3 shows the chosen shape: a modular core with split-ready seams, not full microservices. Identity, tickets, workflow and SLA, knowledge base, AI, analytics, outbox and audit are packages in one deployable. They share one transactional boundary, so a ticket change can write the ticket, an activity row, an audit row and an outbox row in a single commit. Only the parts with a clearly different runtime profile are separate: the gateway at the edge and the notification service on the asynchronous side. The contracts between them are the REST contract in `docs/API-CONTRACT.md` and the event envelope in `platform-events`. ADR-001 records the reasoning and the costs: core scales as a unit, and package boundaries are enforced by convention plus a small set of architecture tests.

4.3 Runtime profiles

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

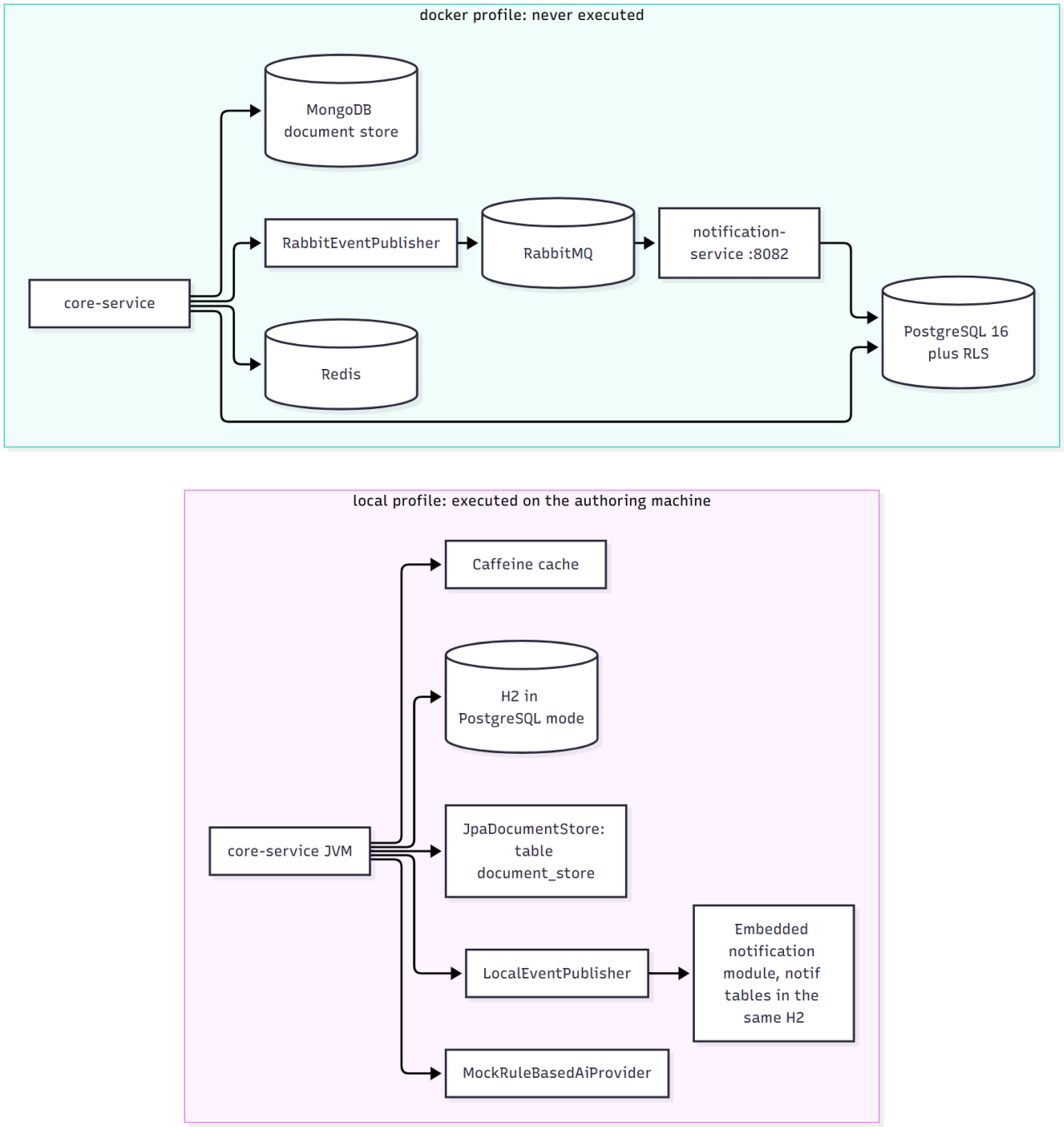


Figure 4: Local profile and docker profile substitutions for database, cache, events, document store and AI

Figure 4 contrasts the two runtime profiles. The `local` profile is what ran: an H2 database in PostgreSQL mode, Caffeine, `LocalEventPublisher` feeding an embedded notification module that writes its own `notif_` tables into the same H2 instance, a JPA-table document store and the mock AI provider. The `docker` profile swaps in PostgreSQL 16 with RLS, Redis, `RabbitEventPublisher` feeding the separate notification service, and MongoDB for the document store. The code paths in the lower box were reviewed and compiled but never started. The Flyway migrations are split in two folders for the same reason: `db/migration` runs on both databases, and `db/migration-pg` holds PostgreSQL-only features and is loaded only in the docker profile.

Concern	local (executed)	docker (not executed)
Database	H2, PostgreSQL mode	PostgreSQL 16
Migrations	db/migration	db/migration plus db/migration-pg
Cache	CaffeineCacheService	RedisCacheService

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Concern	local (executed)	docker (not executed)
Events	LocalEventPublisher to in-JVM listeners	RabbitEventPublisher to exchange cx.events
Notifications	Embedded module in core, API on 8081	Separate service on 8082
Document store	JpaDocumentStore	MongoDocumentStore
Row-level security	Off	On (cx.tenancy.pg-rls=true)
JWT secret	Clearly dummy default	Required from environment; startup refuses the dev value

4.4 A request, end to end

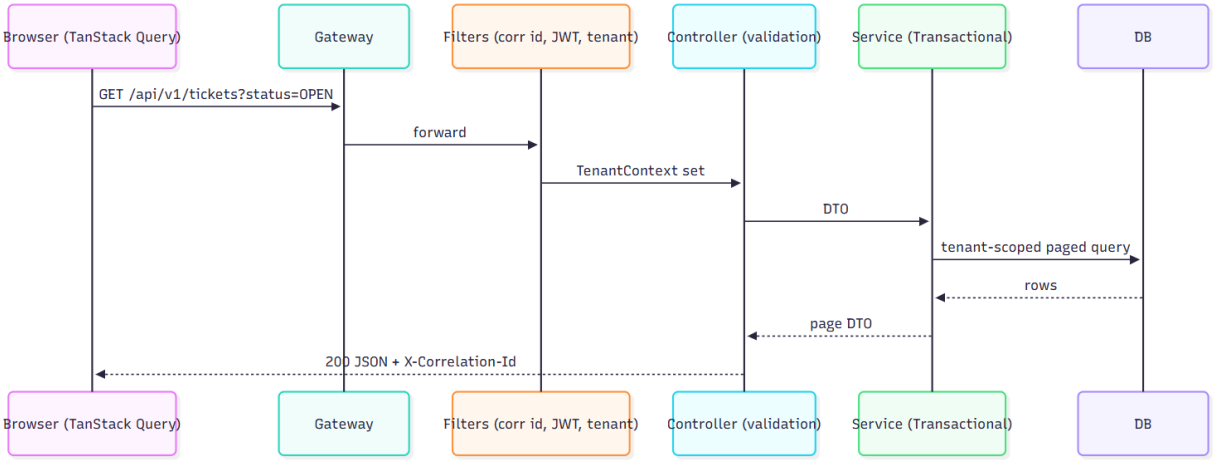


Figure 5: Request lifecycle from the browser through gateway, filters, controller, service and database

Figure 5 follows a ticket list request. The browser (through TanStack Query) calls the gateway. The gateway adds or validates a correlation id, verifies the token signature and expiry, applies a rate limit and proxies the request. In core, a servlet filter chain verifies the token again, checks revocation, and binds the tenant from the `tid` claim. The controller validates input and hands a DTO to a service. The service runs a tenant-scoped, paged query inside a transaction opened by the tenant-aware transaction manager. The response is a page DTO with an `X-Correlation-Id` header. The gateway is not a security boundary on its own: core re-validates everything.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5. Service responsibility matrix

5.1 Deployable units

Unit	Port	Owns	Exposes	Consumes	State	Scaling note
gateway	8080 (manag ement 8090)	Nothing persistent	/api/v1/ proxy, 429 and 503 and 504 mapping, actuator on the management port	Upstream HTTP from core and notification routes	In-memory token buckets	Stateless, but the rate limit is per instance, so N replicas allow N times the configured rate
core-service	8081	All business tables, the outbox, the idempotency keys, the audit log, KB and AI data	Whole REST contract, Swagger UI, actuator	Database, cache, document store, optional AI endpoint	Database is authoritative	Scales as a unit; one Hikari pool of 10 connections per node by default
notification-service	8082 (standalone)	notif_* tables, own Flyway history notif_schema_history	/api/v1/notifications...	RabbitMQ queue cx.notifications	Database (shared instance in compose)	Independent consumers: concurrency 2 to 4, prefetch 10 by default
platform-events	library	Event envelope and listener contract	DomainEvent, EventListener	n/a	none	n/a
frontend	5173 (dev)	Nothing persistent beyond a session token	Static SPA	Gateway only	sessionStorage token	Static assets, long cache headers for hashed files
Backing stores	n/a	PostgreSQL, Redis, RabbitMQ, MongoDB (docker profile only)	Standard ports bound to 127.0.0.1 in compose	n/a	See chapters 9 and 14	Never executed

5.2 Responsibilities inside core-service

Package	Responsibility	Key classes	Writes events	Cache use
tenancy	Bind tenant to persistence for every transaction	TenantContext, TenantAwareJpaTransactionManager, TenantOwned	no	no
security	JWT verification, roles, revocation, login rate limit	JwtAuthFilter, JwtService, SecurityConfig, TokenRevocationService, LoginRateLimiter	no	revocation cache in Caffeine
auth	Login, registration, tenant provisioning	AuthService, TenantProvisioner	no	no
identity	Users, roles, tenant settings	UserService, TenantSettingsService	no	calendar settings 5 min
tickets	Ticket use cases, comments, activity, numbering, idempotency, state machine	TicketService, TicketCommandService, TicketNumberAllocator, TicketStateMachine, TicketEvents	yes	categories 5 min, evicts analytics
workflow	Versioned rule sets and transition tables	WorkflowValidator, WorkflowEngine, WorkflowService	via tickets	active workflow 5 min
sla	Policies, business calendar, due dates, pause and resume, scan job	SlaService, BusinessCalendar, SlaCalculator, SlaMonitorJob	yes (escalated, breached)	policies 5 min
kb	Articles, categories, versions, search, links	KbService, KbSearchScorer	no	category list 5 min, public list 60 s

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Package	Responsibility	Key classes	Writes events	Cache use
ai	Analyze, sanitise, resilience, decisions	AiAssistService, AiDecisionService, AiSanitizer, MockRuleBasedAiProvider	yes (decided)	no
analytics	Manager dashboard	AnalyticsService	no	dashboard 60 s
audit	Append-only audit events	AuditService	no	no
outbox	Event persistence, relay, purge, publishers	OutboxService, OutboxRelay, OutboxPurgeJob, LocalEventPublisher, RabbitEventPublisher	owns delivery	no
cache	Tenant-keyed cache-aside	CacheService, CaffeineCacheService, RedisCacheService	no	is the cache
docstore	Flexible documents behind an interface	DocumentStore, JpaDocumentStore, MongoDocumentStore	no	no
common	Errors, paging, correlation id, redaction, after-commit hooks	GlobalExceptionHandler, PageParams, TxHooks, CorrelationIdFilter	no	no
seed	Synthetic demo tenants	DemoDataSeeder, KbDemoSeeder	no	no

5.3 Who may call what

Capability	CUSTOMER	AGENT	MANAGER	ADMIN
Create ticket, list own tickets, read PUBLIC comments	yes	yes	yes	yes
List all tenant tickets, assign, transition, edit	no (customer may only reopen)	yes	yes	yes
INTERNAL comments	no	yes	yes	yes
Knowledge base read	PUBLISHED and PUBLIC only	all	all	all
Knowledge base write, history, ticket links	no	yes	yes	yes
AI analyze, approve, reject	no	yes	yes	yes
Analytics dashboard	no	no	yes	yes
Workflows and SLA policies read	no	no	yes	yes
Workflows write, SLA policy write, users, settings, categories, audit	no	no	no	yes

The role matrix is asserted endpoint by endpoint in `RoleMatrixTest`, and the front end mirrors it with route guards. The guards are a usability layer only; the server decides.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Frontend architecture

6.1 Stack and structure

The front end is React 18 with strict TypeScript, built by Vite. Server state is handled by TanStack Query, forms by React Hook Form with Zod schemas, routing by React Router 6. Charts are inline SVG components written for the project (bar, donut, line), so there is no charting library. The source tree is organised by role:

Folder	Content
api/	client.ts (fetch wrapper, error mapping), endpoints.ts, types.ts (contract types)
auth/	AuthContext, tokenStore, roles, Guards (RequireAuth, RequireRole)
components/	UI kit (Button, Table, Modal, Tabs, Toast, SlaIndicator, PriorityBadge, StatusBadge, AiBadge, ConfidenceMeter), AiPanel, Layout, NotificationBell, ErrorBoundary, charts
hooks/	useLookups, useNotifications
lib/	queryKeys, queryClient, schemas, ticketFilters, format, formErrors, correlation
pages/	One component per route, all lazy loaded, plus pages/admin/*

6.2 Component tree

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

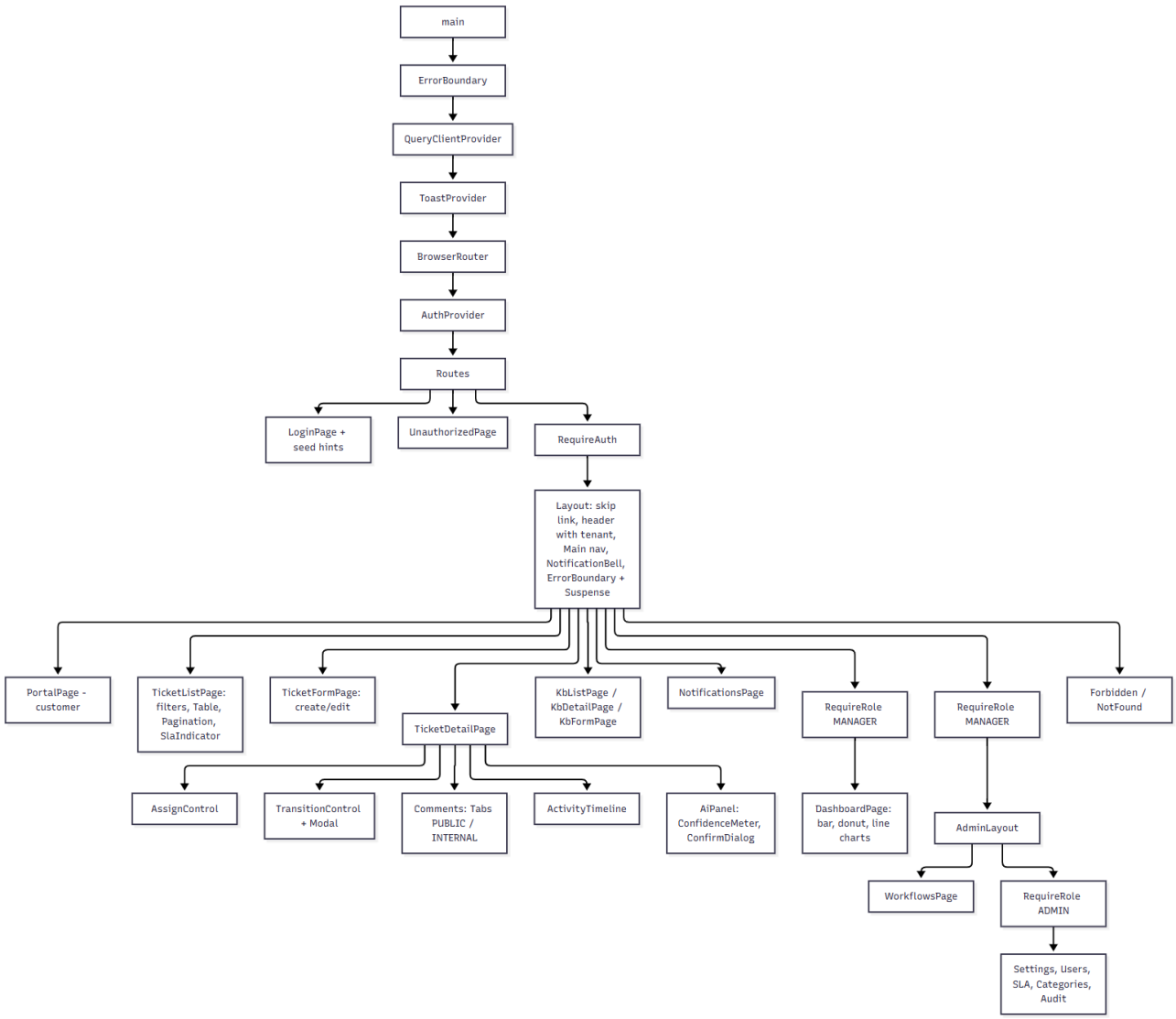


Figure 6: Frontend component tree from main through providers, guards and layout to pages

Figure 6 shows the nesting. `main` wraps the app in an error boundary, the query client provider, a toast provider, the router and the auth provider. Public routes are the login page and the unauthorized page. Everything else sits behind `RequireAuth`, inside a `Layout` that provides a skip link, a header showing the tenant name, the role-aware navigation, the notification bell, and an error boundary with a Suspense fallback for lazily loaded pages. `TicketDetailPage` is the busiest screen: assignment, a transition control with a confirmation modal, comment tabs split into PUBLIC and INTERNAL, an activity timeline and the AI panel. Role guards appear as `RequireRole` nodes: managers and above reach the dashboard and workflows, administrators reach settings, users, SLA, categories and audit.

6.3 Routing and guards

Route	Guard	Notes
/login, /unauthorized	none	Login page shows seed-login hints in dev, or when VITE_SHOW_DEMO_HINTS=true
/	authenticated	Redirects by role: customer to /portal, manager and above to /dashboard, agents to /tickets
/portal	authenticated	Customer home: own tickets, help articles

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Route	Guard	Notes
/tickets, /tickets/new, /tickets/:id	authenticated	Filters, sort and page live in the query string
/tickets/:id/edit	staff only	Customers cannot edit
/kb, /kb/:id	authenticated	Customers see only what the API returns for them
/kb/new, /kb/:id/edit	AGENT and staff only	
/notifications	authenticated	
/dashboard	MANAGER or above	
/admin/workflows	MANAGER or above	
/admin/settings, /users, /sla, /categories, /audit	ADMIN	
*	authenticated	Not-found page

Every page except the shell and the login page is a `React.lazy` chunk, so the first visit to `/login` loads only the shared vendor chunks and the entry bundle.

6.4 State management

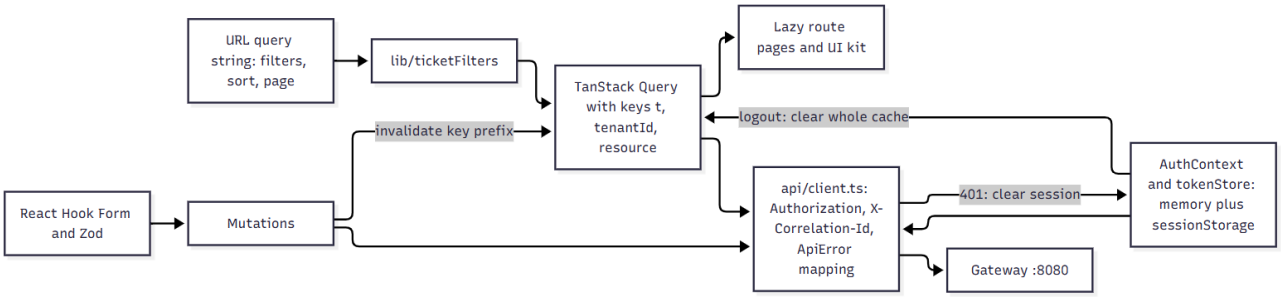


Figure 7: Frontend state flow from URL and forms through TanStack Query and the API client to the gateway

Figure 7 separates four kinds of state and says where each lives.

- **Server state** lives in TanStack Query. Every key starts with `['t', tenantId, ...]` (see `lib/queryKeys.ts`), so cached data of one tenant can never be displayed after a re-login into another. The default `staleTime` is 15 seconds, window-focus refetching is off, and a query is retried once unless the failure is a 4xx.
- **URL state** holds list filters, sort order, page and search text, which makes lists shareable and reload-safe.
- **Form state** lives in React Hook Form with Zod validation. Server `fieldErrors` from a 400 response are mapped back onto the matching inputs.
- **Auth state** is a React context over `tokenStore`. The JWT is held in memory and mirrored to `sessionStorage`, never `localStorage`. A 401 on any authenticated call clears the session and sends the user to `/login` with a "session expired" message. Logout clears the whole query cache.

Optimistic updates are used in exactly one place: marking a notification read, with rollback on error. Everything else waits for the server, because ticket state has version checks that an optimistic guess could hide.

6.5 Handling the API contract in the UI

Server behaviour	UI behaviour
409 TICKET_VERSION_CONFLICT	Conflict notice with a "Reload latest" action

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Server behaviour	UI behaviour
409 DUPLICATE_SUBMISSION	Shown as an error; create form reuses one Idempotency-Key per form mount and disables the button while pending
422 INVALID_TRANSITION	Message from the server. The transition control offers the contract default targets for the current status (a client-side copy of the default table); a tenant workflow with custom transitions is enforced by the server, so a custom table can still produce a 422
429 RATE_LIMITED	Message; a 429 is a 4xx so it is not retried automatically
200 with degraded: true from AI analyze	AI panel shows the fallback message; the agent continues manually
404 on another tenant's id	Treated as not found, which is the intended answer

Every request carries `Authorization` and a fresh `X-Correlation-Id`, so a failing request can be traced to server logs using the id shown with the error.

6.6 Accessibility, performance and testing

- **Accessibility.** Skip link to `#main`, programmatically associated labels, `role="alert"` on errors, `aria-live` for toasts and AI progress, modals that trap focus and close on Escape, ARIA tab pattern with arrow keys, status and SLA conveyed by text as well as colour.
- **Bundle (measured with `npm run build` by the front-end author).** React chunk 164.5 kB (53.7 kB gzip), forms 93.9 kB (26.4 gzip), query 41.5 kB (12.5 gzip), entry 32.4 kB (11.6 gzip); the first visit to `/login` is about 104 kB gzip. The ticket detail page chunk is 15.4 kB (5.0 gzip). Real-device load time and Largest Contentful Paint were not measured.
- **Tests.** Ten Vitest files (UI components, schemas, auth and guards, ticket list filters, AI panel states, duplicate-submit protection, error boundary, API client mapping, notification rollback) and 11 Playwright tests (login, create ticket, assign and transition, customer restrictions, AI approval, tenant isolation). Executed on 2026-10-17: Vitest 86 of 86 passed, and the Playwright suite (now 17 tests) ran 17 and passed 17 with the fault-injected second stack (16 passed and 1 skipped without it). A second UI, `frontend-angular/`, has 83 Karma specs (83 passed) and a 25-step real-backend smoke (25 of 25 passed).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Backend architecture

7.1 Modules and layering

The Maven reactor has these modules: `platform-events`, `notification-service`, `core-service`, `gateway` and `dsa-lab`. Dependencies are acyclic at module level.

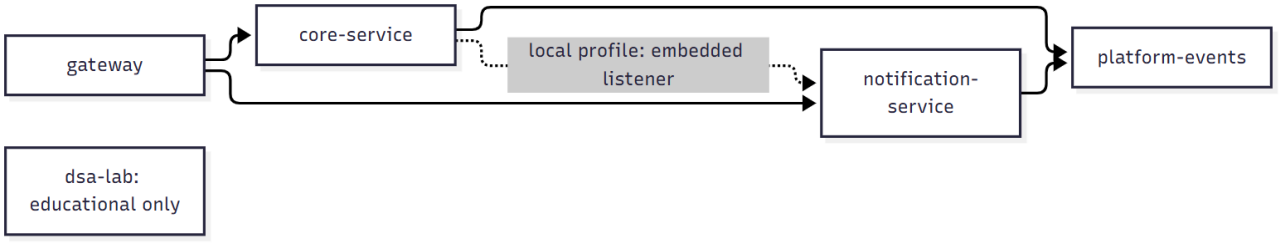


Figure 8: Maven module dependencies between gateway, core, notification and platform-events

Figure 8 shows the module graph. Both core-service and notification-service depend on `platform-events` for the event envelope. The gateway depends on neither at compile time: it speaks HTTP to them. The dashed edge is the local-profile arrangement, where core-service depends on the plain notification-service library jar so the notification module can be embedded. The standalone executable is built as a separate `exec` classifier artifact, which avoids a Spring Boot repackaging conflict. `dsa-lab` is educational and is not on any request path.

Inside core-service the layering is conventional: controllers (validation, DTO mapping, `@PreAuthorize`), services (transactions, authorisation of objects, domain rules), repositories (Spring Data JPA with explicit tenant predicates) and entities. Controllers return DTO records, never entities, and `ArchitectureRulesTest` fails the build if one does.

7.2 Package dependencies

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

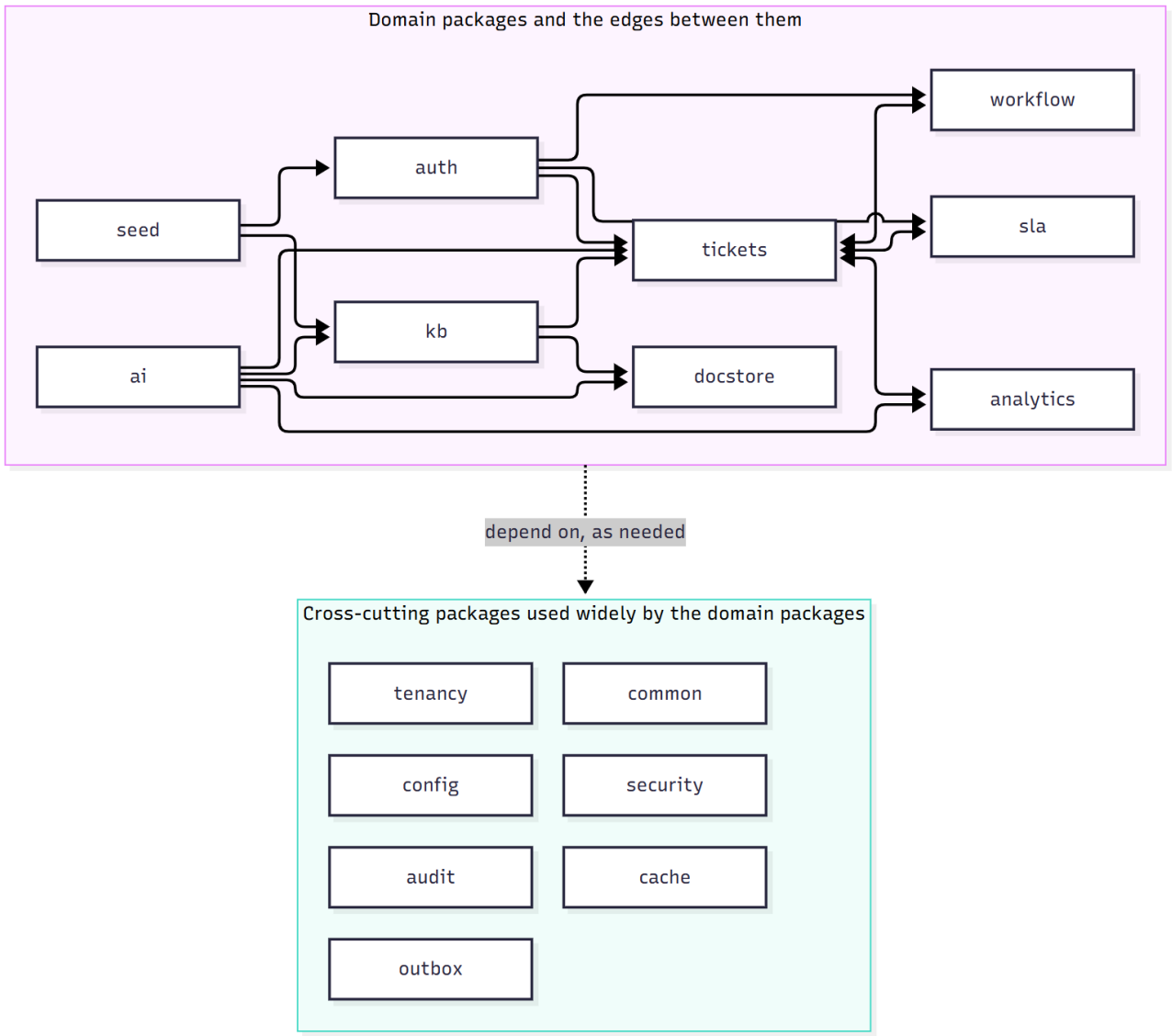


Figure 9: Package-level dependencies inside core-service, grouped into cross-cutting and domain packages

Figure 9 was derived by reading the `import` statements of the `com.example.cx.core` packages. The lower box lists the cross-cutting packages (tenancy, common, config, security, audit, cache, outbox), which most domain packages use; they are shown as a group rather than as edges. The upper box shows the notable edges between domain packages (the widely used `identity` package is omitted for readability). A double-headed arrow marks a package-level cycle: `tickets` depends on `sla`, `workflow` and `analytics`, and each of those depends back on `tickets`. The `ai` package is a leaf that reaches into `kb`, `tickets`, `analytics` and `docstore`, which is by design: it consumes the knowledge base, posts through `TicketService.addComment` and feeds the acceptance-rate metric. `seed` and `auth` sit on top because they provision demo data and tenants through the other packages.

> The cycles are real and are listed as technical debt in chapter 23. The build enforces tenant-binding rules, controller rules and the audit repository rule, but it does not enforce a general package dependency graph. Spring Modulith or ArchUnit rules would be the natural next step.

7.3 Cross-cutting mechanisms

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Mechanism	Where	What it does
Correlation id	CorrelationIdFilter (outermost), gateway filter	Accepts a well-formed id ([A-Za-z0-9._-]{1,64}), otherwise generates one; puts it in the MDC, response header, audit rows and event headers
Tenant binding	JwtAuthFilter, TenantAwareJpaTransactionManager	Sets TenantContext from the verified tid claim; enables the Hibernate filter and, with RLS, set_config per transaction
Error contract	GlobalExceptionHandler, ErrorWriter	One JSON schema for every non-2xx response with a stable code; stack traces and messages are not included by Spring
Paging	PageParams	size 1 to 100, whitelisted sort fields, id tie-break
After-commit hooks	TxHooks.afterCommit	Cache eviction runs only after the transaction commits
Redaction	Redaction	Masks password, token, key and webhook values before audit details are stored
Metrics	Micrometer	Counters such as cx.cache, cx.outbox.*, cx.ai.analyze, cx.sla.job.*

7.4 Configuration

All tunables are typed in `CxProperties` under the prefix `cx`, with safe defaults and secrets taken from the environment. Notable defaults: JWT time-to-live 60 minutes; login limiter 10 burst and 5 per minute (the local profile raises it to 100 and 60 per minute for demos); outbox poll 1000 ms, batch 50, 8 attempts, lease 60 seconds, backoff base 2 seconds; SLA job every 60 seconds, 200 tickets per tenant per run, at-risk threshold 75 percent; AI timeout 3000 ms, 2 retries, 10 second total deadline, 30 analyses per tenant per minute. `StartupGuards` refuses to start the docker profile with the development JWT secret or with AI fault injection switched on. For local runs, core, gateway and the standalone notification service now share the same clearly dummy default signing value (it starts with `dev-only`), so a local gateway accepts tokens issued by a local core; the docker profiles require `CX_JWT_SECRET` from the environment.

7.5 Gateway design

The gateway is a Spring WebFlux application of roughly 300 lines, chosen over Spring Cloud Gateway for explainability and to avoid release-train coupling. Filters run in a fixed order: correlation id, edge authentication (signature and expiry only), a token-bucket rate limit (default burst 100, refill 20 per second, keyed by the token subject or, for anonymous routes such as login, by the socket peer address), then the streaming proxy with a 2 second connect timeout and a 30 second response timeout. Upstream failure maps to 503, a timeout to 504 and a rate-limit rejection to 429 with `Retry-After`, all in the contract error schema. Actuator is on a separate management port. A client-supplied `X-Forwarded-For` is never trusted for rate limiting. The 504 path is implemented but not covered by a test.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8. Tenant isolation architecture

8.1 Model: shared database, shared schema, discriminator column

All tenants share one database and one schema. Every tenant-owned table carries `tenant_id uuid not null`. The alternative models were considered in ADR-002: database per tenant isolates best and costs most to operate; schema per tenant is a middle ground with catalog and migration overhead; a hybrid where large tenants get dedicated databases is the natural evolution but is not built.

8.2 Defence in depth

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

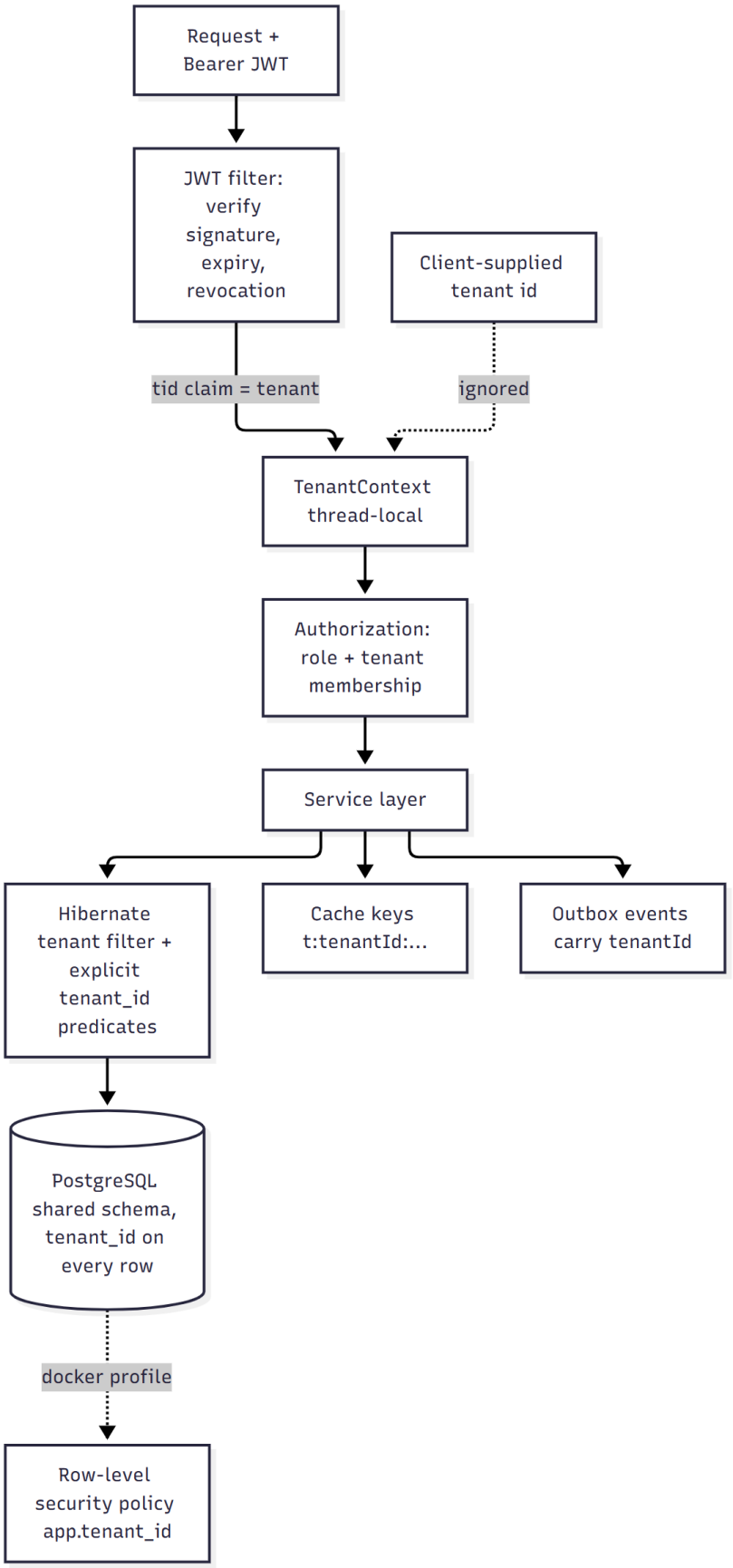


Figure 10: Tenant isolation layers from the verified JWT down to optional row-level security, cache keys and events

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 10 shows the layers a request passes through. The tenant enters the system at exactly one place: the JWT filter verifies signature, expiry and revocation and copies the `tid` claim into a thread-local `TenantContext`. A client-supplied tenant id is ignored at every point. The context then feeds six independent controls:

1. **Hibernate filter.** `TenantAwareJpaTransactionManager` enables the `tenantFilter` on every transaction's session, so even a query that forgets the tenant predicate sees only the bound tenant (proved in `TenancyTest`).
2. **Explicit predicates.** Hibernate filters do not apply to `find-by-id`, so repositories use `findByIdAndTenantId`, and `TicketSearch` always adds the tenant first.
3. **Persist guard.** A `@PrePersist` hook in `TenantOwned` refuses to write a row for another tenant, or for no tenant.
4. **Composite foreign keys.** For example `fk_ticket_requester` references `app_user (tenant_id, id)`, so the database itself rejects a ticket that points at another tenant's user or category.
5. **Tenant-keyed caches and events.** Cache keys are `t:{tenantId}:...` and every event carries `tenantId`.
6. **Optional PostgreSQL RLS.** Policy `tenant_id = cx_current_tenant()` on 16 tables with `FORCE ROW LEVEL SECURITY`. This exists in `db/migration-pg` and is active only in the docker profile. It was never executed.

8.3 The pooled-connection detail

RLS reads the setting `app.tenant_id`. If that value were set per connection, a pooled connection could carry one tenant's id into another request, which is the classic RLS bug. The code instead runs `select set_config('app.tenant_id', ?, true)`; the `true` argument makes it `SET LOCAL`, scoped to the current transaction. The value disappears at commit or rollback, and a connection returned to the pool carries no tenant. The consequence is that every database access must run inside a transaction opened by the tenant-aware manager; a bare `JdbcTemplate` call outside a transaction sees no tenant and, because `cx_current_tenant()` then returns NULL, matches zero rows (fail closed).

Some tables are deliberately outside RLS: `organization` (login resolves the slug to a tenant before any tenant is known), `revoked_token` (no tenant column) and `outbox_event` (the relay claims rows across tenants). They are protected by application logic. RLS also does not bind superusers or roles with `BYPASSRLS`, so the runtime role must be a dedicated non-superuser; the compose file currently uses one owner role for everything, which `FORCE` covers but production should still separate.

8.4 Rules enforced by the build

`ArchitectureRulesTest` fails the build if any class outside an allow-list calls `TenantContext.set`, `callAs` or `runAs`, if a controller has a parameter with "tenant" in its name, if a controller exposes an `@Entity`, or if the audit repository gains an update or delete method. Allowed binders are the JWT filter, login and tenant provisioning (server-side slug resolution) and system jobs that iterate tenants themselves: the SLA monitor, the idempotency-key purge and the seeder.

8.5 Observable behaviour

Situation	Result	Evidence
Tenant B requests tenant A's ticket id	404 NOT_FOUND, never 403, so existence does not leak	TenantIsolationTest, smoke checks
Forged X-Tenant-Id header or ?tenantId=	Ignored	TenantIsolationTest
Customer requests another customer's ticket in the same tenant	404	RoleMatrixTest
Ticket created with another tenant's category id	400 validation error (unknown category)	TenantIsolationTest

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Situation	Result	Evidence
Analytics for two tenants within the cache TTL	Different payloads, because keys are tenant-prefixed	TenantIsolationTest
Switching tenant in the UI	Re-login; the query cache is cleared and keys are tenant-first	frontend/src/lib/queryKeys.ts

8.6 Known weaknesses

Native SQL and `JdbcTemplate` bypass the Hibernate filter, so only RLS (unexecuted) would catch such a mistake. A thread-bound context does not follow `@Async` or virtual-thread hops; the core is blocking MVC, so this holds today. A tenant suspension is not instant, because tokens stay valid until they expire. There is no per-tenant quota, so a large tenant can crowd out others on the shared pool and indexes. These are listed again in chapter 23.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

9. Data model

9.1 Overview

The schema is created by Flyway. Portable migrations in `db/migration` (V1 to V15 and V20 to V22) run on both H2 in PostgreSQL mode and PostgreSQL. V16 to V19 are intentionally unused. PostgreSQL-only migrations in `db/migration-pg` use the V1001 to V1011 range so the two sets can never collide. Hibernate runs with `ddl-auto: validate` in tests, so any drift between an entity and the schema fails the build. There are 19 core tables, and the notification module adds three more (`notif_*`) with its own Flyway history table.

The portable SQL was executed only on H2. A first PostgreSQL run is outstanding, and one real defect has already shown that H2 is not PostgreSQL: null ordering differs, which broke the SLA scan rotation until the query used `coalesce(sla_checked_at, epoch)` instead of `NULLS FIRST`.

9.2 Core tables

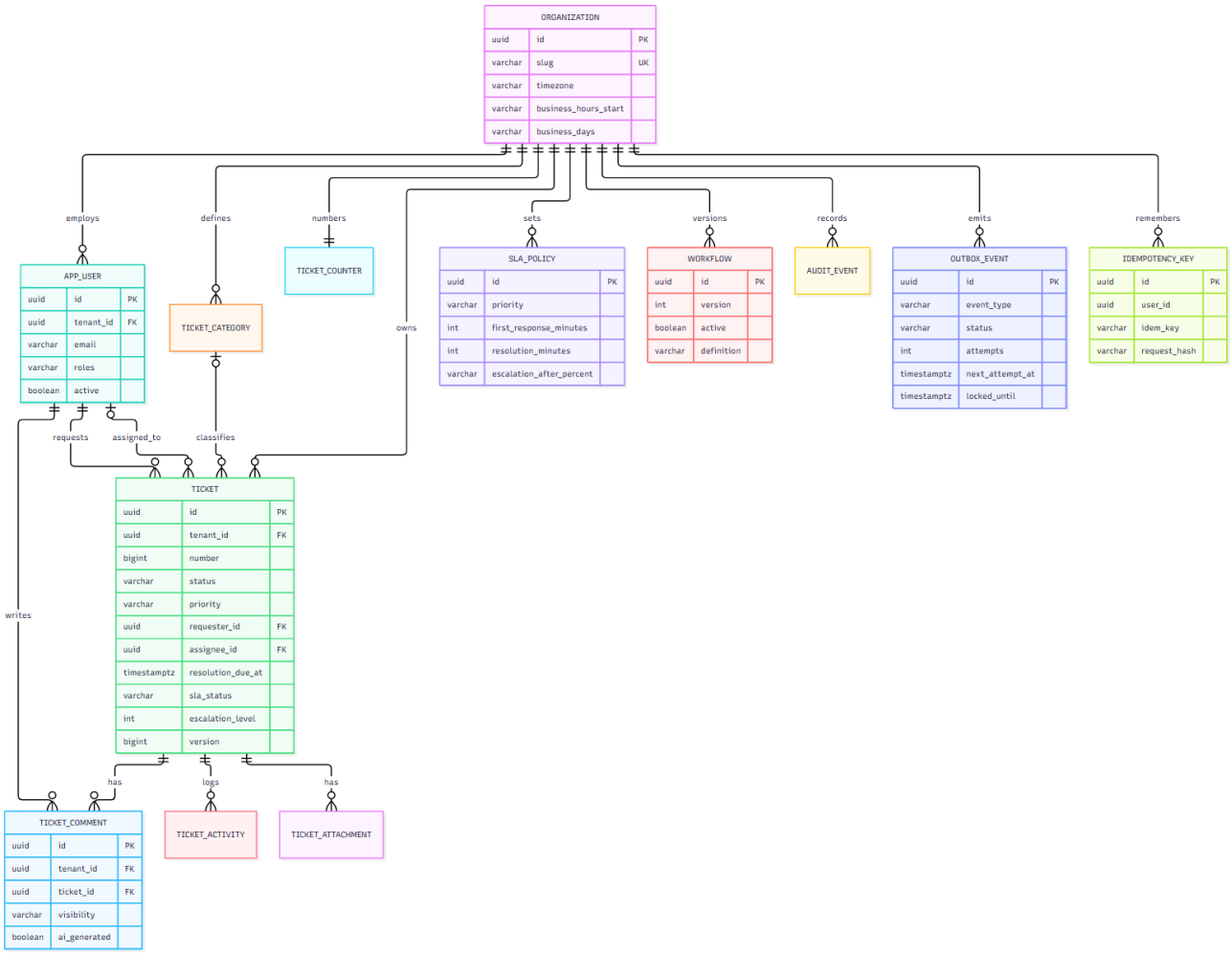


Figure 11: Entity-relationship diagram of the core tables, derived from the Flyway migrations

Figure 11 shows the core relationships, drawn from migrations V1 to V15. The organisation row is the tenant root and its `id` is the `tid` claim in every JWT. Everything else carries `tenant_id`. Tickets reference users and categories through composite foreign keys `(tenant_id, id)`, which is why `app_user` and `ticket_category` each declare a `unique (tenant_id, id)` constraint. Attribute lists in the figure are abbreviated to the columns that matter for the explanation; the migrations are authoritative.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Table	Purpose	Notable columns and constraints	Indexes
organization	Tenant root and tenant settings	slug unique and lower case; timezone, business_hours_start, business_hours_end, business_days, default_language, email_provider, slack_webhook_url, ai_provider	unique slug
app_user	Users of all tenants	unique (tenant_id, email) with lower-case check; roles stored as a delimited string; password_hash (BCrypt); active; unique (tenant_id, id) for composite foreign keys	(tenant_id, active)
revoked_token	Logout list	Primary key jti; expires_at; no tenant column; purged after expiry	(expires_at)
audit_event	Append-only audit	actor_id, actor_email, action, entity_type, entity_id, details, correlation_id	tenant plus time, entity, actor
ticket_category	Per-tenant categories	type SUPPORT or IT_SERVICE; default_priority; unique (tenant_id, name)	
ticket_counter	Per-tenant ticket numbering	next_number >= 1; one row per tenant	primary key
ticket	The central entity	number unique per tenant; status, priority, severity and SLA status check constraints; requester_id, assignee_id, category_id as composite foreign keys; due dates, first_responded_at, resolved_at, sla_paused_at, sla_checked_at, breach_notified, escalation_level, reopen_count, version	Eight composite indexes, all led by tenant_id (V15)
ticket_attachment	Attachment metadata only	size_bytes between 0 and 26 214 400	by ticket
ticket_comment	Comments	visibility PUBLIC or INTERNAL; ai_generated flag; author composite foreign key	by ticket and time
ticket_activity	Per-ticket timeline	type, actor_name, details (up to 1000 characters)	by ticket and time
idempotency_key	Replay store for ticket creation	unique (tenant_id, user_id, idem_key); request_hash (SHA-256); response_body up to 20 000 characters	by creation time
workflow	Versioned workflow definitions	unique (tenant_id, version); active; definition JSON up to 20 000 characters	(tenant_id, active)
sla_policy	One policy per priority	unique (tenant_id, priority); escalation_after_percent as text list; resolution minutes at least first response minutes	

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Table	Purpose	Notable columns and constraints	Indexes
outbox_event	Transactional outbox	status PENDING, PROCESSING, SENT or FAILED; attempts, next_attempt_at, locked_by, locked_until, last_error, sent_at	(status, next_attempt_at), (tenant_id, created_at)

The ticket list path relies on these V15 indexes, all with `tenant_id` first: `(tenant_id, status, priority, created_at)`, `(tenant_id, created_at)`, `(tenant_id, assignee_id, status)`, `(tenant_id, requester_id, created_at)`, `(tenant_id, sla_status)`, `(tenant_id, category_id)`, `(tenant_id, resolved_at)` and `(tenant_id, status, sla_checked_at)`. On H2, an experiment confirmed the "tenant first" rule is also a performance rule: the same status filter without a tenant predicate scanned all 75 001 rows (chapter 21).

9.3 Knowledge base, AI and notification tables

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

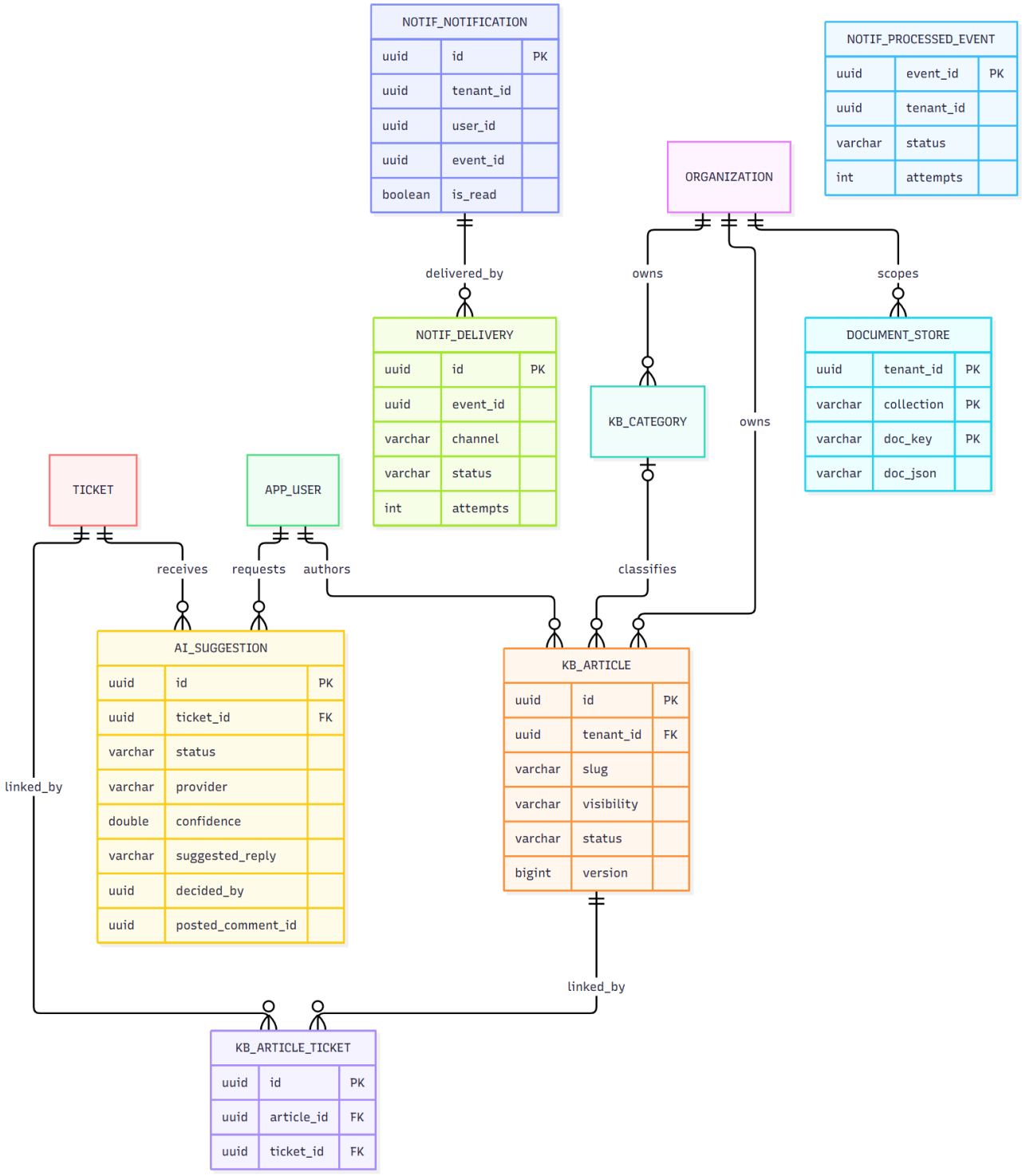


Figure 12: Entity-relationship diagram of the knowledge base, AI suggestion, document store and notification tables

Figure 12 covers migrations V20 to V22 and the notification module's V1. Knowledge base articles hold the current content and are the authoritative record; a link table connects articles to tickets with a unique `(article_id, ticket_id)` pair. `ai_suggestion` holds status, ticket, decision and the suggested fields relationally, because approval is a compare-and-set update that needs a strongly consistent row. `document_store` is a generic JSON table keyed by `(tenant_id, collection, doc_key)` that backs knowledge base version snapshots and AI metadata in the local profile; in the docker profile a MongoDB collection takes its place. The three `notif_` tables are drawn without foreign keys to core tables, because the notification module owns its own schema.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Table	Purpose	Key constraints
kb_category	Article categories	unique (tenant_id, name), unique (tenant_id, slug)
kb_article	Current article state	unique (tenant_id, slug); visibility PUBLIC or INTERNAL; status DRAFT or PUBLISHED; version for optimistic locking; body up to 15 000 characters
kb_article_ticket	Article-to-ticket links	composite foreign keys to article and ticket
document_store	Flexible documents	primary key (tenant_id, collection, doc_key); doc_json up to 32 000 characters
ai_suggestion	Suggestion and decision	status PENDING_REVIEW, APPROVED, EDITED_APPROVED or REJECTED; posted_comment_id; confidence as double
notif_processed_event	Consumer idempotency claim	primary key event_id
notif_notification	Inbox rows	unique (event_id, user_id); index on (tenant_id, user_id, is_read, created_at)
notif_delivery	Per-channel delivery state	unique (event_id, user_id, channel); status PENDING, SENT, FAILED or DEAD

9.4 PostgreSQL-only additions (not executed)

Migration	Content
V1001	Partial index ix_ticket_open_queue on open tickets; GIN full-text index on title and description; pg_trgm and a trigram index on lower(title)
V1002	cx_current_tenant() function; RLS enabled and forced with policy tenant_isolation on 11 tables
V1003	Trigger that raises an exception on any UPDATE or DELETE of audit_event
V1010	Full-text and trigram indexes on kb_article (not used by the portable search path)
V1011	The same RLS policy for kb_category, kb_article, kb_article_ticket, ai_suggestion and document_store (16 tables in total)

9.5 Data ownership and lifecycle

Data	Written by	Retention in code
Tickets, comments, activity	TicketService	none (kept)
Audit events	AuditService in the business transaction	none; append-only; no archival job (gap)
Idempotency keys	TicketCommandService	purged hourly after 24 hours, per tenant
Outbox rows	OutboxService	SENT rows purged hourly after 7 days in bounded batches; FAILED rows kept
Revoked tokens	TokenRevocationService	purged after the token expires
Notification processed-event rows	notification module	no purge job (gap)
KB version snapshots	KbService through DocumentStore	listed newest first, bounded to 500 per article

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

10. Ticket lifecycle and assignment

10.1 Status model

A ticket has one of seven statuses: NEW, OPEN, PENDING, ON_HOLD, RESOLVED, CLOSED and REOPENED. A new ticket always starts as NEW; moving it to OPEN is an explicit act by an agent or a tenant workflow. The default transition table equals the API contract and is asserted by `TicketStateMachineTest` (12 allowed pairs, 37 rejected pairs out of 49).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

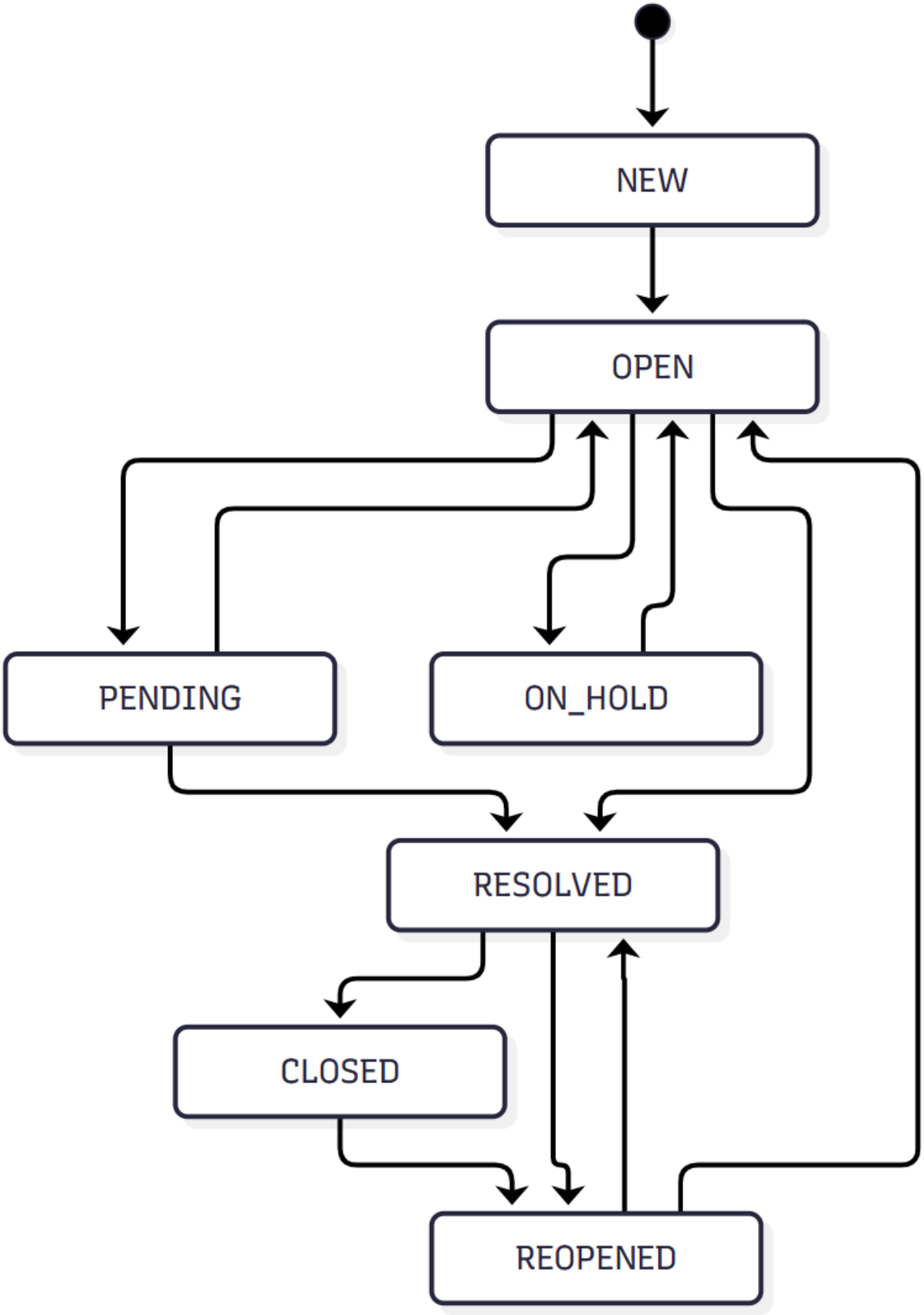


Figure 13: Ticket status state machine showing the twelve allowed default transitions

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 13 shows the default table. Work normally flows NEW to OPEN to RESOLVED to CLOSED. PENDING means waiting on the customer and ON_HOLD means waiting on a third party; both pause the SLA clock. RESOLVED can be closed or reopened, CLOSED can only be reopened, and REOPENED goes back to OPEN or straight to RESOLVED. A tenant's active workflow may replace the table entirely with its own `transitions`; if the stored definition is missing or invalid, the built-in default is used and a fallback counter is incremented.

From	Allowed targets
NEW	OPEN
OPEN	PENDING, ON_HOLD, RESOLVED
PENDING	OPEN, RESOLVED
ON_HOLD	OPEN
RESOLVED	CLOSED, REOPENED
CLOSED	REOPENED
REOPENED	OPEN, RESOLVED

10.2 What a transition does

`TicketService.transition` runs in one transaction and performs these steps in order:

1. Load the ticket with tenant (and, for customers, requester) scoping; unknown or foreign ids give 404.
2. Compare the request `version` with the stored version; a mismatch gives 409 `TICKET_VERSION_CONFLICT`.
3. Customers may only move a RESOLVED or CLOSED ticket to REOPENED; anything else is 403.
4. Check the target against the effective transition table; a violation gives 422 `INVALID_TRANSITION`.
5. Pause the SLA clock when entering PENDING or ON_HOLD, or resume it (shifting due dates) when leaving.
6. Set `resolvedAt` when entering RESOLVED, clear it when leaving a done state, and increment `reopenCount` on REOPENED.
7. Refresh `slaStatus`, run the workflow rules once for this trigger, save, and record activity, an outbox event `ticket.status.changed` and an audit row.
8. Evict the tenant's analytics cache after commit.

10.3 Creating a ticket

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

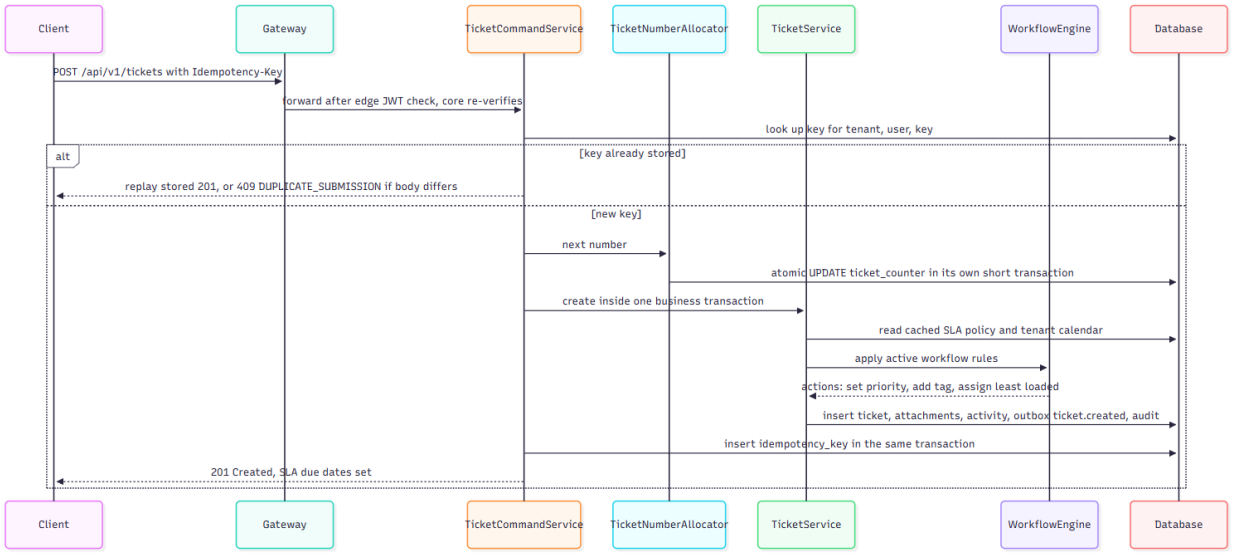


Figure 14: Ticket creation sequence with idempotency check, number allocation, SLA computation, workflow and outbox in one transaction

Figure 14 is the exact order of work for `POST /api/v1/tickets`. First the `Idempotency-Key` header (1 to 100 characters of letters, digits and `. _ : -`) is looked up for the tenant and user. A stored key with the same SHA-256 request hash replays the original 201 response with the header `Idempotent-Replay: true`; a different body gives 409 `DUPLICATE_SUBMISSION`. For a new key, the ticket number is allocated by an atomic `UPDATE ticket_counter SET next_number = next_number + 1` in its own tiny transaction before the business transaction starts. Then one transaction computes SLA due dates from the cached policy and the tenant calendar, applies the active workflow, inserts the ticket, attachment metadata, an activity row, the `ticket.created` outbox row and an audit row, and finally inserts the idempotency key. If two requests race on the same key, they collide on the unique constraint; the loser rolls back completely and replays the winner's response.

The numbering design has one trade-off worth remembering: numbers are unique and increasing per tenant but not gap-free, because a create that fails after allocation burns its number. This was an intentional exchange for removing a tenant-wide row lock that had produced multi-second tail latency (chapter 21).

10.4 Assignment and escalation

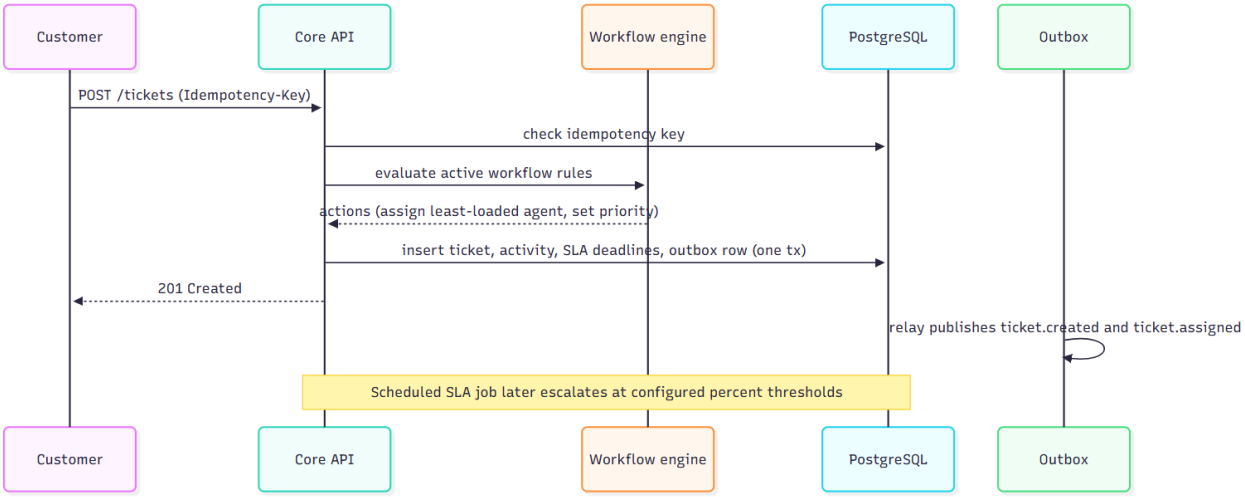


Figure 15: Summary sequence of creation, workflow-driven assignment, outbox relay and later SLA escalation

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 15 is the summary view: the workflow engine returns actions during creation, the ticket is stored with its outbox rows in one transaction, the relay publishes `ticket.created` and `ticket.assigned` afterwards, and the scheduled SLA job escalates later. Figure 14 supplies the detail that the summary omits, such as the number allocation and the idempotency record.

Manual assignment. `POST /tickets/{id}/assign` takes `{assigneeId, version}`. The assignee must be an active staff user of the same tenant (AGENT, MANAGER or ADMIN); otherwise the request fails validation. Assigning the same person again is a no-op. A non-null assignee produces a `ticket.assigned` event; passing null unassigns.

Workflow assignment. The action `ASSIGN_TO_ROLE_LEAST_LOADED` loads up to 100 active users with the given role, counts each candidate's open tickets (not RESOLVED or CLOSED) with one grouped query, and picks the lowest count, breaking ties by user id so the choice is deterministic. It never overrides an existing assignee. The default workflow created for every tenant has three rules:

Rule	Condition	Actions
Security keyword raises priority	title CONTAINS security	SET_PRIORITY HIGH, ADD_TAG security
Urgent tickets alert managers	priority EQ URGENT	ADD_TAG urgent, NOTIFY role MANAGER
Route high priority to least loaded agent	priority IN HIGH, URGENT	ASSIGN_TO_ROLE_LEAST_LOADED role AGENT

Rules are evaluated in order against the current ticket, so the first rule can raise a MEDIUM ticket to HIGH and the third then routes it. If any SET_PRIORITY action fired during creation, due dates are recomputed for the new priority.

Escalation. Escalation is time-driven and happens in `SlaMonitorJob` (chapter 11). A workflow can also escalate immediately with the ESCALATE action, which raises `escalationLevel` by one up to a maximum of 5 and emits `ticket.escalated`.

10.5 Comments, first response and activity

A staff PUBLIC reply from someone other than the requester sets `firstRespondedAt` on the first occurrence, which refreshes the SLA status and bumps the ticket `version` once. INTERNAL comments are invisible to customers and are never announced to the requester. Activity rows are written for creation, updates, assignment, status change, comments, workflow actions, escalation and breach. A customer sees only the CREATED and STATUS_CHANGED entries.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

11. SLA and escalation

11.1 Policies

Every tenant receives four SLA policies at provisioning, expressed in business minutes. An administrator can change them with `PUT /sla-policies/{id}`; the policy priority cannot be changed, resolution minutes must be at least the first-response minutes, and escalation percentages must be unique and ascending (up to five, each 1 to 200).

Priority	First response (business minutes)	Resolution (business minutes)	Escalation thresholds (percent of resolution time)
URGENT	30	240	50, 75, 100
HIGH	60	480	75, 100
MEDIUM	240	1440	75, 100
LOW	480	2880	75, 100

Policy changes affect new tickets and tickets whose priority changes. They do not rewrite due dates already computed for other tickets. Policies are cached per tenant for five minutes and evicted after an update commits.

11.2 Business-hours arithmetic

`BusinessCalendar` is a pure, immutable class built from the tenant's time zone, daily window and working days. It offers two operations:

- `addBusinessMinutes(from, minutes)` walks forward day by day. Non-working days are skipped. On a working day, if the cursor is before opening it moves to opening; if it is at or after closing it moves to the next day's opening. Otherwise it consumes `min(remaining, minutes until close)`. When the remainder fits in the day, the result is returned.
- `businessMinutesBetween(from, to)` sums, for every calendar date in the range, the overlap of `[from, to)` with that date's opening window on working days.

Windows are constructed per local date with `ZonedDateTime`, so daylight-saving changes are handled correctly (spring and fall cases are covered in `BusinessCalendarTest`, nine tests). The loop is bounded to 20 000 iterations. If a tenant is configured with no working days or an empty window, the calendar degrades to plain wall-clock minutes instead of looping.

11.3 Worked example

Use the seeded tenant `acme-retail` (America/New_York, 09:00 to 17:00, Monday to Friday) and the default HIGH policy (60 minutes first response, 480 minutes resolution, thresholds 75 and 100). The calendar dates are illustrative.

A customer creates a HIGH ticket on **Friday 2026-10-16 at 15:30 local time** (19:30 UTC, because daylight saving still applies until 1 November).

First response due. Friday has 90 business minutes left (15:30 to 17:00), and 60 fits, so the due time is Friday 16:30 local (20:30 UTC).

Resolution due. The ticket has 480 minutes to use.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Step	Day	Minutes available	Remaining after step
1	Friday 15:30 to 17:00	90	480 - 90 = 390
2	Saturday, Sunday	skipped (not working days)	390
3	Monday 09:00 onward	480	fits, so 09:00 + 390 min

Resolution is due **Monday 2026-10-19 at 15:30 local** (19:30 UTC).

Pause and resume. On Monday at 10:00 the agent moves the ticket to PENDING, and the customer replies and the agent returns it to OPEN on Tuesday at 11:00. The paused interval contains Monday 10:00 to 17:00 (420 business minutes) and Tuesday 09:00 to 11:00 (120 business minutes), so 540 business minutes. On resume both due dates (the first-response date only if no reply has happened yet) are pushed out by 540 business minutes. Starting from Monday 15:30: 90 minutes remain on Monday, leaving 450 to place; Tuesday offers 480, so the new resolution due time is Tuesday 09:00 + 450 minutes = **Tuesday 16:30 local**.

Escalation arithmetic as implemented. The percent used is measured in business minutes, the same unit as the due dates: `(target minutes - businessMinutesBetween(now, due)) / target minutes`. Closed hours (nights, weekends) therefore consume nothing, and paused time is excluded because the due dates were shifted by the business minutes paused. An earlier version divided wall-clock time since creation by the wall-clock distance to the due date; that defect was found while writing this worked example and has been fixed.

Quantity	Value
Business minutes consumed on Sunday 21:30 local	90 of 480 (18.75 percent): 90 on Friday, none on the weekend
75 percent point (360 business minutes)	Monday 13:30 local (90 on Friday + 270 on Monday)
What the old wall-clock arithmetic said	Friday 19:30 UTC to Monday 19:30 UTC = 4 320 minutes; 75 percent = Sunday 21:30 local, i.e. AT_RISK although only 90 business minutes were used

So with the default HIGH policy (60 first response / 480 resolution minutes, thresholds 75 and 100), the ticket is not AT_RISK on Sunday evening, and the first escalation threshold and AT_RISK are reached on Monday at 13:30. This is demonstrated by tests in `SlaCalculatorTest`: `closedHoursDoNotConsumeBudget_fridayEveningTicketIsNotAtRiskOnSunday` (Friday 15:30 to Sunday 21:30 gives exactly 18.75 percent and ON_TRACK), `thresholdsAreCrossedByBusinessMinutesOnly` (87.5 percent and escalation level 1 on Monday 14:30, ON_TRACK at 13:29, BREACHED after 15:30), `pausedBusinessTimeIsExcludedFromPercentUsed` and `firstResponsePercentUsesBusinessMinutesToo`. The earlier wall-clock behaviour was recorded as technical debt and is now fixed.

11.4 Status values and the scan job

slaStatus	Meaning
ON_TRACK	Open, not paused, below the at-risk percent (75 by default)
AT_RISK	Open and at least 75 percent of the business-minute budget used
BREACHED	Past the resolution due time, or past the first-response due time with no staff reply; also set for a ticket that was resolved late or already flagged
MET	Resolved or closed on time
PAUSED	sla_paused_at is set (status PENDING or ON_HOLD)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

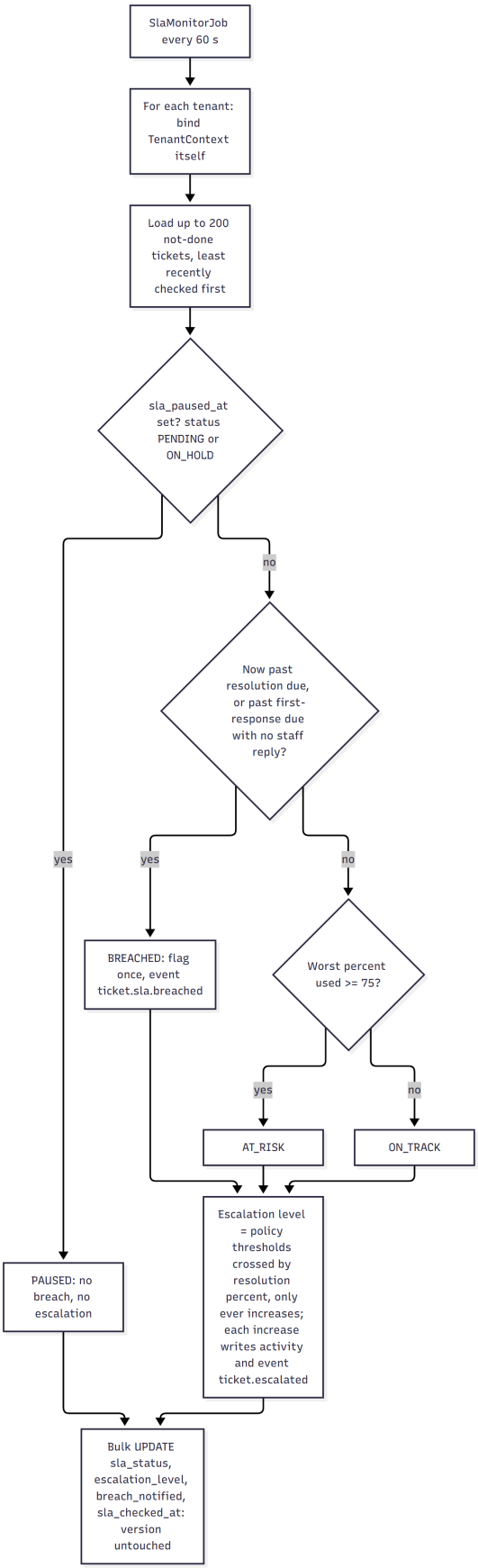


Figure 16: SLA monitor job flow showing pause check, breach check, at-risk check, escalation level and bulk update

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 16 is the per-ticket decision made by `SlaMonitorJob`. Every 60 seconds it pages through all tenants, binds each tenant itself, and examines at most 200 not-done tickets for that tenant, ordered by `coalesce(sla_checked_at, epoch)` so the least recently checked go first. A paused ticket is marked PAUSED and skipped. Otherwise the job compares now against the due dates, computes the escalation level as the number of policy thresholds that the resolution percent has crossed, and writes the result with a bulk UPDATE. The level never decreases, and `breach_notified` flips only once, so re-running the job or running it on two nodes never publishes the same escalation or breach twice. For each new escalation level it writes an activity row and an outbox event `ticket.escalated`; for a new breach it writes `ticket.sla.breached` with the type FIRST_RESPONSE or RESOLUTION.

Because the update is a bulk UPDATE, it does not touch the ticket `version`, so background work never causes a user-facing 409. A failure in one tenant is logged and counted (`cx.sla.job.errors`) and does not stop the other tenants.

11.5 Limits of the design

- **Detection latency grows with open tickets.** A tenant with more than 200 open tickets is covered by rotation, so the worst-case delay is about `ceil(open / 200)` minutes (arithmetic, not measured).
- **Sequential scanning.** Tenants are scanned one after another on one scheduler thread, so one slow tenant delays the next ones.
- **No leader lock.** Every node runs the job. It is idempotent, so overlap is safe but redundant; ShedLock is the suggested fix.
- **Percent-used approximation** as shown in the worked example.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

12. Knowledge base

12.1 Purpose and rules

The knowledge base lets agents publish answers once and reuse them in tickets and in AI recommendations, and lets customers self-serve. An article has a title, a stable slug (unique per tenant), a summary, a body, a category, a visibility (PUBLIC or INTERNAL), a status (DRAFT or PUBLISHED), an author, timestamps and a version.

Rule	Behaviour
Customer visibility	PUBLISHED and PUBLIC only, in list, get and search. A draft or internal id gives 404, and request filters can only narrow, never widen
Staff visibility	Everything in the tenant
Write, history, link and category endpoints	AGENT or above; a customer gets 403
Optimistic locking	version on PUT is optional; when sent and stale the answer is 409
Related tickets	Staff see relatedTicketIds; customers never do
Audit	KB_ARTICLE_CREATED, KB_ARTICLE_UPDATED, KB_ARTICLE_PUBLISHED, KB_ARTICLE_TICKET_LINKED, KB_CATEGORY_CREATED

12.2 Search flow

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

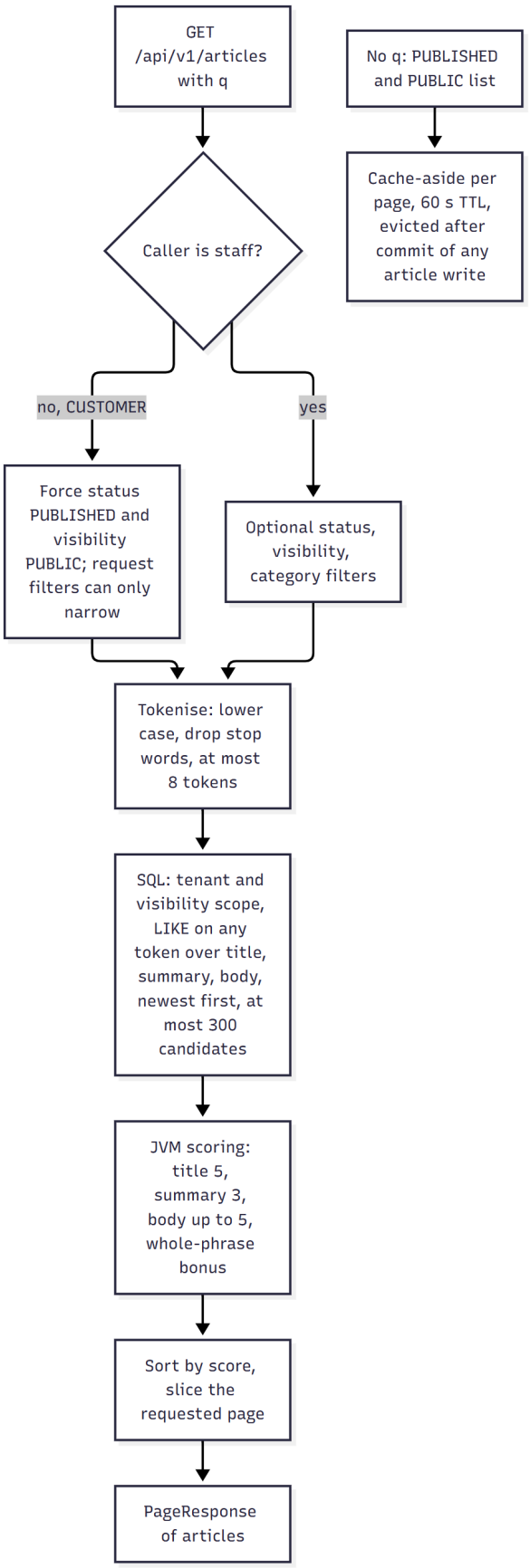


Figure 17: Knowledge base search flow from role-based filters through SQL narrowing to in-JVM scoring and paging

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 17 shows the search used on both H2 and PostgreSQL. The caller's role first fixes the scope: for a customer the status and visibility filters are forced. The query text is tokenised (lower case, alphanumeric tokens of at least two characters, stop words removed, at most eight tokens). The database narrows with a case-insensitive `LIKE` over title, summary and body for any token, scoped by tenant and visibility, newest first and bounded to 300 candidates. The JVM then scores the candidates deterministically: 5 points per token found in the title, 3 per token in the summary, up to 5 for body occurrences, and a bonus of 10 or 5 when the whole phrase appears in the title or summary. The page is sliced after sorting by score. The lower path in the figure is the plain list for customers: the PUBLISHED and PUBLIC list is cached per page for 60 seconds and evicted after the commit of any article write; the category list is cached for five minutes. Search results themselves are not cached because the keys are high-cardinality.

This is explainable and portable, and it is not a relevance model. `totalElements` counts matches only within the 300-candidate bound. PostgreSQL GIN and trigram indexes exist in V1010, but the native `to_tsvector` path is not wired in because it could not be executed here. Beyond roughly 10 000 to 100 000 articles per tenant, the right move is that native path or a search engine behind the same service signature.

12.3 Storage split

The relational database is the source of truth; the document store holds derived, append-only, schema-flexible data. If the document store disappeared, every article, suggestion and decision would still exist; only version history and analysis metadata would be gone.

Data	Store	Reason
Article metadata and current content	Relational (kb_article)	Unique slug, composite foreign keys, transactions with the audit row, optimistic locking, tenant filter and RLS, filtering and paging
Categories, article-ticket links	Relational	Referential integrity and uniqueness
AI suggestion status, ticket, decision	Relational (ai_suggestion)	Approval is a compare-and-set UPDATE ... WHERE status = 'PENDING_REVIEW'
Article version snapshots	DocumentStore collection kb_article_version, key {articleId}:{8-digit version}	Immutable, read by key or prefix, shape may grow without a migration
AI analysis metadata	DocumentStore collection ai_suggestion_meta	Heterogeneous per provider, write-once, diagnostic, never used for decisions

The rule of thumb used: if a query needs a join, a constraint or a transaction with something else, it is relational; if it is a self-contained JSON blob written once and read by key, it is a document.

12.4 Document store implementations

cx.docstore	Class	Profile	Transaction behaviour
jpa (default)	JpaDocumentStore	local and any SQL deployment	Joins the caller's transaction, so a snapshot commits or rolls back with the article change
memory	InMemoryDocumentStore	tests	Volatile
mongo	MongoDocumentStore	docker	Not in the PostgreSQL transaction; snapshot is written last and a failure aborts the article change; the inverse failure can leave one orphan snapshot that the next change overwrites

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The Mongo implementation uses one collection `cx_documents` whose documents are `{tenantId, collection, key, doc, createdAt, updatedAt}`; every query starts with `tenantId`, and three indexes are created at startup. It was never executed; `MongoDocumentStoreDockerIT` is the test that would prove it on a Docker host. If stricter guarantees were needed, the snapshot could travel through the outbox to a consumer that writes it.

12.5 Seed data

Each demo tenant has 12 synthetic articles and 6 categories: 8 published and public, 2 published and internal, 1 draft public and 1 draft internal, plus 12 version-1 snapshots. This exact mix lets the visibility rules be asserted through the API (`KbSeedTest`, `KbApiTest`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13. AI-assisted support and human approval

13.1 What the feature is, and is not

The AI module helps an agent triage and answer a ticket. Its default provider, `MockRuleBasedAiProvider`, is a deterministic rule engine built from fixed keyword tables and arithmetic. It is not machine learning, not a large language model, and it understands nothing: "I do not want a refund" still matches the keyword "refund". Identical input always yields identical output (tested 20 times and over HTTP). The valuable part of the module is the plumbing around a provider, which would stay the same if a real model were plugged in: sanitisation, bounded failure, human approval, audit and metrics.

	MockRuleBasedAiProvider (default)	HttpAiProvider (optional stub)
Selected by	nothing, or AI_PROVIDER=mock	AI_PROVIDER=http
Technology	Keyword tables and arithmetic	A call to an HTTPS endpoint from AI_API_URL
Network and key	None	AI_API_URL (https only), AI_API_KEY; both empty by default
Executed	Fully, in tests and smoke runs	Selection and fail-closed behaviour only; the real call never ran

How the mock decides: the category is the one whose keyword list matches most (title matches count double; ties resolved by table order); the priority is URGENT if any urgent keyword appears, else HIGH, else LOW, else MEDIUM; related articles come from keyword overlap between the ticket and published article titles and summaries (score at least 0.10, top three); the summary is extractive (subject plus the first two sentences); the reply is a fixed paragraph per category that never copies ticket text. Confidence is `0.30 + 0.10 x category matches (cap 0.30) + 0.10 x priority matches (cap 0.20) + 0.15 x best article overlap`, clamped to 0.30 to 0.95. It measures how many rules fired, not the probability of being right.

13.2 Analyze and decide

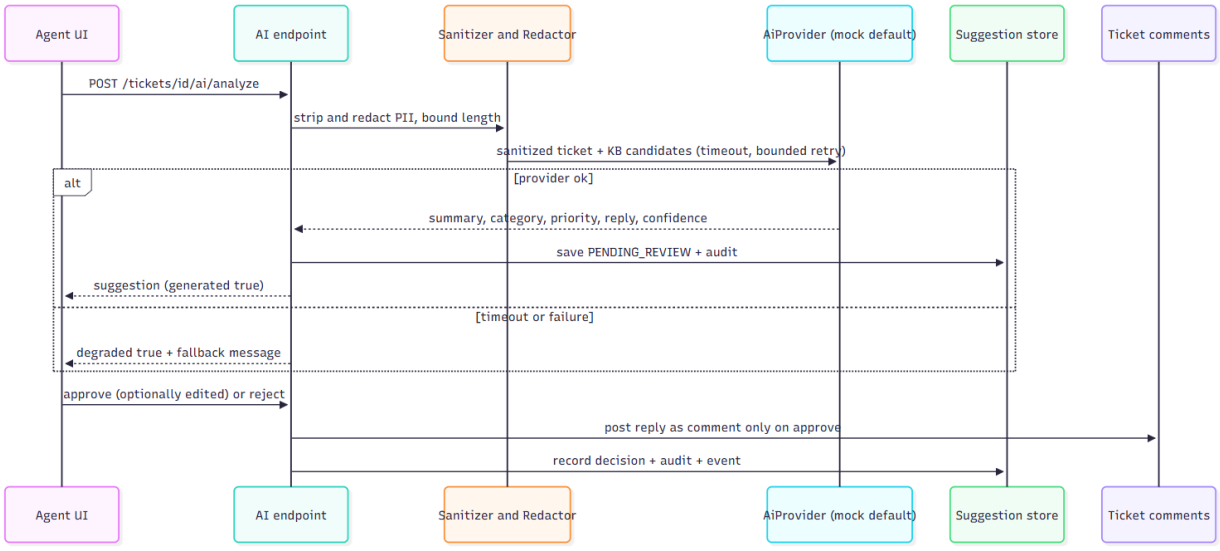


Figure 18: AI suggestion sequence from analyze through sanitising, the provider, storage and the human decision

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 18 shows the two halves of the flow. In the first half an agent calls `POST /tickets/{id}/ai/analyze`. The service loads the ticket, the tenant's categories and up to 100 published article candidates inside a short read transaction, then sanitises the text outside any transaction so that a slow provider can never hold a database connection. On success the suggestion is stored as `PENDING_REVIEW` with an audit row and the response carries `generated: true`. On timeout or failure the agent receives a degraded response instead. In the second half the agent approves, optionally with an edited reply, or rejects. Only approval produces customer-visible text.

An analyze response looks like this (values are illustrative, and the identifiers are placeholders):

```
{
  "suggestionId": "<uuid>",
  "provider": "MOCK_RULE_BASED",
  "generated": true,
  "summary": "Charged twice for the March subscription.",
  "suggestedCategory": { "id": "<uuid>", "name": "Billing" },
  "suggestedPriority": "HIGH",
  "confidence": 0.66,
  "explanation": "Matched billing keywords: charged; priority keywords: ...",
  "recommendedArticles": [ { "id": "<uuid>", "title": "Understanding your invoice", "score": 0.42 } ],
  "suggestedReply": "We are reviewing the billing details and will confirm any correction or refund as soon as
   possible.",
  "status": "PENDING_REVIEW",
  "degraded": false,
  "fallbackMessage": null
}
```

13.3 Suggestion states and the approval gate

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

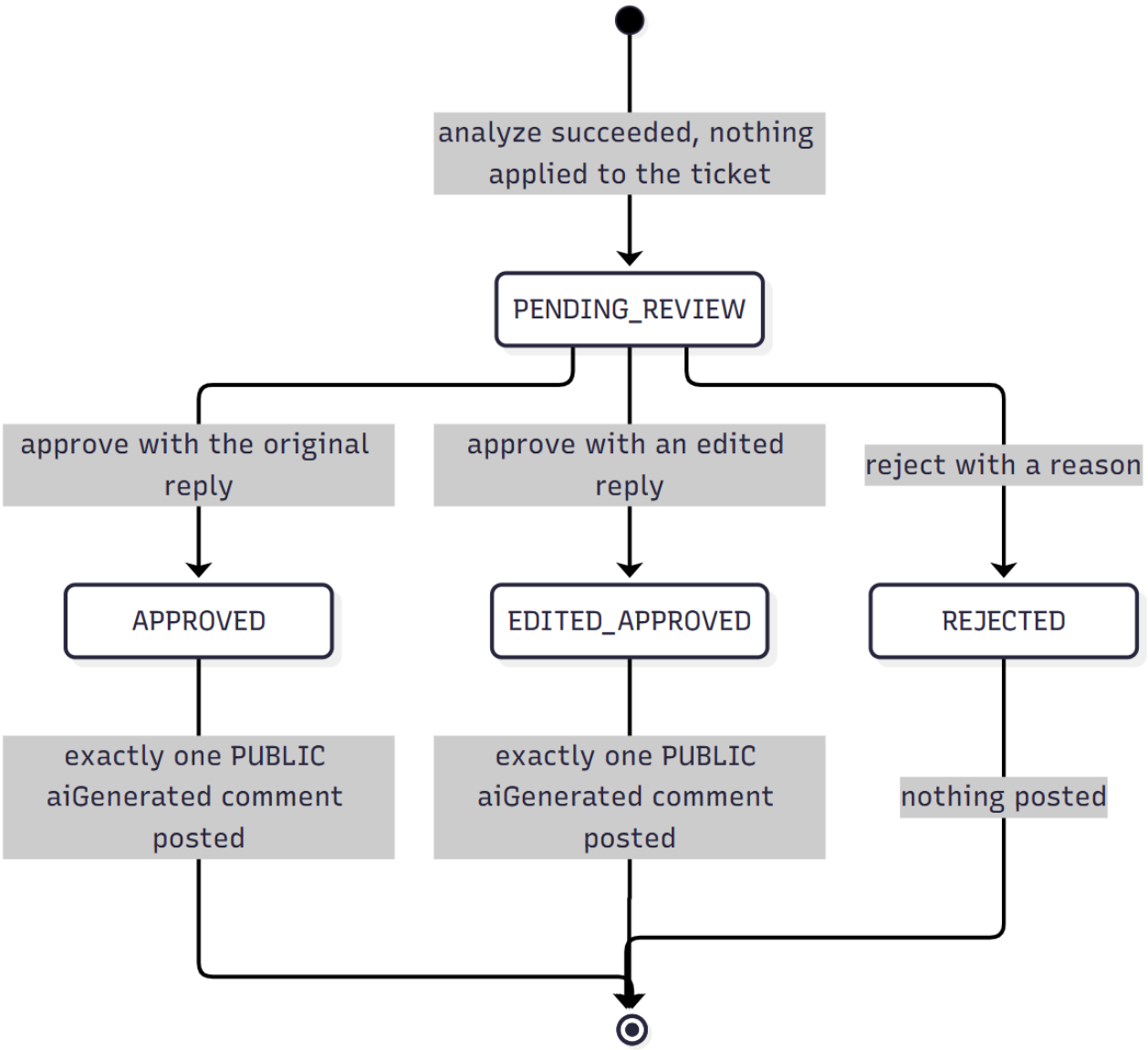


Figure 19: AI suggestion state machine with one pending state and three terminal decisions

Figure 19 shows that a suggestion is created once and decided once. `AiDecisionService.approve` flips `PENDING_REVIEW` to `APPROVED` or `EDITED_APPROVED` with a compare-and-set `UPDATE ... WHERE status = 'PENDING_REVIEW'`. If the update does not change exactly one row, the answer is 409. Then it posts exactly one comment through `TicketService.addComment` with visibility PUBLIC and `aiGenerated=true`, authored by the approving agent, so the normal authorisation, first-response, activity and event logic all apply. Status flip, comment, audit row and the `ai.suggestion.decided` outbox event are one transaction, so a failed comment rolls the status flip back. Four concurrent approvals produce one 200, three 409 and one comment (tested). Reject requires a reason and posts nothing. Approving with a non-PUBLIC visibility is a validation error, because AI replies are PUBLIC only.

13.4 Resilience

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

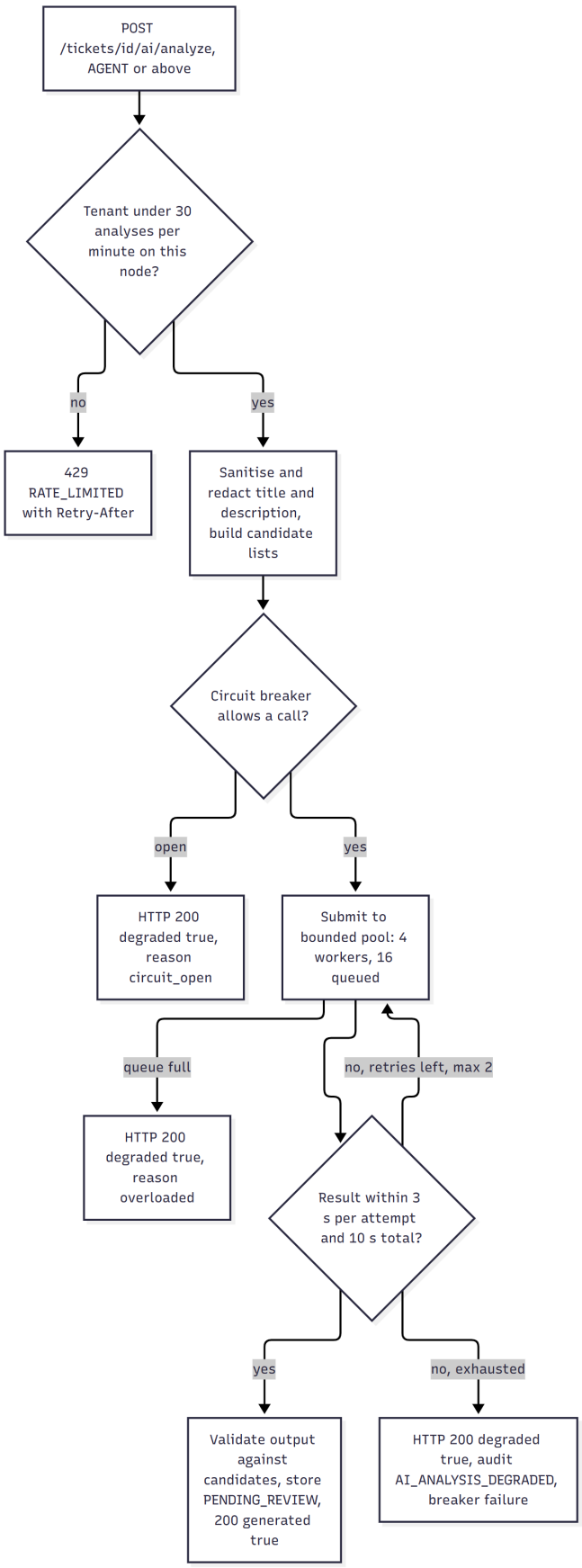


Figure 20: AI analyze resilience flow with rate limit, sanitising, circuit breaker, bounded pool, retries and degraded responses

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 20 lists every exit from `analyze`. A per-tenant rate limit of 30 analyses per minute per node returns 429 with `Retry-After`; this is the one error case, because retrying later is the correct client behaviour. Next the text is sanitised and a circuit breaker is consulted: it opens after 5 consecutive failed analyses for 30 seconds, then allows a single trial. A bounded pool of 4 workers with 16 queued tasks runs the provider; a full queue yields a degraded answer immediately. Each attempt has a 3 second timeout, and up to 2 retries follow the first attempt by default (the setting is clamped to a hard cap of 3 retries), with a short linear pause between them and a 10 second total deadline. With defaults that is 3 attempts and a worst-case wait of about 9.15 seconds. Every failure path returns HTTP 200 with `degraded: true` and a fixed fallback message, no summary and no reply, stores nothing, and writes an `AI_ANALYSIS_DEGRADED` audit row whose `reason` is one of `timeout`, `provider_error`, `deadline`, `circuit_open`, `overloaded` or `interrupted`. The agent keeps working manually; nothing else depends on AI.

A test hook `cx.ai.fault-mode` (`NONE`, `FAIL`, `TIMEOUT`) forces failures in tests. It is not reachable over HTTP and the application refuses to start with it enabled in the docker profile.

13.5 Safety model

Threat	Control	Evidence
Prompt injection in ticket text	Text is data, never instructions. Unicode is normalised (NFKC), zero-width and control characters and HTML tags are removed, and any sentence matching a control phrase ("ignore previous instructions", role markers, "mark this ticket as urgent", ...) is dropped entirely	AiSanitizerTest; AiApiTest shows an attacked ticket yields a suggestion identical to its clean twin
A compromised or misbehaving provider	The provider is a pure function with no access to the database, tenant context or ticket write API; its output is validated against server-side candidate lists (unknown categories, article ids and priorities are dropped) and cleaned of tags and URLs	AiArchitectureTest fails the build if a provider imports a repository, tenant or security class
Nothing applied automatically	A suggestion is an inert row; suggested category and priority are informational and never written to the ticket	AiApiTest: ticket version unchanged, no comment
Data leakage to a provider	E-mail addresses, phone numbers, card-like digit runs, bearer tokens, JWTs and key=value secrets are replaced by [REDACTED_*]; length bounded (title 200, description 4 000); no tenant id, user id, e-mail or ticket number is sent	AiApiTest checks redaction before the provider
Internal knowledge leaking to customers	Internal articles may be recommended to the agent, but a templated reply cites PUBLIC articles only	tested
Hallucination or plain wrong answer	Confidence and explanation shown with every suggestion; real KB ids only; a human reads and may edit; the edit is visible as EDITED_APPROVED; acceptance rate is a KPI	design and tests

> **Limits to state honestly.** Regex lists catch common phrasings, not paraphrases. With the mock provider a paraphrase is harmless because it is just words, but with a real model it could still influence the text of a draft. The architecture (no actions, human approval) is the real protection, not the regexes. Regex redaction misses names, street addresses and free-text secrets.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13.6 Persistence, audit and metrics

The authoritative suggestion row lives in `ai_suggestion` (status, ticket, decision, suggested fields). Diagnostic metadata (matched features, scores, prompt hash, provider details, redaction counts, injection signals) is written to the document store under collection `ai_suggestion_meta`; a failure to write it is logged and counted (`cx.ai.meta.failed`) and never fails the user-visible result. Audit actions are `AI_SUGGESTION_GENERATED`, `AI_ANALYSIS_DEGRADED`, `AI_SUGGESTION_APPROVED`, `AI_SUGGESTION_EDITED_APPROVED` and `AI_SUGGESTION_REJECTED`, with ids and counts but never ticket text or reply text. The dashboard field `aiAcceptanceRatePercent` is approved plus edited-approved over all decided suggestions, and is 0 when none are decided; the smoke test observed it moving 0, 50.0, 66.7, 50.0 as decisions were made.

13.7 Evaluation

The only evaluation that exists is a golden set: 12 tickets (account lockout, VAT invoice, VPN drops, printer, outage, crash, question, refund, onboarding request, access request, security incident, no-signal) times three checks (category, priority, top recommended article), 36 of 36 matching. Each expected value is what the rules were designed to produce, so 100 percent means nothing drifted, not that the system is accurate. It paid for itself once: the first run revealed that two-letter words such as "to" caused a wrong article recommendation, fixed by extending the stop-word list. A real evaluation would need labelled, anonymised tickets, per-category precision and recall, a priority confusion matrix, calibration, a red-team set of paraphrased injections and the online metrics already exposed (acceptance, edit rate, degraded rate). None of this has been done.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

14. Messaging and the transactional outbox

14.1 The problem

A ticket change must produce a notification event. Publishing to the broker inside the database transaction cannot be atomic (a dual write), and publishing after the commit can lose the event if the process dies in between. The outbox pattern removes the problem by making the event a row in the same database transaction as the business change, and delivering it later.

14.2 Outbox and relay

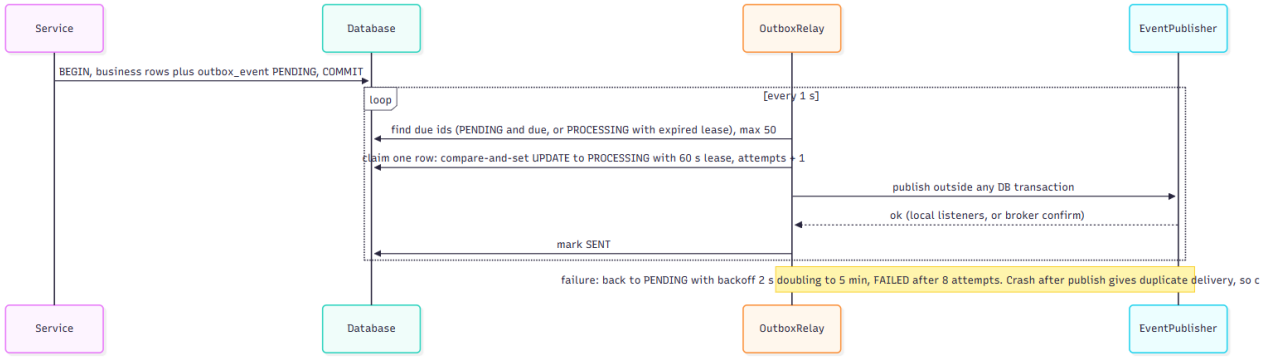


Figure 21: Transactional outbox sequence showing the commit, polling, claim, publish and mark steps

Figure 21 shows the life of one event. `OutboxService.publish` is annotated `Propagation.MANDATORY`, so it can only run inside the business transaction; if there is none, it fails. It inserts an `outbox_event` row with a `DomainEvent` JSON envelope (`eventId`, `type`, `schemaVersion`, `tenantId`, `correlationId`, `occurredAt`, a flat string payload). `OutboxRelay` polls every second for up to 50 due rows. It claims each row with a compare-and-set `UPDATE` that moves it to PROCESSING, sets a 60 second lease and increments `attempts`; exactly one of N concurrent claimers wins (tested with 6 claimers). This is the portable stand-in for `FOR UPDATE SKIP LOCKED`. The publish call happens outside any database transaction, and a successful publish marks the row SENT.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

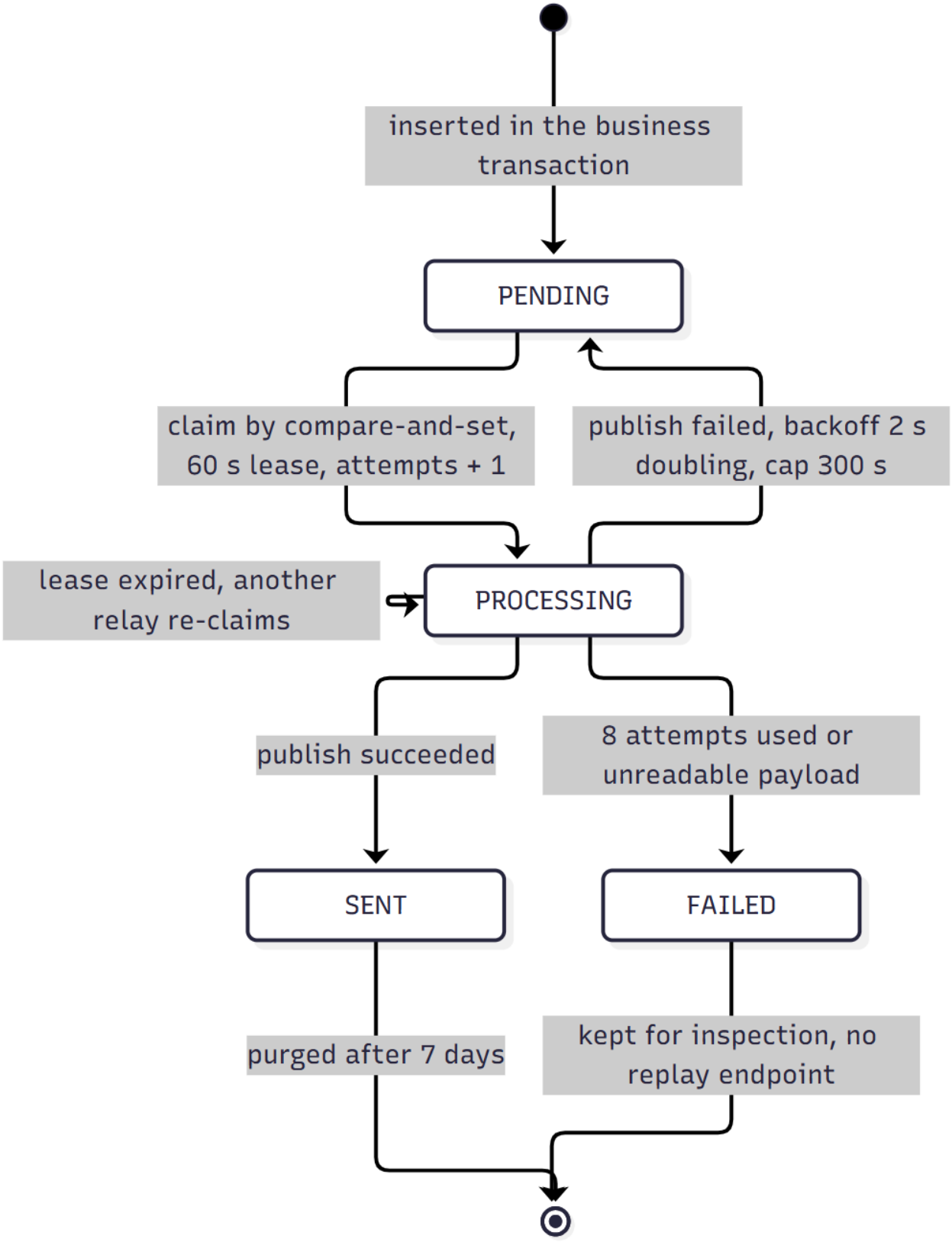


Figure 22: Outbox row state machine with claim, retry, lease re-claim, failure and purge transitions

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 22 is the row state machine. On failure the row returns to PENDING with the next attempt scheduled after `2 s x 2^(attempts-1)`, capped at 300 seconds. After 8 attempts, or immediately for an unreadable payload, the row becomes FAILED, a counter `cx.outbox.failed` is incremented and the row is kept for inspection. If a relay crashes mid-publish, the row stays PROCESSING; when the lease expires another relay re-claims it. That gives at-least-once delivery. `OutboxPurgeJob` runs hourly and deletes SENT rows older than seven days in batches of 500, at most 20 batches per run; PENDING, PROCESSING and FAILED rows are never purged. There is no replay endpoint for FAILED rows; the troubleshooting guide documents a manual SQL reset.

14.3 Event catalogue

Event type	Raised by	Typical recipients
ticket.created	ticket creation	active agent-level staff (up to 20) plus any assignee, excluding the actor
ticket.assigned	manual or workflow assignment	the assignee, excluding the actor
ticket.status.changed	transition	requester and assignee, excluding the actor
ticket.commented	PUBLIC or INTERNAL comment	assignee, plus the requester only for PUBLIC comments; INTERNAL comments are never announced to the requester
ticket.escalated	SLA job or ESCALATE action	active managers (up to 20) plus the assignee
ticket.sla.breached	SLA job, once per ticket	active managers (up to 20) plus the assignee
ai.suggestion.decided	approve or reject	the assignee if not the decider; ids and decision only, no reply text
ticket.workflow.notified	workflow NOTIFY action	extra type beyond the original seven; rendered by a generic template

14.4 Topology and consumer

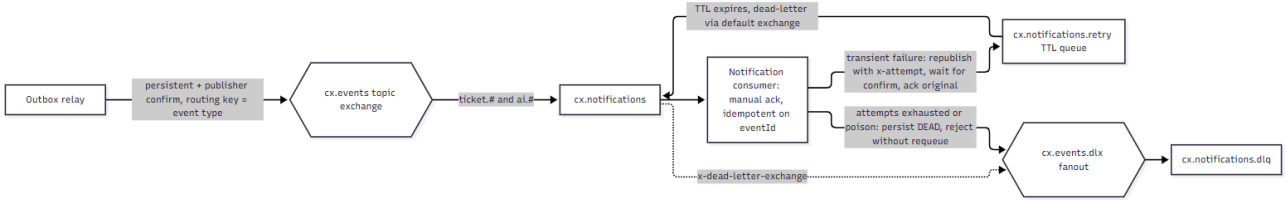


Figure 23: RabbitMQ topology with the topic exchange, notification queue, TTL retry queue, dead-letter exchange and DLQ

Figure 23 shows the broker layout declared by `NotificationRabbitConfiguration`. The relay publishes persistent messages with publisher confirms to the durable topic exchange `cx.events`, using the event type as routing key. The queue `cx.notifications` is bound with `ticket.#` and `ai.#` (not `.*`, because types such as `ticket.sla.breached` have three words) and declares `cx.events.dlx` as its dead-letter exchange. The consumer uses manual acknowledgements, prefetch 10 and 2 to 4 concurrent consumers by default, and applies this logic for each message:

Outcome	Action
Success or duplicate	basicAck
Unparseable or NonRetryableException (missing eventId, tenantId or type, payload tenant differs from envelope)	basicReject without requeue, so the broker dead-letters it to cx.notifications.dlq

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Outcome	Action
Retryable failure with attempts left	Republish persistently to cx.notifications.retry with header x-attempt + 1, wait for the publisher confirm, then ack the original; if the republish is not confirmed the original is requeued
Retries exhausted (5 attempts by default)	Persist status DEAD, then reject to the DLQ

The retry queue has a message TTL (5 seconds by default) and dead-letters through the default exchange straight back into `cx.notifications`, which is how the delay is implemented. If the multiplier were greater than 1, per-message expiration would be used instead of the fixed queue TTL. All of this was exercised only against a fake channel in `NotificationMessageHandlerTest` (ack, duplicate, poison, retry publish, retry exhaustion, unconfirmed publish); the real broker was never started.

14.5 Idempotent processing

`NotificationProcessor` is a plain class with no broker dependency, and it works in four stages. First it claims the `eventId` by inserting into `notif_processed_event`; a completed or dead event is a duplicate and a no-op. Second it resolves recipients from the payload and keeps only users of the event's tenant (checked against `app_user` through `JdbcRecipientDirectory`; if that table is unreachable the check abstains and only the payload-tenant check applies). Third it creates notification and delivery rows with unique keys `(event_id, user_id)` and `(event_id, user_id, channel)`, so a retry re-sends only channels not yet SENT. Fourth it marks the event done only when every delivery is SENT. Channels are IN_APP (real) and EMAIL (simulated: it records SENT and logs). Redelivery is therefore harmless (`NotificationProcessorTest`).

Ordering is not guaranteed across retries: a retried `ticket.assigned` can arrive after a later `ticket.status.changed`. Payloads are self-contained so consumers do not need order. Idempotency is per consumer, so any new consumer must implement it again.

14.6 Local mode: the embedded module

Without a broker, the same logic runs in-process. Core-service depends on the notification library jar, and `application-local.yml` sets `cx.notifications.embedded=true`. `NotificationAutoConfiguration` then registers the whole module: its own Flyway location (`db/notification-migration`, history table `notif_schema_history`), `notif_*` tables, uniquely prefixed beans and a controller advice limited to its own controller. `LocalEventPublisher` hands each event from the relay to every `EventListener` bean in the JVM; the module's `InProcessEventListener` calls the same `NotificationProcessor`, retries inline with the same `RetryPolicy`, persists DEAD when exhausted and never throws to the caller. The notification API is then served by core on port 8081, and the gateway's notifications route defaults to the core URI. `EmbeddedNotificationsTest` proves the chain from ticket creation through the outbox to a notification row readable through `GET /notifications`.

One consequence: the embedded listener runs synchronously on the relay thread, so a slow notification channel delays the relay in local mode only. In docker mode `application-docker.yml` sets embedded to false and the separate service consumes RabbitMQ.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

15. Caching

15.1 Design

`CacheService.getOrLoad(tenantId, key, ttl, type, loader)` implements tenant-keyed cache-aside. Keys are `t:{tenantId}:{key}`, built by one function, so a key without a tenant prefix cannot be produced by accident. Only DTOs and records are cached, never entities. There are two implementations: `CaffeineCacheService` (profile not docker; bounded by `cx.cache.max-entries`, default 10 000; per-entry TTL) and `RedisCacheService` (docker; JSON values; plus or minus 10 percent TTL jitter). Both provide per-node single-flight with one `CompletableFuture` per key, so 12 concurrent misses produce one load (tested). The loader runs outside any map lock, so nested cache use cannot deadlock.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

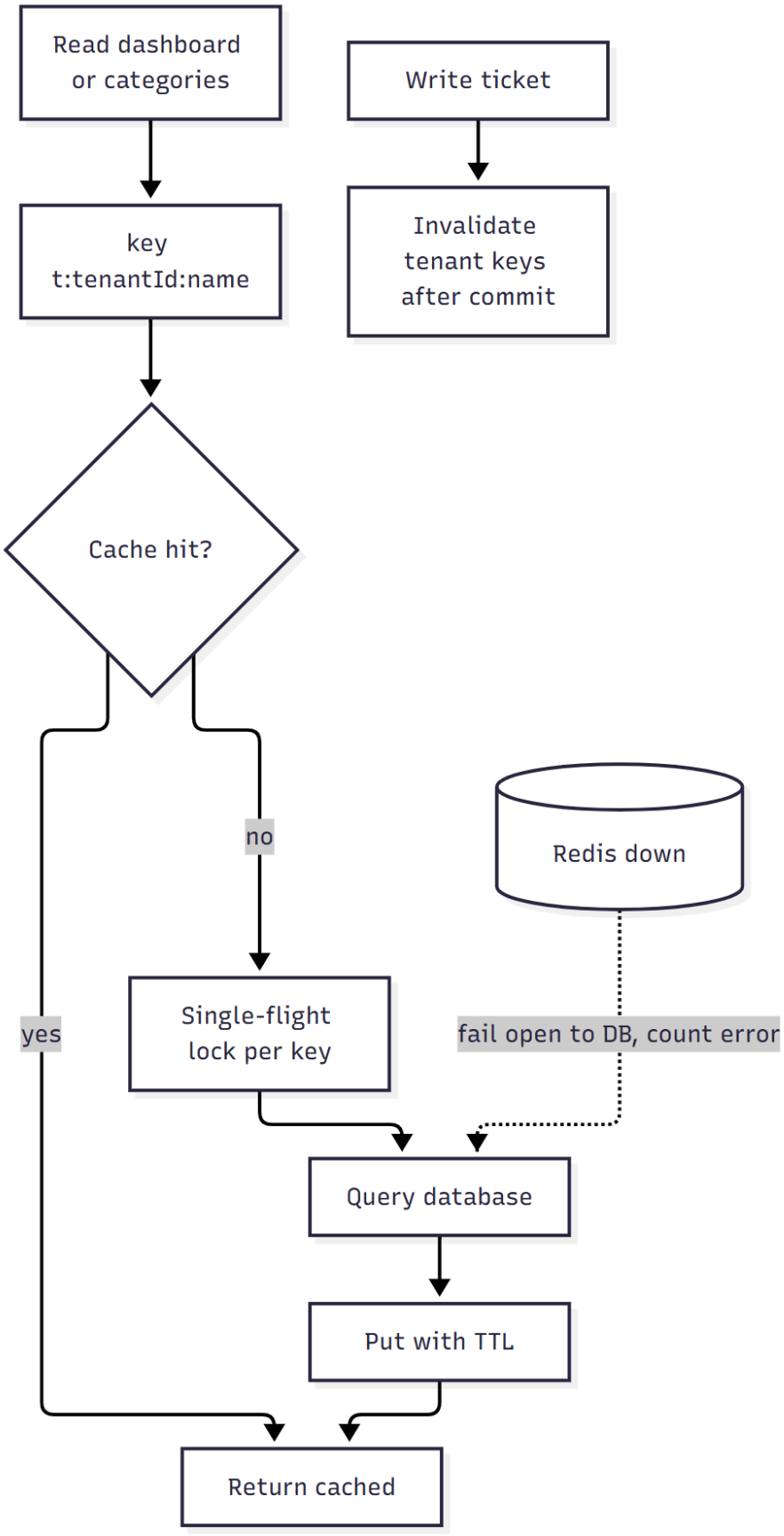


Figure 24: Cache-aside flow with single-flight loading, TTL, post-commit invalidation and Redis fail-open behaviour

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 24 shows the read and write paths. A read builds the tenant key and looks it up; on a miss a single-flight lock per key lets one caller query the database and store the result with a TTL while others wait for that result. A write evicts affected keys only after the transaction commits, using `TxHooks.afterCommit`, so a rollback never evicts and a reader cannot repopulate stale data between eviction and commit. The dotted path is the failure mode: if Redis is down, the loader runs, a fallback counter is incremented and a five second in-process breaker stops contacting Redis, so requests become slower, not failed.

15.2 What is cached

Data	Key stem	TTL	Evicted by
Manager dashboard	analytics:dashboard:{days}	60 s	any ticket write, comment, AI decision (by prefix, after commit)
Ticket categories	categories	5 min	category write
SLA policies	sla:policies	5 min	policy update
Tenant calendar settings	settings	5 min	settings update
Active workflow	workflow	5 min	activation
KB category list	kb categories	5 min	any KB write
KB PUBLISHED and PUBLIC list	per page, separate keys for staff and customer shapes	60 s	any KB write

Metric: `cx_cache_total{cache, result}` with cache labels `analytics`, `categories`, `sla`, `settings`, `workflow` and `kb`. Separate from `CacheService`, token revocation uses its own small Caffeine caches: revoked ids for 10 minutes and "not revoked" answers for 3 seconds (chapter 16).

15.3 Measured effect and caveats

On the local profile (H2, Caffeine, one warm JVM, 3 753 tickets in the tenant), a dashboard cache hit took a median 3.23 ms against 21.73 ms for a miss, about 6.7 times faster. Under the write-heavy load test the `analytics` cache hit ratio was 95.0 percent (31 983 hits, 1 697 misses), because writes evict the tenant's dashboard entries. Reference caches (settings, SLA, workflow, categories) are effectively always warm. These are local numbers; Redis adds a network round trip and JSON decoding, and that path was never executed.

15.4 Trade-offs

- Eviction on every ticket write makes the dashboard hit ratio depend on the write rate.
- Single-flight is per node: N nodes may each load once per TTL.
- `evictByPrefix` on Redis uses `SCAN` with a pattern, which is O(keyspace); acceptable for a few keys per tenant, not at large scale.
- A failed post-commit eviction leaves stale data until the TTL (at most 60 seconds for the dashboard, 5 minutes for the others).
- Only writes through this service invalidate the cache; direct database edits are invisible.
- The `local` Caffeine tier says nothing about Redis behaviour, and the Redis outage path was proven only with a mocked template.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

16. Authentication and authorization flow

16.1 Four different questions

The design keeps four concerns separate. **Authentication** asks who the caller is (login, JWT). **Tenant resolution** asks which tenant the caller belongs to (the `tid` claim, never client input). **Authorization** asks what the caller may do (roles and object ownership). **Data isolation** asks what rows the caller can reach (chapter 8).

16.2 Login and request flow

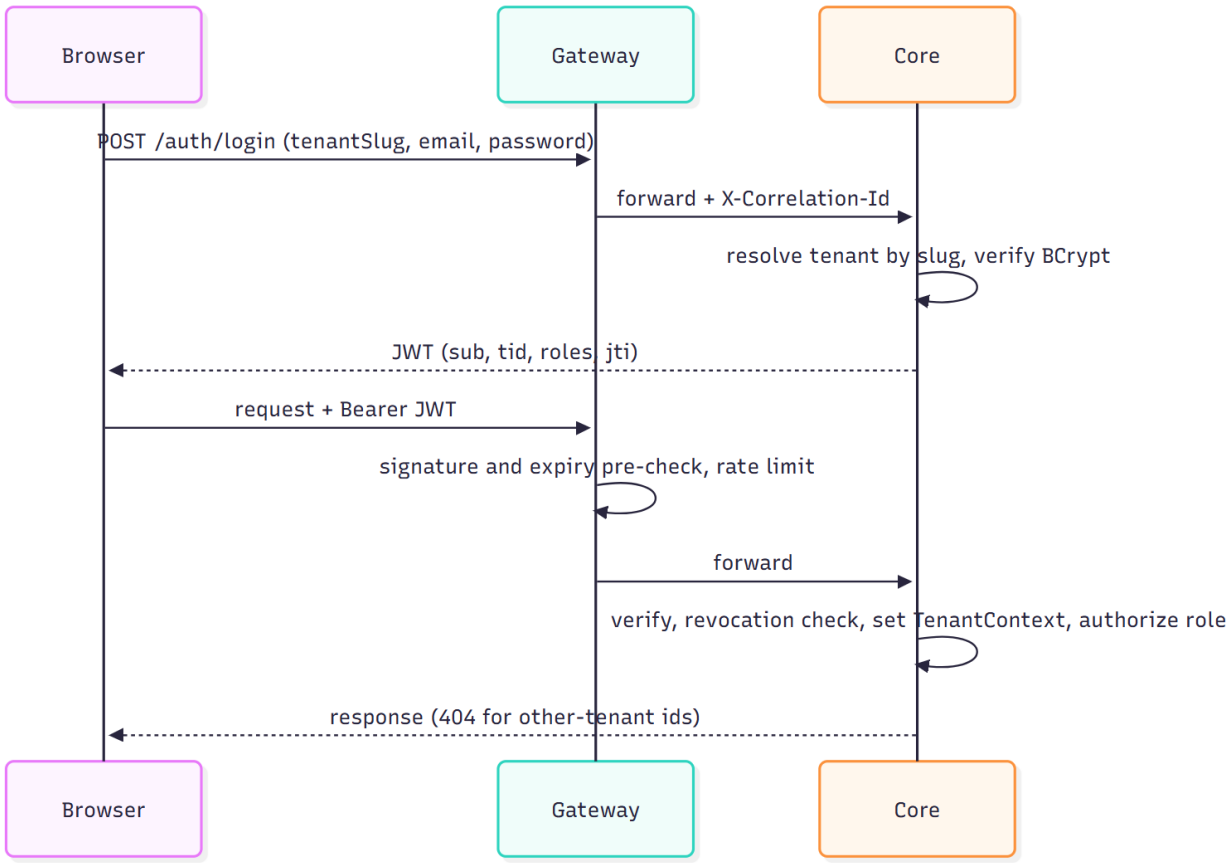


Figure 25: Authentication sequence from login through JWT issue, gateway pre-check and core verification

Figure 25 follows login and one authenticated call. The login body is `{tenantSlug, email, password}`. Core resolves the tenant by slug on the server, which only selects whose credentials to verify, then checks the password with BCrypt (cost 10). If the tenant or user is unknown, a dummy hash is verified so timing stays roughly constant, and the same generic "Invalid credentials" message is returned for every failure. A token-bucket limiter keyed by client address and by account protects login and registration (10 burst and 5 per minute by default, bounded in memory, 429 with `Retry-After`; the local profile is looser). Success and failure are audited.

The JWT is HS256 (jjwt 0.12.6) with claims `sub` (user id), `tid` (tenant id), `roles`, `email`, `jti`, `iss`, `iat` and `exp` (60 minutes). On each later call the gateway verifies signature and expiry with the shared secret, applies its rate limit and forwards. Core then verifies again (issuer and a 5 second clock skew), checks the `jti` against revocation, binds `TenantContext` from `tid`, builds the authorities and applies `@PreAuthorize`. Forged signatures, expired tokens, `alg=none` and tampered payloads all give 401 (`AuthApiTest`). The notification service holds the same secret and verifies the token itself.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

16.3 Authorization layers

Layer	Mechanism
Role hierarchy	ROLE_ADMIN > ROLE_MANAGER > ROLE_AGENT; CUSTOMER is outside the hierarchy
Method security	@PreAuthorize on every privileged endpoint, with a hierarchy-aware expression handler
Object level	findByIdAndTenantId everywhere; customers are also scoped to their own requester id; foreign objects give 404, never 403
Property level	DTO records only; customers get PUBLIC comments and a filtered activity list; roles editable only by ADMIN
Business rules	Customers may only reopen; last admin cannot be removed

16.4 Logout and revocation

Logout inserts the token's `jti` into `revoked_token`, a table shared by all nodes and purged after expiry. Checking that table on every request would cost a query per request, so `TokenRevocationService` keeps revoked ids cached for 10 minutes and caches "not revoked" for 3 seconds (`cx.security.revocation-negative-cache-seconds`, 0 turns it off). The node that handled the logout rejects the token immediately; other nodes reject it within the 3 second window. A single-node deployment is unaffected.

16.5 Known gaps

There is no refresh token and no key rotation. A symmetric key is held by core, gateway and notification service, so a leak of any one lets an attacker forge tokens for any tenant; an identity provider or RS256 with JWKS would limit verifiers to a public key. Deactivating a user does not revoke tokens already issued, and there is no tenant-level kill switch. The SPA keeps the token in `sessionStorage`, so cross-site scripting is the main browser risk; CSRF protection is disabled on purpose because the API is stateless and authenticates with a header that browsers never attach automatically. Registration of a new organisation is open, rate-limited and audited, with no verification step. The security hardening document maps the controls against the OWASP API Security Top 10 and lists the gaps against each item.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

17. Analytics and reporting

17.1 The dashboard contract

`GET /api/v1/analytics/dashboard?days=30` is available to MANAGER and ADMIN. The `days` parameter must be between 1 and 365; anything else is a validation error. The response is computed per tenant and cached for 60 seconds. A shortened example (all values illustrative):

```
{
  "openTickets": 79,
  "ticketsByPriority": { "LOW": 18, "MEDIUM": 30, "HIGH": 21, "URGENT": 10 },
  "avgFirstResponseMinutes": 814.7,
  "avgResolutionMinutes": 2210.4,
  "slaCompliancePercent": 81.3,
  "backlog": 41,
  "reopenedTickets": 6,
  "aiAcceptanceRatePercent": 50.0,
  "agentWorkload": [ { "agentId": "<uuid>", "name": "Agent One", "openCount": 27 } ],
  "resolutionTrend": [ { "date": "2026-10-15", "resolved": 4, "created": 6 } ],
  "tenantActivity": { "ticketsCreatedLast7d": 38, "commentsLast7d": 112, "activeUsers": 7 }
}
```

17.2 How each number is defined

Field	Definition in AnalyticsService
openTickets	Count of tenant tickets whose status is not RESOLVED or CLOSED
ticketsByPriority	Open tickets grouped by priority, with zero for missing priorities
backlog	Open tickets created more than seven days ago
avgFirstResponseMinutes	Mean minutes from creation to firstRespondedAt for tickets created in the window that have a response
avgResolutionMinutes	Mean minutes from creation to resolvedAt for tickets resolved in the window
slaCompliancePercent	Share of tickets resolved in the window whose SLA status is MET; 100 when none were resolved
reopenedTickets	Tickets created in the window with a reopen count above zero
aiAcceptanceRatePercent	Approved plus edited-approved over all decided suggestions (0 without decisions); supplied through the AiAcceptanceProvider extension point
agentWorkload	Open tickets per assignee, at most 50 rows, with display names resolved in one batched query
resolutionTrend	One point per UTC date in the window with resolved and created counts
tenantActivity	Tickets and comments created in the last seven days and active users in that period

17.3 Live versus precomputed

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Counts and group-bys are live, index-backed aggregate queries. The window metrics (averages, compliance, trend) read a bounded narrow projection of at most 50 000 rows with five columns and fold it in memory. That is fine at the small to medium tenant sizes assumed here, and it is documented in the class comment that the scale-up path is a precomputed rollup table maintained from outbox events, with the same cache in front. A cache miss runs about nine queries; the measured median was 21.7 ms at 3 753 tickets locally and it grows with tenant size. The percentages are rounded to one decimal.

Two caveats are worth stating to a reader. The trend's date buckets follow the tenant's time zone (an earlier version used UTC days; fixed and covered by `AnalyticsTimeZoneTest`, and an unknown stored zone falls back to UTC). And because any ticket write evicts the dashboard entry, a busy tenant will mostly see fresh numbers while a quiet tenant sees numbers up to 60 seconds old.

17.4 Reporting beyond the dashboard

There is no export, no scheduled report and no custom report builder. The `audit-events` endpoint (ADMIN) supports paging with `entityType` and `actorId` filters and serves as the operational history. Prometheus metrics (chapter 18) provide the operational numbers: ticket creation count, cache hit ratios, outbox counters, AI outcomes and SLA job runs.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

18. Local infrastructure and observability

18.1 What runs locally

The executed local runtime needs no external software. `scripts/run-local.sh` starts core-service with embedded notifications on 8081, the gateway on 8080 and the Vite dev server on 5173 (the script was written but not run end to end by its author; the pieces were run individually). The seeder creates three tenants with 150, 100 and 60 deterministic synthetic tickets over the last 45 days, with comments and activity, plus 12 knowledge base articles per tenant.

18.2 The Docker Compose stack

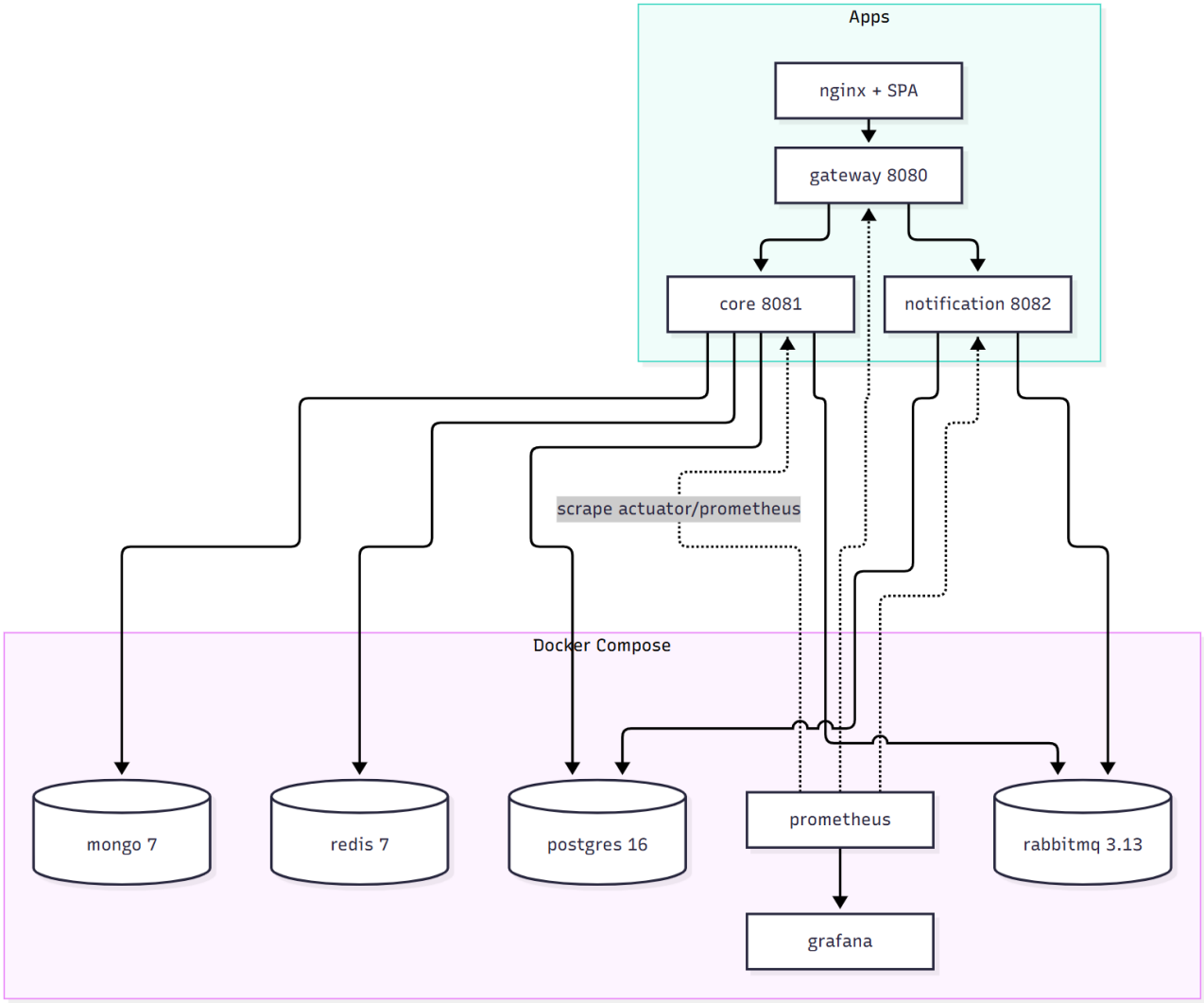


Figure 26: Docker Compose topology with PostgreSQL, MongoDB, Redis, RabbitMQ, Prometheus, Grafana and the four application containers

Figure 26 shows the unexecuted compose stack. The infrastructure services are `postgres:16`, `mongo:7`, `redis:7`, `rabbitmq:3.13-management`, `prom/prometheus:v2.54.1` and `grafana/grafana:11.2.2`. Application services (core, notification, gateway and an nginx-served front end) belong to the compose profile `apps`, so infrastructure can be started alone. Prometheus scrapes core, notification, the gateway management port 8090 and the RabbitMQ Prometheus plugin on 15692.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Property	Value
Published ports	All bound to 127.0.0.1
Required secrets	Declared as ${VAR:?...}, so compose refuses to start when they are missing
App images	Runtime-only (a pre-built jar is copied in), non-root user app, JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError
Front-end image	Built with Node, served by unprivileged nginx with X-Content-Type-Options, X-Frame-Options DENY and Referrer-Policy no-referrer; hashed assets are cached for a year
Variable names	The same CX_DB_*, CX_RABBIT_*, CX_JWT_SECRET names are used by core and notification service; compose derives them from env.example; CX_MONGO_URI is built from the Mongo root credentials and passed to core
AI settings	AI_PROVIDER, AI_API_URL, AI_API_KEY with the CX_AI_* names accepted as aliases; empty values mean the mock provider
Verification status	Not run. Only YAML and JSON syntax were parsed

> Images use floating tags such as `eclipse-temurin:21-jre`, the compose file uses one database role that owns the schema, and seeding is enabled by default in the docker profile (`CX_SEED_ENABLED` defaults to true). Section 18.5 and chapter 23 list what to change before anything beyond a local experiment.

18.3 Observability design

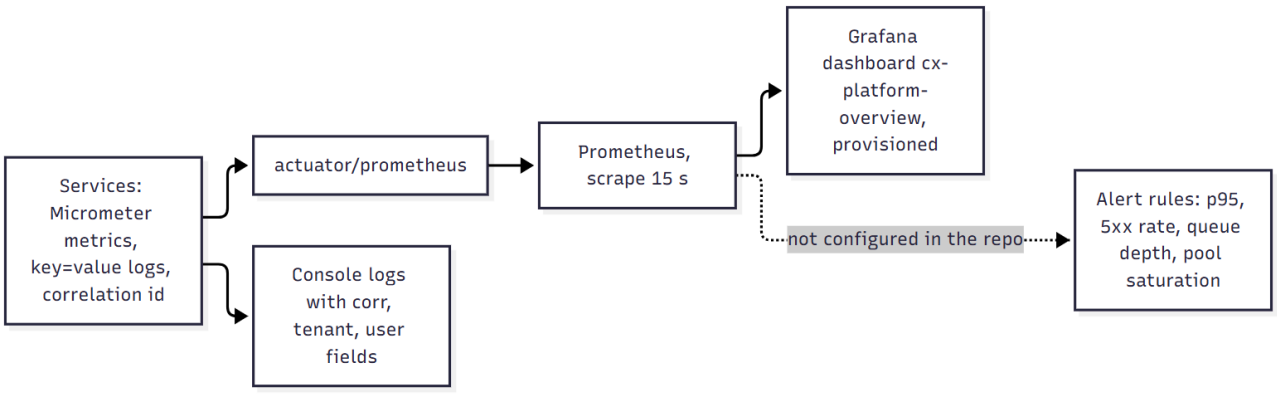


Figure 27: Observability flow from Micrometer metrics and logs through Prometheus to Grafana, with alert rules not yet configured

Figure 27 shows what exists and what does not. Each service exposes Micrometer metrics at `/actuator/prometheus`; Prometheus scrapes every 15 seconds; a provisioned Grafana dashboard (`cx-platform-overview.json`) shows request rate, 5xx, latency, Hikari pool state, cache hit ratio, RabbitMQ queue depth, notification outcomes, gateway counters and JVM metrics. The dashed box marks alert rules: they are described in the runbooks but are not configured in the repository. Logs are plain key=value console lines.

Signal	Detail
Log line	ts level corr tenant user logger msg, from MDC fields; the http.access line records method, path, status and duration only, never bodies, headers or query strings
Correlation id	Echoed in X-Correlation-Id, in error bodies, in audit_event.correlation_id and in event headers

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Signal	Detail
Health	/actuator/health, /actuator/health/readiness, /actuator/health/liveness
HTTP metrics	http_server_requests_seconds_*; histogram buckets are off by default, so the p95 panel stays empty until management.metrics.distribution.percentiles-histogram.http.server.requests=true is set
Pool	hikaricp_connections_active, idle, pending, max, acquire_seconds, usage_seconds, timeout_total
Cache	cx_cache_total{cache,result}, cx_cache_fallback_total{op}
Outbox	cx_outbox_enqueued_total, published, retried, failed, purged
Domain	cx_tickets_total, cx_auth_login_total{result}, cx_sla_job_runs_total, cx_sla_job_errors_total, cx_workflow_fallback_total{reason}, cx_ai_analyze_total{result}, cx_ai_meta_failed_total
Notifications	cx_notifications_events_total{result} with results processed, duplicate, no_recipients, recipient_rejected, failed, dead
Gateway	gateway_proxied_total{route,status}, gateway_rate_limited_total, gateway_upstream_errors_total{route,type}

The `local` profile has Prometheus endpoints and the core platform test asserts that `cx_cache_total{result}` appears there. Not present anywhere: distributed tracing, alert rules, SLO burn alerts and log shipping.

18.4 Testing and measurement tools

Tool	Purpose	Status
JUnit 5, MockMvc, H2	179 core tests	Executed
core-service/scripts/smoke-test.mjs	57 HTTP checks against a running local app	Executed
perf/load-test.mjs	Dependency-free load generator for ticket list, dashboard, ticket create and a mixed scenario	Executed on local profile
perf/dashboard-cache-probe.mjs	Times cache hit versus miss	Executed logic; the committed file was only syntax-checked
perf/sql/H2Explain.java	Prints index choice and rows examined for key queries on H2	Executed
perf/k6/*.js	Load scripts	Not executed (k6 not installed)
Testcontainers tests	PostgreSQL RLS, RabbitMQ and Redis, MongoDB, notification broker	Compiled only
Vitest, Karma, Playwright, Angular smoke	Front-end unit and browser tests	Executed 2026-10-17: 86/86, 83/83, 17/17 (16 + 1 skipped without the fault stack), 25/25

18.5 What would change before sharing the stack

Disable seeding with `CX_SEED_ENABLED=false`; protect or disable Swagger and the Prometheus endpoint on public ports; run the docker profile end to end on a Docker host and fix what breaks; separate the migration role from the runtime role and avoid `BYPASSRLS`; pin image digests and scan images; add alert rules; put TLS in front. These items are traced to their sources in the security hardening notes.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

19. End-to-end business scenarios

All scenarios use the seeded demo tenants. Identifiers are shown as placeholders, token values are never printed, and values such as ticket numbers or timestamps are illustrative. Requests go through the gateway at `http://localhost:8080`. The demo password is the published synthetic seed value listed in `docs/API-CONTRACT.md`.

19.1 Scenario A: a customer reports a security issue and double-clicks submit

Business context. A customer at Acme Retail sees a login alert and files a ticket. The tenant's default workflow raises tickets whose title mentions "security" to HIGH and routes HIGH tickets to the least-loaded agent.

Step 1: log in.

```
POST /api/v1/auth/login
Content-Type: application/json

{ "tenantSlug": "acme-retail", "email": "customer1@acme.test", "password": "<seed demo password>" }
```

The response carries `accessToken`, `expiresInSeconds: 3600` and a `user` object with `roles: ["CUSTOMER"]` and the tenant. The token is held in memory and `sessionStorage`.

Step 2: create the ticket. The form generated one idempotency key when it mounted.

```
POST /api/v1/tickets
Authorization: Bearer <JWT>
Idempotency-Key: 5d0c3b1e-demo-key-001
Content-Type: application/json

{
  "type": "SUPPORT",
  "title": "Security alert on my account",
  "description": "I received a login alert from a device I do not recognise.",
  "priority": "MEDIUM",
  "severity": "MAJOR",
  "categoryId": "<Account Access category id>",
  "tags": []
}
```

Expected result (shortened): `201 Created` with `status: "NEW"`, `priority: "HIGH"` (raised by rule 1), `tags: ["security"]`, an `assignee` chosen by rule 3 (the active agent with the fewest open tickets), `slaStatus: "ON_TRACK"`, `firstResponseDueAt` and `resolutionDueAt` computed from the HIGH policy (60 and 480 business minutes in the tenant's calendar), `version: 0`, and a per-tenant `number`. Because the priority changed during creation, due dates were recomputed for HIGH.

Step 3: the browser retries (double-click or network timeout) with the same key and body. The server replays the original 201 body and adds `Idempotent-Replay: true`; no second ticket exists. If the same key is sent with a different title, the answer is `409` with code `DUPLICATE_SUBMISSION`.

Side effects: in one transaction the service wrote the ticket, an activity row `CREATED`, workflow activity rows, outbox events `ticket.created` and `ticket.assigned`, and an audit row `TICKET_CREATED`. Within about a second the relay delivers the events; the assigned agent finds an in-app notification at `GET /api/v1/notifications?unread=true`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

19.2 Scenario B: two agents race to move the same ticket

Business context. Agent One and Agent Two both have the ticket open.

```
POST /api/v1/tickets/<ticketId>/transition
Authorization: Bearer <agent1 JWT>

{ "toStatus": "OPEN", "reason": "Picked up", "version": 0 }
```

Agent One receives 200; the ticket now has `status: "OPEN"` and `version: 1`. Agent Two, still holding version 0, sends:

```
POST /api/v1/tickets/<ticketId>/transition
Authorization: Bearer <agent2 JWT>

{ "toStatus": "PENDING", "reason": "Waiting for customer", "version": 0 }
```

```
{
  "timestamp": "2026-10-16T19:41:07Z",
  "status": 409,
  "error": "CONFLICT",
  "code": "TICKET_VERSION_CONFLICT",
  "message": "Ticket was modified by someone else (current version 1); reload and retry",
  "path": "/api/v1/tickets/<ticketId>/transition",
  "correlationId": "<id>"
}
```

The UI shows a conflict notice with "Reload latest". After reloading, Agent Two moves OPEN to PENDING with version 1. That transition pauses the SLA clock (status PAUSED) and notifies the requester. A test proves the same behaviour at the HTTP level: two parallel updates with the same version produce exactly one 200 and one 409.

If an agent tries OPEN to CLOSED, the result is `422 INVALID_TRANSITION`, because OPEN allows only PENDING, ON_HOLD and RESOLVED.

19.3 Scenario C: AI-assisted reply with human approval

Business context. An agent opens a billing ticket: "Charged twice for my subscription".

```
POST /api/v1/tickets/<ticketId>/ai/analyze
Authorization: Bearer <agent JWT>
```

The response is `200` with `provider: "MOCK_RULE_BASED"`, `generated: true`, category Billing, a suggested priority, a confidence value (for example 0.66), an explanation naming matched keywords, up to three recommended published articles and a templated reply. The ticket itself is unchanged and no comment exists. The agent reads the draft, edits one sentence and approves:

```
POST /api/v1/ai/suggestions/<suggestionId>/approve
Authorization: Bearer <agent JWT>
Content-Type: application/json

{ "editedReply": "Thanks for reporting this. We have found the duplicate charge and started a refund.", "visib
  ility": "PUBLIC" }
```

Result: status `EDITED_APPROVED`, exactly one PUBLIC comment with `aiGenerated: true` authored by the agent, a first-response timestamp if this is the first staff reply, an activity entry "PUBLIC comment (AI-assisted, human approved)", audit action `AI_SUGGESTION_EDITED_APPROVED`, an outbox event `ai.suggestion.decided` and a `ticket.commented` event that notifies the requester. The dashboard cache for the tenant is evicted so `aiAcceptanceRatePercent` updates. A second approve or reject of the same suggestion returns 409.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

If the analyze call had failed (provider timeout, open breaker), the answer would be `200` with `degraded: true`, the fixed fallback message and no reply text, and the agent would simply write the reply by hand.

19.4 Scenario D: a ticket breaches its SLA over a weekend

Business context. The Friday 15:30 HIGH ticket from chapter 11 is created and nobody replies.

1. At Friday 16:30 local (first response due) the next scan finds no staff reply and the first-response due date in the past. `slaStatus` becomes BREACHED with breach type FIRST_RESPONSE; `breach_notified` flips once; the job writes activity "SLA breached (FIRST_RESPONSE)" and an outbox event `ticket.sla.breached` addressed to active managers and the assignee.
2. The business-minute percent used crosses 75 on Monday at 13:30 (not on Sunday evening: closed hours consume nothing); the escalation level rises to 1 and `ticket.escalated` is published once. At 100 percent (Monday 15:30) it rises to 2; no second breach event is published when the resolution date passes, because the breach flag was already set by the first-response breach.
3. Managers see the notifications in their bell (the SPA polls every 30 seconds) and the ticket in the dashboard's SLA compliance figure once it is resolved late.
4. When an agent finally resolves the ticket, the status evaluates to BREACHED rather than MET because `breach_notified` was already true.

Re-running the job, or running it on two nodes, publishes none of these events twice. The unit and integration tests `SlaJobTest` and `SlaCalculatorTest` cover the breach, the once-only flag, pause and fair rotation.

19.5 Scenario E: a tenant administrator changes the workflow

Business context. The Acme admin wants every URGENT ticket tagged and assigned to the least-loaded agent, and wants to test the rule first.

```
POST /api/v1/workflows/validate
Authorization: Bearer <admin JWT>
Content-Type: application/json

{
  "name": "Urgent routing",
  "rules": [ { "name": "Urgent to agent", "when": [ { "field": "priority", "op": "EQ", "value": "URGENT" } ],
               "then": [ { "action": "ADD_TAG", "params": { "tag": "urgent" } },
                         { "action": "ASSIGN_TO_ROLE_LEAST_LOADED", "params": { "role": "AGENT" } } ] } ],
  "transitions": []
}
```

A valid definition returns `{ "valid": true, "errors": [] }`. An unknown operator such as `STARTS_WITH`, an unknown action, a value containing `${` or `#{`, or more than 50 rules returns `valid: false` with messages. The admin then posts the same body to `POST /api/v1/workflows`, which creates a new inactive version, and activates it with `POST /api/v1/workflows/<id>/activate`, which flips the single active version and writes audit events. New tickets use it immediately (the active workflow is cached for five minutes and evicted on activation). If the stored definition were ever corrupted, ticket handling would fall back to the built-in default and increment `cx_workflow_fallback_total`.

19.6 Scenario F: another tenant probes for data

Business context. An agent at Globex IT obtains an Acme ticket id (for example from a leaked log) and tries to read it, forging a tenant header.

```
GET /api/v1/tickets/<acme ticket id>
Authorization: Bearer <globex agent JWT>
X-Tenant-Id: <acme tenant id>
```

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The tenant is taken from the token's `tid` claim and the header is ignored. The tenant-scoped lookup finds nothing and the response is `404 NOT_FOUND`, the same answer as for an id that never existed, so existence is not leaked. The same applies to update, transition, assign, comment, AI endpoints, knowledge base articles and every list endpoint, where the other tenant's rows simply never appear. `TenantIsolationTest`, the smoke test and the Playwright spec `tenant-isolation.spec.ts` all assert this (the Playwright suite has no recorded run).

19.7 Scenario G: a customer finds an answer in the knowledge base

A customer opens the portal and searches `GET /api/v1/articles?q=vpn&size=10`. Only PUBLISHED and PUBLIC articles are considered, ranked by title, summary and body matches. An internal runbook about the same topic and a draft article are invisible; requesting one by id gives 404. An agent who writes a new article creates a DRAFT (version 1), edits it (version 2) and publishes it (version 3); each change writes an immutable snapshot, visible at `GET /api/v1/articles/<id>/versions`. After publishing, the next customer search finds it, because the cached public list is evicted after the commit.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

20. Failure and recovery workflows

20.1 Failure catalogue

Failure	How it is detected	Behaviour of the system	Data risk	Recovery	Verified by
Redis down	cx_cache_fallback_total rises, WARN log "Redis unavailable"	Loader runs, 5 second breaker, requests slower not failing	None	Restore Redis; entries refill on demand	Mocked template test; real Redis not executed
RabbitMQ down	cx_outbox_retried_total rises, outbox PENDING rows grow	Relay retries with backoff 2 s to 5 min, FAILED after 8 attempts	None while rows are not FAILED	Restore broker; reset FAILED rows by SQL	Relay behaviour tested; broker not executed
Notification consumer down	Queue depth grows	Messages wait durably	None	Restart consumer	Not executed
Poison message	DLQ depth, result="dead"	Straight to DLQ, DEAD state stored	Message parked	Inspect, fix, republish	Fake-channel test
PostgreSQL down	Readiness fails, Hikari timeouts	Requests fail; no partial commits	None	Restore database	Not executed
Connection pool exhausted	hikaricp_connections_pending > 0	Requests queue then time out	None	Find the slow transaction first, then tune	Runbook only
AI provider timeout	degraded: true, cx_ai_analyze_total{result="degraded"}	HTTP 200 degraded, no reply text	None	Fix provider or switch to mock	AiApiTest, AiResilienceUnitTest
Core node crash mid-relay	Row stays PROCESSING	Lease expires after 60 s, another relay re-claims	Duplicate delivery possible	Consumers dedupe	Expired-lease test
Invalid workflow stored	cx_workflow_fallback_total	Built-in default used	None	Fix and re-activate	WorkflowApiTest
Stale cache after missed eviction	Users report stale dashboard	Stale until TTL (60 s or 5 min)	Stale read only	Wait or evict	Design
JWT secret mismatch between services	Every authenticated call 401 at the gateway	Calls fail at the edge	None	Align CX_JWT_SECRET; users re-login	Runbook
Retried comment POST	Duplicate comment	A second comment is created	Duplicate content	None built in (comments are not idempotent)	Documented gap
Gateway down	External probe	All API access lost	None	Replicas; gateway is tier-0	Not applicable locally
Duplicate submit of ticket creation	Same key twice	Original response replayed	None	None needed	TicketApiTest

20.2 Recovery procedures

Reset FAILED outbox rows after the cause is fixed. There is no replay endpoint. Identify the rows (`status = 'FAILED'`), confirm why they failed (`last_error`), then set `status = 'PENDING'`, `attempts = 0` and `next_attempt_at = now()` for the chosen ids. Consumers de-duplicate on `eventId`, so a re-sent event that was already processed is a no-op. This procedure is documented, not executed on PostgreSQL.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Drain the DLQ. Inspect messages in `cx.notifications.dlq`, fix the root cause (a bad payload shape or a consumer bug), then re-publish through a shovel or by hand to `cx.events`. Idempotency makes replay safe. Not executed.

Rotate the JWT secret. Set the same new value for core, gateway and notification service and restart. Every existing token becomes invalid and all users log in again. There is no overlap period or key identifier.

Suspected cross-tenant exposure (severity 1). Get the correlation id and the entity id; reproduce with the reporter's token (the expected answer is 404); run the guard tests (`TenantIsolationTest`, `TenancyTest`, `ArchitectureRulesTest`); search for bypasses (`JdbcTemplate`, native queries, `findById(` without a tenant check); on PostgreSQL verify RLS is on, forced and that the role is not a superuser and has no `BYPASSRLS`; contain by revoking tokens or rotating the secret; fix the predicate and add a regression test. The PostgreSQL steps were never executed.

AI degraded for a long time. Check `reason` values in `AI_ANALYSIS_DEGRADED` audit events, correct the provider key or URL, or unset `cx.ai.provider` to return to the mock provider immediately; agents are never blocked.

Slow API. Check in order: pool queuing, a missing tenant predicate or index, cache misses or Redis fallbacks, stuck transactions, large-tenant dashboard misses, an N+1 regression, then GC or CPU saturation. The troubleshooting guide holds ten runbooks with concrete metrics and commands; only the ones against the local app were exercised.

20.3 What the tests prove about failure

Behaviour	Test
Event not written when the business transaction rolls back	OutboxTest
Retry with backoff succeeds on the second attempt; FAILED at the maximum	OutboxTest
Exactly one of six concurrent claimers wins; an expired lease is re-claimed	OutboxTest
Purge deletes only old SENT rows	OutboxPurgeTest
Cache loader result returned when the (mocked) Redis throws; breaker stops further calls	CacheServiceTest
Provider FAIL gives HTTP 200 degraded after exactly 3 attempts; TIMEOUT gives degraded in under 4 seconds; breaker opens and stops calling	AiApiTest, AiBreakerAndProviderSelectionTest
Corrupt stored workflow falls back to the default	WorkflowApiTest
Duplicate and poison events handled; retries and exhaustion	NotificationProcessorTest, NotificationMessageHandlerTest
Upstream down gives 503; rate limit gives 429 with Retry-After	gateway tests

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

21. Performance bottlenecks and optimisation decisions

> Every number in this chapter was measured on one Windows laptop (Intel Core i5-1135G7, 4 cores and 8 threads, GraalVM JDK 21.0.7, default G1, no JVM tuning) on 2026-10-17, with the load generator on the same machine, against core-service on profile `local` (H2 in memory, Caffeine, in-JVM events), directly on port 8093 or 8094 (no gateway, no TLS, no network). They are **local** numbers. They are not production figures, not service-level objectives and not Freshworks metrics. Nothing here was measured on PostgreSQL, Redis, RabbitMQ or MongoDB.

21.1 Baseline load runs

Command shape: `node perf/load-test.mjs`, 20 second window after a 3 second warm-up, concurrency 10 unless stated, zero errors in every run.

Scenario	Tenant size at start	Requests	RPS	p50 ms	p95 ms	p99 ms	max ms
Ticket list, agent, page size 20	150	9 516	475.5	14.0	58.4	91.3	240.4
Dashboard, manager	150	28 061	1 402.1	5.1	17.9	37.6	160.5
Ticket create (unique key), manager, before the numbering fix	150	2 085	104.0	28.9	200.6	1 943.5	7 707.1
Mixed: 70 percent list, 20 percent dashboard, 10 percent create	about 2 235	11 849	592.1	11.9	49.2	82.1	174.6
Ticket list, concurrency 1	about 3 400	3 292	219.4	4.2	6.6	8.3	17.1

Reading the table. With one request in flight the median list latency is 4.2 ms; with ten it is 14.0 ms. Throughput rose only 2.2 times for ten times the concurrency, so the laptop (four cores shared with the load generator) was already close to saturation, and the growth in latency is queuing, not slower code. The dashboard's three-fold throughput advantage over the list is an effect of caching, not of the dashboard being cheaper to compute: a miss is slower than a list page (section 21.3). These were single runs, so run-to-run variance is unknown except for ticket create, where it was large.

21.2 The ticket-create tail: hypothesis, experiment, fix

The first ticket-create runs showed a heavy tail at concurrency (c=1: max 44.9 ms; c=10: p99 about 1 082 ms and max 2 828 ms). Two checks ruled out the usual suspects. `jvm_gc_pause_seconds_max` was 0.010 seconds, so garbage collection did not explain multi-second tails. The Hikari pool was not exhausted: maximum acquire time was 0.106 seconds and there were no timeouts. However `hikaricp_connections_usage_seconds_max` was 6.077 seconds, meaning some transaction held its connection for six seconds, which points at waiting on a lock rather than on the pool.

The hypothesis was the per-tenant `ticket_counter` row lock: every create took `SELECT ... FOR UPDATE` on that row and held it until the whole create transaction committed (ticket, activity, audit, outbox, idempotency rows), so creates of one tenant queued behind each other. A differential experiment tested it. Different tenants have different counter rows, so spreading the same ten clients across three tenants removes exactly that contention and nothing else.

Before the fix	p99 ms	max ms
10 clients, one tenant (four runs)	261 to 1 033	506 to 3 472

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Before the fix	p99 ms	max ms
20 clients, one tenant	69 to 101	12 753 to 12 940
10 clients spread over three tenants (4+3+3)	45 to 72 per tenant	at most about 203

After the fix	p99 ms	max ms
10 clients, one tenant	81 to 84	151 to 220
20 clients, one tenant	108 to 133	186 to 230
10 clients spread over three tenants	79 to 93 per tenant	209 to 254

Conclusion: the counter lock was the dominant cause of the tail on this setup, confirmed by the differential experiment, not by a lock trace. The fix is `TicketNumberAllocator`: one atomic `UPDATE ... next_number = next_number + 1` in its own tiny transaction before the business transaction begins, so the lock is held for well under a millisecond. The price is that numbers are no longer gap-free. Single-tenant c=20 p99 got slightly worse (69 to 101 ms before, 108 to 133 ms after) because more requests now truly run in parallel and compete for the four cores and the pool of 10. Caveats: one laptop, H2 (a PostgreSQL row lock behaves similarly in kind but with different numbers), one or two runs per row, and the before and after series ran on different JVM instances.

21.3 Cache hit versus miss

Run	JVM state	Tickets in tenant	Hit median (range)	Miss median (range)
A	Fresh JVM, about 25 s after start	150	11.02 ms (9.30 to 16.21), n=30	26.81 ms (18.53 to 47.46), n=15
B	Warm, after about 50 000 earlier requests	3 753	3.23 ms (2.36 to 5.16), n=30	21.73 ms (19.72 to 23.61), n=15

On the warm JVM a hit was 6.7 times faster than a miss. The first call on a cold JVM was 119.5 ms (cold JIT plus cache miss). Under the write-heavy load test the cache counters read 31 983 hits and 1 697 misses, a 95.0 percent hit ratio for the analytics cache, because create and mixed scenarios evict the tenant's entry on every write.

21.4 Query shape evidence (H2 only)

`perf/sql/H2Explain.java` loaded the real migrations V1 to V15 into H2 2.2.224 (PostgreSQL mode) with three tenants of 50 000, 20 000 and 5 000 tickets and reported rows examined. Wall-clock timings were discarded as unreliable at this size.

Query	Index chosen	Rows examined	Reading
List page: tenant, status OPEN, order by created desc, limit 20	ix_ticket_sla_scan	14 287	Index serves the filter, but all of the tenant's OPEN rows are read and then sorted
Same on a table copy without indexes	table scan	75 001	5.2 times more rows
Status filter without the tenant predicate	table scan	75 001	Tenant-first indexes are useless without a tenant predicate
Contains search on lower(title)	tenant index	50 001	A leading wildcard cannot use a btree
Offset page at offset 40 000	tenant index	50 001	Reads and discards 40 000 rows
Keyset page at the same position	ix_ticket_tenant_created	10 001	5 times fewer rows

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Query	Index chosen	Rows examined	Reading
Open count by priority (dashboard)	ix_ticket_tenant_status_prio_created	50 001	The NOT IN status range spans the tenant index
List query with an extra candidate index	H2 still chose the old index	14 287	No improvement shown on H2; neither proves nor disproves it for PostgreSQL

Safe conclusions: indexes lead with `tenant_id` and need a tenant predicate; contains-search and deep offsets scale with tenant size; a keyset form examines fewer rows. Unsafe conclusions: any speed-up figure, or that the candidate index helps. The PostgreSQL `EXPLAIN (ANALYZE, BUFFERS)` commands to settle this are written in the troubleshooting guide and were not executed.

21.5 N+1 evidence

`NPlusOneGuardTest` counts JDBC statements prepared by Hibernate for `GET /tickets?size=2` and `size=30`. Re-run with a print added, it reported **6 statements for 2 rows and 6 statements for 30 rows**: page query, count query, actor lookup, users `IN`, categories `IN` and attachments `IN`. The test asserts at most 8 and at most 6 for a single-ticket read. It holds because entities store ids as plain columns and `TicketMapper.toDtos` resolves names with one batched query per kind. A naive mapper doing a user lookup for requester and assignee and a category lookup per row would issue `1 + 3N` statements, 91 for N=30 (arithmetic, hypothetical).

21.6 Front-end bundle

First visit to `/login` loads about 104 kB gzip (react 53.7, forms 26.4, query 12.5, entry 11.6); the ticket detail page adds 5.0 kB gzip, the workflows page 2.8 and the dashboard page 2.3. These came from the front-end build report and the raw file sizes were cross-checked. Not measured: real-device load time, Largest Contentful Paint and runtime render cost.

21.7 Decisions and their price

Decision	Evidence	Price paid
Allocate ticket numbers in a separate tiny transaction	Single-tenant c=10 p99 from 261 to 1 033 ms down to 81 to 84 ms	Gaps in numbers; only TicketCommandService may call the allocator, and never with a transaction open
Cache-aside dashboard with 60 second TTL and post-commit eviction	Hit 3.23 ms versus miss 21.73 ms	Hit ratio depends on write rate; stale up to 60 seconds if an eviction fails
Batched name lookups, ids as plain columns	6 statements regardless of page size	No lazy associations; mapping is explicit
Tenant-first composite indexes	Scan of 75 001 rows without a tenant predicate	Every query must carry the tenant
Negative revocation cache of 3 seconds	Reduces a query per request to about one per 3 seconds per node	Up to 3 seconds of delayed logout on other nodes
Bounded paging (size at most 100)	Unbounded queries impossible by construction	Deep offsets still cost; the page number itself is not capped
Outbox purge in bounded batches	Prevents unbounded table growth	Only SENT rows are removed

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

21.8 Not measured

Any PostgreSQL, Redis, RabbitMQ or MongoDB latency; gateway throughput and added latency; notification service throughput; end-to-end event latency from outbox to in-app notification; behaviour above about 4 000 tickets per tenant; more than three tenants at once; real browser performance; the k6 scripts; run-to-run variance for the list, dashboard and mixed scenarios.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

22. Architecture decision records: summary

The full records, with context, consequences and alternatives, are in `ARCHITECTURE-DECISIONS.md`. "Not executed" means the code exists but needs Docker and never ran.

ADR	Decision	Status	Main trade-off
001	Modular core plus split-ready services, not full microservices	Implemented	Core scales as a unit; boundaries are by convention
002	Shared database with tenant_id, Hibernate filter and composite foreign keys	Implemented, tested on H2	Noisy neighbours; native SQL bypasses the filter
003	PostgreSQL row-level security as defence in depth	Written, not executed	Reviewed design, not a verified control; role must not bypass RLS
004	Tenant from the signed JWT claim, never from the client	Implemented, tested	Switching tenant means a new login; no tenant kill switch
005	Stateless HS256 JWT with a database-backed jti revocation list	Implemented, tested	Shared symmetric secret; no refresh or rotation
006	Transactional outbox for domain events	Implemented, tested on H2	Polling latency of about one second; FAILED rows have no replay endpoint
007	At-least-once delivery with idempotent consumers	Processor tested; broker not executed	Idempotency is per consumer; ordering is not guaranteed
008	Idempotency-Key for ticket creation	Implemented, tested	Applies only to ticket creation; comments are not idempotent
009	Optimistic locking for ticket edits	Implemented, tested	User resolves conflicts by hand; numbering may have gaps
010	Tenant-keyed cache-aside with graceful degradation	Caffeine tested; Redis tested with mocks	Per-node single-flight; invalidation covers service writes only
011	DocumentStore abstraction: Mongo for flexible documents, relational authoritative	Jpa and in-memory tested; Mongo not executed	Two stores to operate; Mongo isolation is application-enforced
012	Constrained workflow rules as data, not scripting	Implemented, tested	Less expressive; rules run inside the ticket transaction
013	Mock AI provider by default, human approval before customer text	Implemented, tested	Says nothing about real model quality; approval caps throughput
014	Runtime profiles local and docker	Implemented	The production-like path is the least verified path
015	Hand-written WebFlux gateway instead of Spring Cloud Gateway	Implemented, tested with stubs	No retries, breaker or distributed rate limit; the 504 path is untested
016	TanStack Query with tenant-first keys; token in sessionStorage	Implemented, unit tested	XSS exposure; 15 second staleness; polling per tab
017	Business-minute SLA clock; bulk updates without version bump	Implemented, tested	No leader lock; detection latency grows with open tickets
018	Two Flyway locations: portable SQL and PostgreSQL-only SQL	Portable executed on H2; PG set not executed	Portable SQL unproven on PostgreSQL
019	Bounded offset pagination, whitelisted sort	Implemented, tested	Offset cost grows with page; contract exposes page numbers

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ADR	Decision	Status	Main trade-off
020	Notification service with its own tables, Flyway history and retry and DLQ topology	Implemented; broker path not executed	Shared database is a hidden coupling; email is simulated

Decisions deliberately not taken: no service mesh or mTLS between services, no distributed tracing backend, no per-tenant quotas or kill switch, no search engine, and no file storage (attachments are metadata only).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

23. Engineering roadmap and technical debt

23.1 Honest gaps

> The most important gap: the production-like path has never run. PostgreSQL, Redis, RabbitMQ, MongoDB, RLS, the compose stack, the Dockerfiles, Prometheus and Grafana are code and configuration that was compiled or syntax-checked only. Until they run, they are reviewed designs, not verified controls.

#	Gap or debt	Impact	Evidence	Proposed fix	Effort
1	Docker profile never executed	RLS, Redis, RabbitMQ, Mongo, compose unverified; H2 differs from PostgreSQL	Validation record section 4	Run the stack on a Docker host; run the seven docker-tagged core tests and the notification broker test; fix findings	M
2	SLA percent-used was wall-clock based (FIXED)	At-risk and first escalation fired over a weekend after few business minutes; found by reading SlaCalculator.percentUsed; now computed in business minutes, paused time excluded	Chapter 11 worked example	Done; tests SlaCalculatorTest.closedHoursDoNotConsumeBudget_fridayEveningTicketIsNotAtRiskOnSunday, thresholdsAreCrossedByBusinessMinutesOnly, pausedBusinessTimeIsExcludedFromPercentUsed, firstResponsePercentUsesBusinessMinutesToo	done
3	Package cycles between tickets, sla, workflow, analytics	Harder to split or reason about modules	Figure 9	ArchUnit or Spring Modulith rules; extract interfaces	M
4	Comment creation is not idempotent	A client retry duplicates a comment	Runbook 5	Accept Idempotency-Key on comments	S
5	No CI/CD pipeline	No automated regression gate, scanning or image builds	Traceability matrix	Pipeline with build, tests, dependency and container scanning	M
6	Front-end and browser tests: recorded runs exist but are single local runs	Vitest 86/86, Karma 83/83, Playwright 17/17 and Angular smoke 25/25 were executed once on one laptop; no CI	Chapter 6	Run them in CI on every change	S
7	HS256 shared secret, no refresh, no rotation, no tenant kill switch, deactivation does not revoke	Large blast radius of a leak; slow offboarding	ADR-004, ADR-005	IdP or RS256 with JWKS; refresh tokens; user token version; tenant status check	L
8	Broker trust and shared broker user	Anyone who can publish to cx.events can target any tenant (recipient check mitigates part)	Security hardening API10	Per-service broker users and vhost permissions; publisher identity	M
9	Seeding on by default in docker profile; Swagger and Prometheus open	Demo accounts or metrics exposed if the stack is shared	Security hardening section 1	Default seeding off; protect or disable endpoints	S
10	Rate limits are per node; no per-tenant quotas	N replicas multiply the limit; noisy neighbours	ADR-015	Redis-backed or edge limiter; per-tenant quotas	M

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Gap or debt	Impact	Evidence	Proposed fix	Effort
11	SLA job: no leader lock, sequential tenants, 200 tickets per tenant per minute	Redundant work, delayed detection for large tenants	ADR-017	ShedLock or per-tenant partitioning; due-date index query	M
12	Dashboard folds up to 50 000 rows in memory (date buckets now use the tenant zone)	Cost grows with tenant size; the UTC-bucket defect is FIXED (AnalyticsTimeZoneTest)	AnalyticsService	Rollup table from outbox events	M
13	Search is LIKE plus keyword scoring bounded to 300 candidates	Poor at scale; contains-search scans	Chapters 12 and 21	Wire the PostgreSQL full-text path or add a search engine	M
14	Offset pagination; page unbounded	Deep pages scan and discard rows	H2 plan evidence	Keyset pagination (a contract change)	M
15	No retention for notif_processed_event and audit_event	Unbounded growth	Chapter 9	Purge and archive jobs	S
16	FAILED outbox rows have no replay endpoint	Manual SQL during incidents	ADR-006	Admin replay tool with audit	S
17	Attachments are metadata only; email and Slack are placeholders	Features incomplete	Chapter 3	Object storage with signed URLs and scanning; real channels	L
18	Real LLM never called; golden set proves rules only	Quality, cost, latency and leakage unmeasured	AI safety notes	Offline evaluation on labelled data; vendor review; keep approval gate	L
19	Transition buttons in the UI use the default table	A custom workflow table can give a 422	Chapter 6	Serve allowed targets from the API	S
20	Documentation drift between older documents (FIXED)	Older notes quoted 108 tests or 13 gateway tests and one runbook said notifications are not served by core locally; all were updated to the validation record (core 179, notification 46, gateway 14, dsa-lab 53)	Cross-reading the documents	Keep counts in one place (docs/core-service-validation.md)	done
21	No distributed tracing, alert rules or SLOs; p95 histograms off	Operations blind spots	Chapter 18	OpenTelemetry; alert rules; enable histograms	M

23.2 Suggested roadmap

Phase 0, prove it (one to two weeks). Run the compose stack and the seven docker-tagged tests; record results; fix what breaks (the first PostgreSQL run of `db/migration` is the biggest unknown). Add a CI pipeline that builds, tests and scans.

Phase 1, correctness (two weeks). Fix the SLA percent-used calculation with a weekend test; serve allowed transitions from the API; make comments idempotent; add the PostgreSQL EXPLAIN checks to the performance guide and settle the candidate index question.

Phase 2, security (three to four weeks). Move to an identity provider or RS256 with key identifiers; refresh tokens; user token version for instant deactivation; tenant status check; per-service broker users; seeding off by default; protect Swagger and Prometheus; image scanning and pinned digests.

Phase 3, scale (four to six weeks). Shared rate limits and per-tenant quotas; SLA job leader lock and due-date indexed query; rollup tables for analytics; PostgreSQL full-text or a search engine; keyset pagination; PgBouncer in transaction mode (compatible with `SET LOCAL`); retention jobs.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Phase 4, product depth. Attachments in object storage; real email and chat channels; a real model provider behind the existing seam, only after an offline evaluation and a data-handling review.

23.3 What is solid

The following are backed by executed tests on the local profile: the ticket state machine, optimistic locking, idempotent creation, tenant isolation at the application layer, the role matrix, JWT handling and revocation, business-hours arithmetic, the SLA job's idempotence, the outbox relay (claim, retry, lease, purge), the cache behaviour including a simulated Redis outage, the workflow validator and engine, the knowledge base visibility rules, the AI safety and resilience behaviours with the mock provider, and the gateway's routing, rate limit and error mapping against stubs.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

24. Interview questions and detailed answers about the project

How to use this chapter: the answers are written in the first person as a candidate who built the project, drawn from the repository question banks and rewritten to match the code. Each answer states what was built, what was proven by tests and what was not run. A strong interview answer names the trade-off and the limitation, so those are included on purpose.

24.1 Architecture and design

Q1. Why a modular core with two split-out services instead of microservices?

A ticket change touches the ticket, an activity row, an audit row and an outbox row, and I wanted that to be one ACID transaction, not a saga. So tickets, workflow, SLA, identity, knowledge base, AI and analytics are packages in one deployable. I split out only what has a different runtime profile: the gateway at the edge and the notification service, which is a natural asynchronous consumer with its own retry behaviour. The seams are real: `EventPublisher`, `CacheService` and `DocumentStore` are interfaces, and the notification module can run embedded as a library or standalone. The costs are that core scales as a unit with one connection pool, and package boundaries are conventions; I found package cycles between tickets, sla and workflow when I mapped the imports, and I would add ArchUnit or Spring Modulith rules next.

Q2. How do you isolate tenants, and which parts are actually verified?

The tenant comes only from the `tid` claim of a verified JWT and is held in a thread-local that is cleared in a `finally` block. From there, six layers repeat the check: a Hibernate filter enabled by a tenant-aware transaction manager, explicit tenant predicates because filters do not apply to `find-by-id`, a persist guard, composite foreign keys such as `(tenant_id, requester_id)`, tenant-prefixed cache keys and events, and optional PostgreSQL RLS. The application layers are verified on H2 by `TenantIsolationTest`, `TenancyTest` and `ArchitectureRulesTest`. RLS is written (`V1002`, `V1011`) but never executed because Docker was not available, so I describe it as a reviewed design, not a verified control. `PostgresRlsDockerIT` is the test that would prove it.

Q3. Why return 404 rather than 403 for another tenant's ticket?

A 403 confirms that the id exists, which lets an attacker enumerate other tenants' data. A 404 is indistinguishable from an id that never existed. The same rule applies inside a tenant: a customer asking for another customer's ticket gets 404. The cost is slightly harder debugging for legitimate support staff, which the correlation id and audit log compensate for.

Q4. How would RLS work with a connection pool, and what goes wrong?

Setting a session variable per connection is the classic mistake: a pooled connection returns to the pool still carrying the previous tenant. I use `select set_config('app.tenant_id', ?, true)` at the start of each transaction, where `true` means transaction-local (`SET LOCAL`), so the value vanishes at commit or rollback. The policy function `cx_current_tenant()` returns NULL if nothing is set, so an access outside a transaction matches zero rows and fails closed. Caveats I would raise: it works only because every database access runs in a transaction from the tenant-aware manager; superusers and `BYPASSRLS` roles ignore RLS, so the app needs a dedicated role; and PgBouncer must be in transaction mode. None of this ran here.

Q5. How would you take this to production? What are your first three priorities?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

First, run the unexecuted path: bring up PostgreSQL, Redis, RabbitMQ and MongoDB, run the seven docker-tagged core tests and the notification broker test, and fix what the first PostgreSQL run finds. Second, security: replace the shared HS256 secret with an identity provider or RS256 with key identifiers, add refresh tokens and immediate deactivation, turn seeding off, protect Swagger and Prometheus, and use per-service broker users. Third, operations: a CI pipeline with scanning, alert rules, p95 histograms and OpenTelemetry. After that come the scale items, such as keyset pagination, rollup analytics and a leader lock for the SLA job.

24.2 Data, concurrency and APIs

Q6. How does optimistic locking surface in the API and the UI?

The ticket has a `version` column. Update, assign and transition requests carry the version they were based on. If it does not match, the service answers 409 `TICKET_VERSION_CONFLICT`, and Hibernate's `@Version` also guards a concurrent flush. The UI shows a conflict notice with "Reload latest". A test sends two parallel updates with the same version and expects exactly one 200 and one 409. Background work should not cause conflicts, so the SLA job uses bulk updates that do not bump the version. The user has to resolve conflicts by hand; I did not build an automatic merge.

Q7. How does Idempotency-Key work, and what races did you handle?

The key is scoped to tenant and user, validated against `[A-Za-z0-9._:-]{1,100}`, and stored with a SHA-256 hash of the request body and the original response. The same key and body replays the stored 201 with `Idempotent-Replay: true`; the same key with a different body gives 409 `DUPLICATE_SUBMISSION`. The key row is inserted in the same transaction as the ticket, so a failed creation never consumes the key. Two concurrent requests race on the unique constraint; the loser rolls back everything and replays the winner. A test with four concurrent same-key requests creates exactly one ticket. Limits: only ticket creation is idempotent (a retried comment duplicates), keys are purged after 24 hours, and a replay returns the response as it was at creation time.

Q8. Tell me about the ticket-number contention you found.

Under load, ticket creation had a heavy tail: p99 about 1 second and a maximum of several seconds at concurrency 10. I ruled out GC (max pause 10 ms) and the connection pool (no timeouts, 0.1 second maximum acquire), but one transaction had held a connection for 6 seconds, which pointed at a lock. The per-tenant counter row was locked with `SELECT ... FOR UPDATE` until the whole create transaction committed. A differential test confirmed it: spreading the same ten clients across three tenants gave p99 of about 45 to 87 milliseconds. I changed numbering to one atomic `UPDATE next_number = next_number + 1` in its own tiny transaction before the business transaction. Single-tenant p99 at concurrency 10 fell to about 81 to 84 milliseconds. The price is that numbers can have gaps, which I documented and tested. All of this is on H2 on one laptop, so the numbers are indicative, not a benchmark.

Q9. How do you prevent N+1 queries, and how do you know it works?

Entities store ids as plain columns rather than lazy associations, and `TicketMapper.toDtos` resolves requester, assignee, category and attachment data with one batched `IN` query per kind. `NPlusOneGuardTest` counts the JDBC statements Hibernate prepares: 6 for a page of 2 and 6 for a page of 30, asserted to be at most 8. A naive mapper would issue about `1 + 3N` statements. In production I would compare request counts with `pg_stat_statements` calls per endpoint, since a ratio that grows with page size is the signature of a regression.

Q10. Why offset pagination, and when would you change it?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Offset gives random page access and a simple contract, and I bounded it: size 1 to 100, whitelisted sort fields, an id tie-break. The weakness is that deep pages read and discard rows. On H2 with 50 000 tickets in one tenant, an offset of 40 000 examined 50 001 index entries, while a keyset form examined 10 001. I would switch to keyset pagination for deep lists, which is a contract change, and cap or estimate the page count. I also noted that `page` itself is not capped today.

Q11. Why keep both PostgreSQL and a document store? Would you choose MongoDB again?

Authoritative data (articles, suggestion status and decision) must be transactional, constrained and under RLS, so it is relational. Derived, write-once, schema-flexible data, namely article version snapshots and AI metadata, is accessed by key or prefix, so it goes behind a `DocumentStore` interface. The default implementation is a JPA table that joins the caller's transaction; the MongoDB implementation exists for the docker profile but is not in the PostgreSQL transaction, which is why it holds only non-authoritative data. Honestly, PostgreSQL `jsonb` would meet these needs more simply, and I would choose it unless a team already operated MongoDB. The Mongo code and its indexes never ran here.

24.3 Messaging and caching

Q12. Why a transactional outbox, and how does the relay avoid duplicates?

Publishing to the broker inside the database transaction is a dual write that cannot be atomic, and publishing after commit can lose the event on a crash. The outbox makes the event a row in the same transaction; `OutboxService.publish` is `Propagation.MANDATORY`, so it cannot be called outside one. The relay polls every second, claims each due row with a compare-and-set `UPDATE` that sets a 60 second lease, publishes outside any transaction and marks the row SENT. It cannot prevent duplicates entirely: a crash after publishing but before marking leaves the row PROCESSING, and after the lease expires another relay re-claims it. So delivery is at-least-once, and consumers de-duplicate on `eventId`. That claim is tested with six concurrent claimers where exactly one wins.

Q13. Exactly-once, at-least-once or at-most-once?

At-least-once, with idempotent consumers giving an effectively-once result. Exactly-once across a database and a broker is not achievable cheaply. The notification processor inserts the `eventId` into a table with a primary key first; a duplicate is a no-op. Notification and delivery rows have unique keys on event, user and channel, so a retry resends only channels that have not yet succeeded. Ordering is not guaranteed across retries, so payloads are self-contained. The weakness is that idempotency is a per-consumer responsibility; nothing in the platform enforces it for a new consumer.

Q14. Walk me through the retry and dead-letter design.

On a retryable failure the consumer republishes the message persistently to a TTL retry queue with an attempt header, waits for the publisher confirm, then acknowledges the original, so a message is never lost between steps; if the republish is not confirmed, the original is requeued. When the TTL expires the broker dead-letters the message back into the main queue. After five attempts, or immediately for a poison message such as a missing `eventId` or a payload tenant that differs from the envelope, the consumer stores status DEAD and rejects without requeue, so the broker routes it through the dead-letter exchange to the DLQ. I tested this logic against a fake channel; the real broker topology never ran.

Q15. How does your caching work and what happens if Redis dies?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`CacheService.getOrLoad` is tenant-keyed cache-aside with a TTL and per-node single-flight, so twelve concurrent misses produce one load. Keys are `t:{tenantId}:...` built in one place. Writes evict after the transaction commits, so a rollback never evicts. If Redis fails, the loader runs, a fallback counter increments and a five second in-process breaker stops contacting Redis; requests are slower, not failed. That behaviour is tested with a mocked template, not a real Redis. Measured locally, a dashboard hit took 3.23 ms against 21.73 ms for a miss. The trade-offs are write-rate-dependent hit ratios, per-node single-flight and invalidation that covers only writes through the service.

24.4 SLA, workflow and the lifecycle

Q16. How do you compute an SLA due date with business hours, time zones and pauses?

`BusinessCalendar` takes the tenant's zone, daily window and working days. To add business minutes it walks forward day by day, skips non-working days, moves to opening time if before open, and consumes whichever is smaller of the remaining minutes and the minutes until close. Windows are built per local date with `ZonedDateTime`, so daylight-saving changes are handled; `BusinessCalendarTest` covers spring and fall cases. PENDING and ON_HOLD pause the clock; on resume both due dates are pushed out by the business minutes spent paused. Example: a HIGH ticket created Friday 15:30 with 480 resolution minutes uses 90 on Friday and 390 on Monday and is due Monday 15:30.

Q17. What was wrong with your SLA percentage calculation, and what did you do?

When I wrote the worked example I noticed the job computed percent used from wall-clock time between creation and the due date, not from business minutes consumed. For the Friday example the wall-clock span is 72 hours, so 75 percent was reached on Sunday evening when only 90 of 480 business minutes had elapsed, and AT_RISK and the first escalation fired over the weekend. The due dates themselves were correct. I fixed it: percent used is now `(target - businessMinutesBetween(now, due)) / target` in the tenant's calendar, so closed hours consume nothing and paused time drops out because the due date was shifted on resume. Four tests pin it, including the Friday 15:30 to Sunday 21:30 case (18.75 percent, ON_TRACK) and a pause case. The lesson I take from it: a derived metric must use the same unit as the thing it is compared with, and a worked example is a cheap way to find that kind of defect.

Q18. How do you make the SLA job safe with several nodes?

The job is idempotent rather than locked. The escalation level only increases, `breach_notified` flips once, and updates are bulk updates, so two nodes running at once publish each event at most once per level and never bump the user-facing version. It works per tenant, binds the tenant itself, examines at most 200 tickets per tenant ordered by least recently checked, and isolates failures so one tenant's error does not stop the others. What is missing is a leader lock (ShedLock) to avoid redundant work, and concurrency across tenants. Detection latency for a tenant with more than 200 open tickets is roughly `ceil(open / 200)` minutes.

Q19. Why is the workflow engine not scriptable?

Tenant-controlled configuration that allows expressions or scripts is a remote-code-execution risk. A workflow is versioned JSON with 10 fields, six operators and five actions; the validator type-checks values, rejects expression markers such as `${` and `#{`, rejects unknown parameters and caps rules, conditions, actions and transitions. The engine is a plain switch interpreter that runs once per trigger and stops at 20 actions, so loops are impossible. If the stored definition is corrupted, ticket handling falls back to a built-in default and a counter increments; a test covers it. The trade-offs are limited expressiveness (all conditions AND together, no time-based triggers) and that rules run inside the ticket transaction.

Q20. How does least-loaded assignment work and what are its races?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

The action loads up to 100 active users with the role, counts each one's open tickets in one grouped query, and picks the lowest count, breaking ties by user id for determinism. It never overrides an existing assignee. Two concurrent creates may pick the same agent, which only affects balance and is acceptable; a hard cap would need a row lock or an atomic counter. The assignment runs inside the create transaction, so it adds latency bounded by the candidate limit.

24.5 AI, security and operations

Q21. How do you keep AI output safe, and is it real AI?

The default provider is a deterministic keyword rule engine, not machine learning, and I label it so in the API (`MOCK_RULE_BASED`) and in the UI. Safety comes from architecture. Ticket text is treated as data: it is normalised, instruction-like sentences are dropped, and secrets and personal data patterns are redacted before any provider sees it. The provider is a pure function with no access to the database or the ticket write API, and its output is validated against server-side candidate lists. A suggestion is an inert `PENDING_REVIEW` row; only an agent's approval posts exactly one PUBLIC comment, via a compare-and-set update in one transaction with audit and an outbox event. Regex defences miss paraphrased injections, so the human gate, not the regexes, is the real protection.

Q22. What happens when the AI provider is slow or down?

The analyze call degrades instead of failing. A per-tenant limit of 30 analyses per minute returns 429; otherwise the call goes to a bounded pool of 4 workers with 16 queued, with a 3 second timeout per attempt, up to 2 retries and a 10 second total deadline, behind a circuit breaker that opens after 5 consecutive failed analyses for 30 seconds. Every failure returns HTTP 200 with `degraded: true`, a fixed fallback message and no reply text, plus an `AI_ANALYSIS_DEGRADED` audit row with a reason. The provider call is outside any database transaction so a slow provider cannot hold a connection. Tests with a fault injector show exactly 3 attempts for FAIL and a degraded answer in under 4 seconds for TIMEOUT.

Q23. How would you evaluate the AI feature properly?

What exists is a golden set of 12 tickets by 3 checks, 36 of 36 matching; it is a regression test of fixed rules and proves nothing about accuracy. A real evaluation needs labelled, anonymised tickets per tenant, precision and recall per category, a priority confusion matrix, calibration of the confidence value, a red-team set of paraphrased injections, and online metrics: acceptance rate (already on the dashboard), edit rate and degraded rate. Confidence in this code is the number of rules that fired, so it must never be used to auto-approve anything, and nothing does.

Q24. Describe your JWT approach, its weaknesses and what you would change.

Tokens are HS256 with `sub`, `tid`, `roles`, `email`, `jti`, `iss` and a 60 minute expiry. Logout writes the `jti` into a shared table, cached so a valid token costs about one lookup per 3 seconds per node; the cost is up to 3 seconds of delayed revocation on other nodes. Weaknesses I would name: one symmetric secret is shared by core, gateway and notification service, so any leak lets an attacker forge tokens for any tenant; there is no refresh token or key rotation; deactivating a user does not revoke existing tokens; and the SPA keeps the token in `sessionStorage`, exposed to XSS. In production I would use an identity provider or RS256 with JWKS, short access tokens with rotating refresh tokens, and a tenant and user status check.

Q25. A customer says they can see another organisation's ticket. What do you do?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Treat it as severity 1. Get the correlation id and the entity id from the report and reproduce with that user's token; the expected answer is 404. Run the guard tests. Search for bypass paths: native SQL, `JdbcTemplate`, `findById(` without a tenant check, a cache key built without the tenant prefix, a background thread with a stale tenant context, or notification queries without tenant and user. On PostgreSQL, check that RLS is enabled and forced and that the application role is not a superuser or `BYPASSRLS`. Contain by revoking tokens or rotating the secret, then fix the predicate and add a regression test next to `TenantIsolationTest`. I would be clear that the PostgreSQL checks are documented procedures here, not ones I executed.

Q26. What did you measure, and what would you not claim?

I measured on one laptop, on H2 with Caffeine, without a gateway or network: list, dashboard, create and mixed load, the create tail before and after the numbering fix, cache hit versus miss, the N+1 statement counts and H2 plan row counts. I would not claim any PostgreSQL, Redis, RabbitMQ or MongoDB number, any production throughput or service-level objective, gateway overhead, end-to-end event latency, or real-browser performance. H2 in PostgreSQL mode is not PostgreSQL: its planner, locking and null ordering differ, and a null-ordering difference already caused one real defect. The honest framing is that local numbers locate bottlenecks and validate fixes, and PostgreSQL `EXPLAIN (ANALYZE, BUFFERS)` runs are the next step.

Q27. How does the front end avoid cross-tenant cache bleed, duplicate submits and lost updates?

Every TanStack Query key starts with `['t', tenantId, ...]` and the cache is cleared on logout, so a re-login into another tenant cannot show the previous tenant's data. The create-ticket form generates one `Idempotency-Key` per mount, reuses it on retry, disables the button while pending and blocks double submits with a ref. A 409 on edit, assign or transition shows a conflict notice with "Reload latest". The route guards mirror the role matrix for usability only; the server remains the authority. These behaviours are covered by Vitest suites that exist in the repository, but no recorded run of those suites exists, so I would run and record them before claiming they pass.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.